Martin Bergstø

MLguide: Decision Support for Machine Learning Method Selection in Product Development and Production

Combining Ontology-Based Knowledge, Semantic Literature Retrieval, and Implementation Support

Master's thesis in Engineering and ICT
Supervisor: Andrei Lobov
June 2026

Norwegian University of Science and Technology
Faculty of Engineering
Department of Mechanical and Industrial Engineering

NTNU

ABSTRACT

Write an abstract/summary of your thesis, and state your main findings here.

A summary should be included in both English and any second language, if this is applicable, regardless if the thesis is written in English or in your preferred language. These should be on separate pages, the English version first.

PREFACE

This master's thesis was written as part of the course TMM4935 "Industrial ICT" at the Department of Mechanical and Industrial Engineering at the Norwegian University of Science and Technology (NTNU). The thesis accounts for 30 credits and was completed in the spring of 2026.

The thesis builds on my specialisation project, *Supporting Method Selection for Machine Learning in Product Development and Production: A Multi-Dimensional Classification of Machine Learning Research* (Bergstø 2026), completed in January 2026. The specialisation project performed an analysis of machine learning research that the present work draws on.

I would like to thank Professor Andrei Lobov from the Department of Mechanical and Industrial Engineering, who supervised this work. The weekly meetings and continued discussions with Professor Lobov gave me clear direction and steady support while I completed the thesis.

I would also like to thank the participants who took part in the user testing. Their time and feedback were valuable to this work.

CONTENTS

LIST OF FIGURES

LIST OF TABLES

ABBREVIATIONS

List of all abbreviations in alphabetic order:

- **AC** Actor-Critic
- **AI** Artificial Intelligence
- **AIoDP** AI-on-Demand Platform
- **API** Application Programming Interface
- **AutoML** Automated Machine Learning
- **BERT** Bidirectional Encoder Representations from Transformers
- **BM25** Best Match 25
- **CNN** Convolutional Neural Network
- **DBSCAN** Density-Based Spatial Clustering of Applications with Noise
- **DOI** Digital Object Identifier
- **DQN** Deep Q-Network
- **DRL** Deep Reinforcement Learning
- **DSR** Design Science Research
- **EID** Electronic Identifier
- **GMM** Gaussian Mixture Model
- **HTTP** Hypertext Transfer Protocol
- **IR** Information Retrieval
- **IRI** Internationalized Resource Identifier
- **ITO** Intelligence Task Ontology
- **LSTM** Long Short-Term Memory
- **ML** Machine Learning
- **MLP** Multilayer Perceptron
- **NTNU** Norwegian University of Science and Technology

- **OMA-ML** Ontology-Based Meta AutoML
- **OWL** Web Ontology Language
- **PCA** Principal Component Analysis
- **PLC** Product Lifecycle
- **PPO** Proximal Policy Optimization
- **RDF** Resource Description Framework
- **RDFS** RDF Schema
- **RL** Reinforcement Learning
- **RNN** Recurrent Neural Network
- **SAC** Soft Actor-Critic
- **SBERT** Sentence-BERT
- **SKOS** Simple Knowledge Organization System
- **SME** Small and Medium-sized Enterprise
- **SPARQL** SPARQL Protocol and RDF Query Language
- **SQL** Structured Query Language
- **SVM** Support Vector Machine
- **tf-idf** Term Frequency-Inverse Document Frequency
- **UML** Unified Modeling Language
- **W3C** World Wide Web Consortium
- **YAML** YAML Ain't Markup Language

CHAPTER

ONE

INTRODUCTION

This chapter introduces the purpose and overall direction of the thesis. It presents the motivation for the study and describes the main problem that is addressed. A hypothesis is then stated and broken down into research questions that guide the work. Finally, the chapter provides an overview of the structure of the thesis.

1.1 Motivation

Machine learning (ML) is widely applied in product development and production, where it is used for tasks such as optimisation, process monitoring, quality control, and fault detection (Wuest et al. 2016; Dogan and Birant 2021). As a result, machine learning plays an important role in improving efficiency and supporting decision-making in engineering and manufacturing (Rai et al. 2021).

At the same time, the field is large and diverse. Many different algorithms and methods are available, each with its own strengths and weaknesses (Wuest et al. 2016). No single method performs best for all problems, and performance depends on the specific application and data (Nti et al. 2022; I. Lee and Shin 2020; Dogan and Birant 2021). A method that performs well on one criterion may perform worse on another, so strengths in one respect are often offset by weaknesses in another (Ali, S. Lee, and Chung 2017). For engineers and practitioners, the large number of options can act as a barrier that limits the effective use of machine learning in practice (Nti et al. 2022).

Selecting an appropriate method is not straightforward. It requires balancing competing factors such as accuracy and interpretability (I. Lee and Shin 2020). If the selection is not done carefully, the resulting model may therefore fail to provide useful results (Ali, S. Lee, and Chung 2017). In practice, simpler and more interpretable models are often preferred, even when more complex models could offer higher accuracy (Paleyes, Urma, and Lawrence 2022).

Method selection is further complicated by a lack of knowledge and experience when adopting machine learning (Sonntag, Luttmer, et al. 2023). There is also

pressure to adopt machine learning without a clear plan, which increases the risk of unsuccessful implementations (Plathottam et al. 2023). These challenges highlight the need for more structured approaches that can support method selection and make machine learning more accessible.

1.2 Problem Description

Despite the growing adoption of machine learning, practitioners often lack adequate support when selecting methods for their specific problems. There is no guidance or standard for developing machine learning solutions from ideation to deployment, and it remains difficult for non-experts to begin developing solutions for their own problems (Tingting Chen et al. 2023). Identifying a suitable algorithm from the large number of available options requires a certain amount of experience, and the procedure is often based on trial and error, which is laborious (Sonntag, Luttmer, et al. 2023). Many practitioners lack this experience, and no solution currently allows for an informed selection of methods based on a well-defined problem within the product development process (Sonntag, Luttmer, et al. 2023).

Existing frameworks tend to use accuracy or prediction speed as the primary selection criteria, which are often too general and can lead to unsuitable methods being chosen (Sala et al. 2018). Research also focuses primarily on developing and evaluating models, while less attention is given to what conditions lead to selecting one method over another in a specific context (Arena et al. 2024).

A further barrier is interpretability. Many machine learning models act as black boxes that do not make clear why they produce a given output, and this is a key obstacle to their adoption in practice (Presciuttini et al. 2024). The relevant knowledge is also distributed across a large and fragmented body of literature, spread across many journals and research communities (Nti et al. 2022). This breadth creates extensive knowledge on the topic, but can also cause disorientation when selecting an approach (Sala et al. 2018).

Two needs follow from the challenges described above. The first is a structured representation of machine learning methods and their characteristics, so that method selection can be guided by more than accuracy alone and the basis for a recommendation can be made explicit. The second is a way to draw on the knowledge held in the scientific literature, so that recommendations can be grounded in prior research rather than in experience alone. Because this knowledge is spread across many sources, a way of connecting a given problem to the research relevant to it is needed, without the manual search that the fragmented literature makes impractical.

This thesis investigates whether an ontology-based representation of machine learning knowledge, combined with semantic retrieval from scientific literature, can provide such support. The aim is to offer interpretable method recommendations that are grounded in prior research, alongside guidance for early-stage implementation.

1.3 Hypothesis and Research Questions

This thesis approaches the challenges outlined above by proposing a single system that addresses them together. Whether such a system can both recommend suitable machine learning methods and help put them into practice is an empirical question, and the proposed approach is therefore stated as a hypothesis to be tested:

> A decision-support system that combines ontology-based knowledge representation, semantic similarity between problem descriptions and scientific literature, and structured implementation support can provide relevant and justified machine learning method recommendations for users with limited machine learning expertise.

To investigate this hypothesis, a web-based decision-support system, called MLguide, was developed. MLguide combines an ontology-based representation of machine learning knowledge with semantic retrieval from scientific literature, and provides support for the first steps of implementation once a method has been chosen.

The hypothesis is examined through the following research questions:

- How can ontology-based knowledge representation support the selection of machine learning methods?
- How can semantic retrieval from scientific literature support and justify the recommended methods?
- To what extent can such a system support users with limited machine learning expertise in the early stages of implementation?

1.4 Thesis Structure

The thesis is structured as follows:

- Chapter 1 introduces the motivation, problem description, and research questions.
- Chapter 2 presents background and related work.
- Chapter 3 presents the methodology and system overview.
- Chapter 4 presents the system implementation.
- Chapter 5 presents the results.
- Chapter 6 discusses the findings and limitations.
- Chapter 7 concludes the thesis and suggests future work.

CHAPTER

TWO

BACKGROUND AND RELATED WORK

This chapter introduces the theoretical and technical background for the work presented in this thesis. It covers machine learning methods and their application in product development and production, ontologies and knowledge representation, and semantic retrieval from scientific literature. The chapter ends with a review of related work and existing tools.

2.1 Machine Learning

This section introduces the machine learning concepts that the rest of the thesis builds on. It first describes the main learning paradigms and tasks associated with them, then presents a selection of common methods with emphasis on attributes that distinguish them.

2.1.1 Machine Learning Paradigms and Tasks

Machine learning refers to algorithms that improve their performance at some task through experience (Mitchell 1997). Such algorithms can be broadly categorized according to the type of supervision they receive during that experience, also called training. There are three main paradigms commonly distinguished in the literature: supervised learning, unsupervised learning, and reinforcement learning (RL) (Bishop 2006; Goodfellow, Bengio, and Courville 2016, Chapter 5; Sarker 2021). The following sections outline each paradigm and introduce some common tasks.

In supervised learning, the training data consists of input vectors paired with corresponding target values. The two most common supervised tasks are classification, where the goal is to assign an input to one of a finite number of discrete categories, and regression, where the goal is to predict a continuous numerical value (Bishop 2006; Goodfellow, Bengio, and Courville 2016, Chapter 5).

In unsupervised learning, no target values are provided. The algorithm must instead identify structure in the data directly (Bishop 2006). Common tasks include

clustering, dimensionality reduction, and anomaly detection (Sarker 2021; Goodfellow, Bengio, and Courville 2016, Chapter 5). Clustering groups data points such that objects within the same group are more similar to each other than to those in other groups (Sarker 2021). Dimensionality reduction simplifies data by reducing the number of features, which leads to better human interpretations and lower computational costs (Sarker 2021). Anomaly detection identifies observations that deviate significantly from the expected distribution, and is used in applications such as fault monitoring (Goodfellow, Bengio, and Courville 2016, Chapter 5). The boundary between supervised and unsupervised learning is not always well-defined, and many methods can be applied across both types of problems (Goodfellow, Bengio, and Courville 2016, Chapter 5).

Reinforcement learning differs fundamentally from the other paradigms. Rather than learning from a fixed dataset, an agent interacts with an environment and learns through trial and error, receiving rewards or penalties based on its actions (Bishop 2006; Sarker 2021). Within reinforcement learning, methods can be broadly divided according to what they learn. In value-based methods, a value function is learned from which the policy is derived either explicitly or implicitly, whereas in policy-based methods the policy is learned directly without the need for a value function (Sewak 2019; Richard S. Sutton and Barto 2015, Chapter 11). A third class, actor-critic methods, combines both approaches by maintaining a separate policy structure and value function simultaneously (Richard S. Sutton and Barto 2015, Chapter 11).

Figure 2.1.1 gives an overview of the three paradigms and common tasks associated with each.

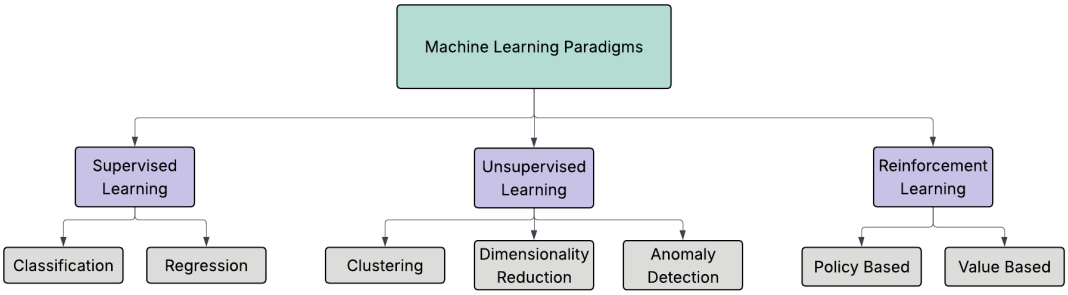

Figure 2.1.1: Overview of machine learning paradigms with common tasks.

2.1.2 Machine Learning Methods

The following sections introduce a selection of common and established machine learning methods and place them within the paradigms and tasks described above. The aim is not to provide a comprehensive description of each method or to cover underlying mathematical details, but to give an overview of common method types and what fundamentally distinguishes them.

Supervised methods

Tree-based methods learn by recursively partitioning the input space according to feature values. A decision tree splits data into regions by asking a sequence of questions about the input features, assigning each region a predicted output. They can be used for both classification and regression tasks and require little data preparation. (Geron 2019, Chapter 6; Sarker 2021). Their decisions are intuitive and easy to interpret, as the classification rules they produce can even be applied manually (Geron 2019, Chapter 6). Random forests extend this by training many trees on different subsets of the data and aggregating their predictions, which makes them more robust and generally improves predictive accuracy (Breiman 2001). However, combining many trees makes the model harder to interpret than a single decision tree (Geron 2019, Chapter 6). Gradient boosting methods such as XGBoost build trees sequentially to improve on the predictions of previous trees, and are known for strong generalisation performance on large datasets (Tianqi Chen and Guestrin 2016; Sarker 2021).

Linear and polynomial regression predict a continuous output and are among the most widely used methods for regression tasks (Sarker 2021). Regularised variants constrain the model weights to improve generalisation. Ridge regression keeps weights small but never eliminates them entirely, while Lasso regression tends to set some weights exactly to zero, which effectively performs feature selection and produces a sparse model (Tibshirani 1996; Geron 2019). Logistic regression estimates the probability of class membership using a linear combination of input features, and is commonly used for classification tasks, particularly when a simple and interpretable model is preferred (Geron 2019, Chapter 4; Sarker 2021).

Support vector machines (SVMs) find a decision boundary that maximises the margin between classes (Cortes and Vapnik 1995). By applying kernel functions, SVMs can model non-linear decision boundaries and are effective in high-dimensional spaces. They can be used for both classification and regression tasks (Geron 2019, Chapter 5; Sarker 2021).

K-nearest neighbours classifies a new data point based on the majority class among its closest neighbours in the training data, using a distance measure such as Euclidean distance. The method requires no explicit training step but can be computationally expensive at prediction time for large datasets (Cunningham and Delany 2021; Sarker 2021).

Unsupervised methods

Clustering algorithms group data points based on similarity without the use of labels. K-means partitions data into a fixed number of clusters by assigning each instance to the nearest centroid and updating the centroids iteratively until convergence (Geron 2019, Chapter 9). The number of clusters must be specified in advance, and the algorithm is sensitive to the initial centroid placement (Geron 2019, Chapter 9). Density-Based Spatial Clustering of Applications with Noise (DBSCAN) takes a different approach by defining clusters as dense regions in the data space, which allows it to find clusters of arbitrary shape and identify outliers as noise without requiring the number of clusters to be specified in advance (Ester

et al. 1996; Sarker 2021).

For dimensionality reduction, principal component analysis (PCA) identifies the axes of greatest variance in the data and projects it onto a lower-dimensional space defined by these axes, called principal components (Hotelling 1932; Geron 2019). This reduces computational cost and can aid visualisation while preserving as much of the original variance as possible (Geron 2019, Chapter 8; Sarker 2021).

Anomaly detection methods identify observations that deviate significantly from the expected distribution. Isolation Forest isolates anomalies by randomly partitioning the data, exploiting the fact that anomalies are few and structurally different from normal observations (Liu, Ting, and Zhou 2008). One-class SVM, a variant of the support vector machine, learns a boundary around normal data and flags observations outside this boundary as anomalies (Geron 2019, Chapter 9).

Reinforcement learning methods

Within reinforcement learning, methods differ in what they learn. Value-based methods such as Q-learning estimate the expected return for each action in a given state, and the policy is derived by selecting the action with the highest value (Watkins and Dayan 1992; Zhang and Yu 2020). This approach is oriented towards deterministic policies, and small changes in the estimated values can cause sudden changes in the selected action (Richard S Sutton et al. 1999). Policy-based methods such as policy gradient algorithms instead represent the policy directly and update it along the gradient of the expected reward (Richard S Sutton et al. 1999). This allows them to represent stochastic policies and makes them better suited for continuous or high-dimensional action spaces (Zhang and Yu 2020). Actor-critic methods combine the two by learning a value function and a policy at the same time, using the value function to guide the policy updates. The basic form is the actor-critic (AC) algorithm, with later variants such as soft actor-critic (SAC) (Zhang and Yu 2020).

Figure 2.1.2 (on the following page) extends the paradigm and task overview in Figure 2.1.1 with a further level, organising common machine learning methods by paradigm and task. It includes the methods described in this section, as well as additional commonly used methods not covered in detail here. The overview combines taxonomies and categorisations from the literature, drawing in particular on Sarker (2021), Geron (2019), and Zhang and Yu (2020).

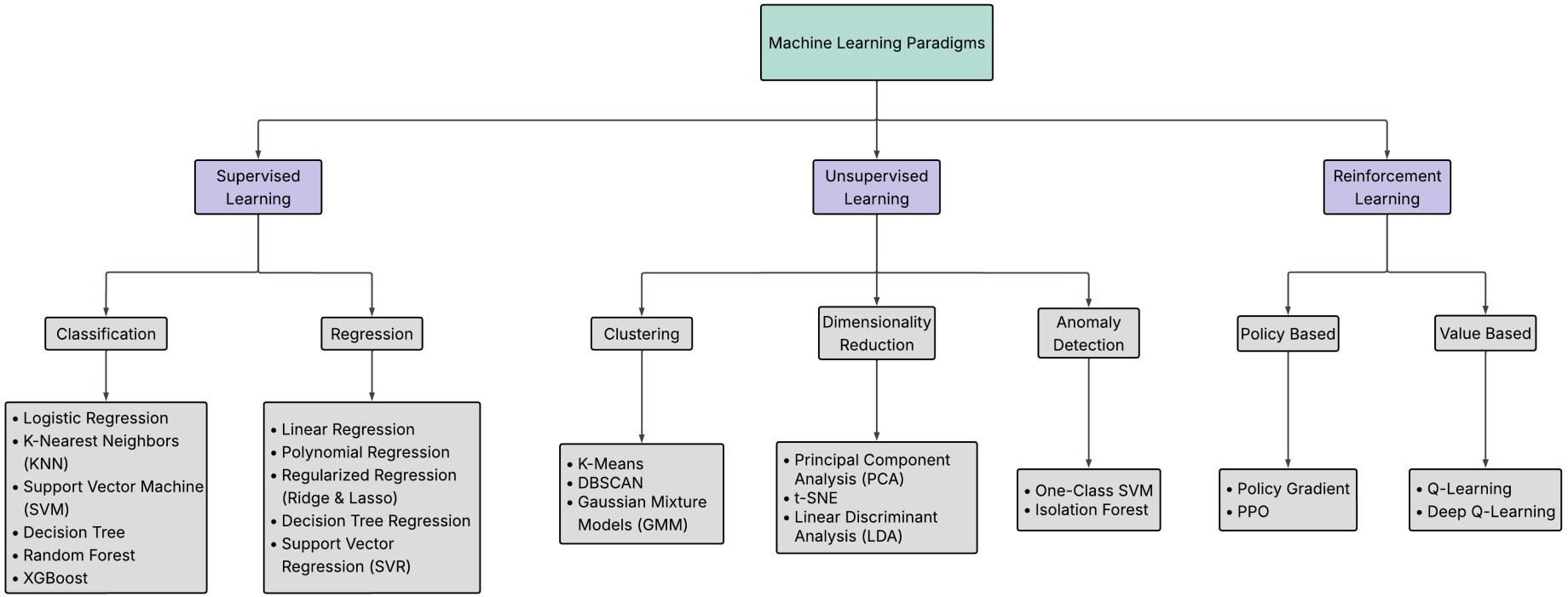

Figure 2.1.2: Overview of common machine learning methods categorised by paradigms and tasks, based on Sarker (2021), Geron (2019), and Zhang and Yu (2020).

2.1.3 Neural Networks and Deep Learning

Neural networks and deep learning are often treated as a separate category in the machine learning literature. The following sections give a brief introduction to some of the most common neural network architectures and their typical use cases.

Neural networks are machine learning models inspired by the structure of the human brain, consisting of processing units organised in input, hidden, and output layers (Shrestha and Mahmood 2019). They can be applied to a wide range of problems, including classification, regression, clustering, dimensionality reduction, computer vision, and natural language processing (Shrestha and Mahmood 2019). Deep learning refers to neural networks with many layers. By stacking multiple layers, the network learns increasingly abstract representations of the input data, where lower layers detect simple patterns and higher layers combine these into more complex concepts (LeCun, Bengio, and Hinton 2015). This makes deep learning particularly well suited for complex, high-dimensional data such as images, audio, and text. (LeCun, Bengio, and Hinton 2015; Alzubaidi et al. 2021). A practical limitation is that deep learning models generally require large amounts of training data to perform well (Alzubaidi et al. 2021).

Feedforward networks and multilayer perceptrons

The simplest deep learning architecture is the multilayer perceptron (MLP), also known as the feedforward neural network. In an MLP, information flows in one direction from input to output through one or more hidden layers, without any feedback connections (Shrestha and Mahmood 2019; Sarker 2021). Each node in one layer connects to each node in the following layer, and the network is trained using backpropagation to adjust the weights (LeCun, Bengio, and Hinton 2015).

Convolutional neural networks

Convolutional neural networks (CNNs) were originally developed for image recognition, where the spatial structure of the input is important (LeCun, Bengio, and Hinton 2015; Lecun et al. 1998). Rather than connecting every neuron to all neurons in the previous layer, CNNs use local connections and shared weights, which reduces the number of parameters and makes the network faster to train (Shrestha and Mahmood 2019). A typical CNN consists of convolutional layers, pooling layers, and fully connected layers. The convolutional layers progressively extract higher-level features from the input, while the pooling layers reduce the spatial size of the representations (Pouyanfar et al. 2018; Sarker 2021). CNNs have achieved strong results in image recognition and computer vision, and are also applied to other data types such as audio and text (LeCun, Bengio, and Hinton 2015; Pouyanfar et al. 2018).

Recurrent neural networks and long short-term memory

Recurrent neural networks (RNNs) differ from feedforward networks in that the processing units form a cycle, allowing the network to maintain a state based on previous inputs (Shrestha and Mahmood 2019). This makes RNNs well suited

for sequential data such as time series, speech, and text (LeCun, Bengio, and Hinton 2015; Pouyanfar et al. 2018). A known limitation of standard RNNs is the vanishing gradient problem, where information about earlier inputs is lost during training as the network processes longer sequences (Pouyanfar et al. 2018; Shrestha and Mahmood 2019).

Long short-term memory networks (LSTMs) address this by introducing memory cells and gating mechanisms that control what information is stored, read, and written (Hochreiter and Schmidhuber 1997; Shrestha and Mahmood 2019). This allows LSTMs to learn from sequences with long-term dependencies, which makes them commonly used for time series analysis, speech recognition, and natural language processing (LeCun, Bengio, and Hinton 2015; Sarker 2021).

Autoencoders

Autoencoders are neural networks trained to compress input data into a lower-dimensional representation and then reconstruct the original input from this representation (Shrestha and Mahmood 2019). Because they learn to reconstruct the input rather than predict a separate target, autoencoders are considered unsupervised models (Shrestha and Mahmood 2019). They are commonly used for dimensionality reduction and anomaly detection, where observations that are difficult to reconstruct may indicate anomalies (Sarker 2021; Shrestha and Mahmood 2019).

Figure 2.1.3 summarises the architectures described above.

Architecture	Description	Training Type	Common Applications
CNN (Convolutional Neural Network)	Uses convolutional layers to automatically learn spatial hierarchies of features from input data, especially images.	**Supervised**	• Image classification • Object detection • Image segmentation
RNN (Recurrent Neural Network)	Designed to handle sequential data by maintaining a hidden state that captures dependencies across time steps.	**Supervised**	• Sequence modeling • Natural language processing • Time series forecasting
LSTM (Long Short-Term Memory)	A type of RNN that uses gates and cell states to better capture long-term dependencies.	**Supervised**	• Natural language processing • Speech recognition • Time series prediction
Autoencoder (AE)	An unsupervised network that learns to encode input data into a lower-dimensional representation and then reconstruct it.	**Unsupervised**	• Dimensionality reduction • Representation learning • Anomaly detection

Figure 2.1.3: Summary and comparison of common neural network architectures, based on Sarker (2021), Shrestha and Mahmood (2019), Pouyanfar et al. (2018), and LeCun, Bengio, and Hinton (2015).

Deep reinforcement learning

Deep reinforcement learning (DRL) combines deep learning with reinforcement learning to build agents that can operate in high-dimensional environments. The

main motivation for using deep networks in RL is to handle complex, high-dimensional state spaces that cannot be represented by simple tabular or linear methods (Zhang and Yu 2020). A key development in DRL was the deep Q-network (DQN), which used a neural network to approximate the Q-value function and enabled agents to learn directly from raw inputs such as image pixels (Zhang and Yu 2020). DRL has demonstrated strong performance in domains such as game playing, robotics, and control, where the state and action spaces are too large for traditional RL methods (Shrestha and Mahmood 2019; Zhang and Yu 2020).

2.2 Machine Learning in Product Development and Production

This section reviews how machine learning is applied in industrial product development and production. It first describes common application areas, then the learning paradigms used in these settings. The last part summarises the findings of the pre-master project that this thesis builds on.

2.2.1 Applications of Machine Learning

Machine learning is widely applied in industrial product development and production, where it supports tasks such as process optimisation, quality prediction, fault diagnosis, predictive maintenance, and anomaly detection (Wuest et al. 2016; Plathottam et al. 2023). The application of ML in these domains continues to grow, driven by the increasing availability of production data and the improved usability of ML tools (Wuest et al. 2016). Unlike traditional rule-based approaches, ML methods can learn directly from data and adapt to changing conditions, making them well suited to the complex and dynamic nature of manufacturing environments (Wuest et al. 2016). In practice, ML is used both to improve product quality and to optimise production processes, for example through early prediction of manufacturing outcomes, root cause analysis, and condition monitoring of equipment (Weichert et al. 2019). As well as individual processes, ML also supports broader operational tasks such as production planning, scheduling, and supply chain management (Plathottam et al. 2023; Presciuttini et al. 2024).

2.2.2 Learning Paradigms in Industrial Applications

Across reported applications in manufacturing and production, supervised learning is the dominant paradigm, accounting for the large majority of studies in most application areas (Wuest et al. 2016; Nti et al. 2022). This reflects the nature of many industrial problems, where labelled data are available through quality inspections, sensor logs, and operator feedback, and where the goal is typically to predict a known outcome such as product quality or equipment failure (Wuest et al. 2016; Presciuttini et al. 2024). Unsupervised learning plays a smaller but important role, particularly in anomaly detection and condition monitoring, where labelled examples of faults are scarce or costly to obtain (Presciuttini et al. 2024). Reinforcement learning is less commonly reported in industrial settings (Wuest

et al. 2016), though it has shown strong results in process control and scheduling problems, where decisions must be made sequentially over time (Panzer and Bender 2022). In practice, the choice of learning paradigm is often shaped by what data is available rather than by the structure of the problem itself (Wuest et al. 2016).

2.2.3 Pre-Master Findings

In a preceding specialisation project (Bergstø 2026), referred to as the pre-master project, a large-scale analysis of machine learning research in product development and production was conducted, classifying over 33,000 article abstracts across product lifecycle (PLC) phase, application area, learning paradigm, and reported ML methods. The results showed that the literature is strongly concentrated in the optimisation phase of the product lifecycle, while early phases such as planning and concept development contain comparatively few studies. The analysis confirmed the paradigm distribution described in Section 2.2.2: supervised learning dominated across all phases and application areas, accounting for approximately 73–80% of the articles in each lifecycle phase, with neural networks, random forests, and support vector machines as the most frequently reported methods. Unsupervised learning appeared mainly in anomaly detection and predictive maintenance, while reinforcement learning was the least frequent paradigm and was largely limited to control and scheduling problems.

2.3 Ontologies and Knowledge Representation

This section presents the background on ontologies and knowledge representation that the system in this thesis builds on. It first describes what an ontology is and how one is structured, then explains why the ML domain is suited to ontological representation. It then introduces the Semantic Web standards used to implement ontologies, and reviews existing ML ontologies. The last part examines ML Ontology in more detail, since it is used as a starting point for the work in this thesis.

2.3.1 Ontology Description

An ontology is an explicit specification of a conceptualization, where a conceptualization is an abstract model of a domain that identifies its relevant concepts and the relationships between them (Gruber 1995). Studer, Benjamins, and Fensel (1998) refine this by describing an ontology as a *formal, explicit specification of a shared conceptualization*. Each term in this definition carries a distinct meaning. *Formal* means the ontology is machine-readable and expressed in a language with precise semantics. *Explicit* means that all concepts and the constraints on their use are clearly defined, rather than left implicit in code or documentation. *Shared* means the ontology captures knowledge that is accepted by a group, rather than reflecting the perspective of a single individual (Studer, Benjamins, and Fensel 1998). In practical terms, an ontology defines a common vocabulary for a domain, including machine-interpretable definitions of its basic concepts and the relations among them (Noy and McGuinness 2001).

An ontology consists of classes, which represent concepts in the domain, properties that describe features of those concepts, and relations that link concepts to one another (Noy and McGuinness 2001). Classes are typically organised in a hierarchy where subclasses inherit the properties of their parent classes. Taken together, these elements allow a domain to be described in a way that both humans and software can interpret and reason over (Gruber 1995).

The design of an ontology involves several practical constraints. Gruber (1995) argues that an ontology should require only the minimal ontological commitment needed for its intended purpose, meaning it should make as few claims about the world as necessary. This principle reflects a trade-off identified by Studer, Benjamins, and Fensel (1998): the more reusable an ontology is, the less directly applicable it tends to be in practice, since a more general representation provides less operational guidance for a specific task. Noy and McGuinness (2001) point to a related constraint on scope. An ontology should not attempt to capture everything that could be said about a domain, but only what is needed for the application at hand. Expanding scope beyond what the application requires adds complexity without benefit, and may introduce inconsistencies that are difficult to resolve. Ontology development is also not a one-time activity. As domain knowledge evolves and new requirements emerge, the ontology must be revised, which means the initial design should anticipate extension and allow the vocabulary to be specialised without requiring changes to existing definitions (Gruber 1995; Noy and McGuinness 2001).

2.3.2 Machine Learning and Ontology

The machine learning domain has several properties that make ontological representation a natural fit.

ML methods are not a flat collection of independent techniques. They form hierarchical structures where more specific methods are specialisations of more general ones. Humm (2026) illustrates this directly: a multilayer perceptron is a specialisation of an artificial neural network, and if the latter can be used for classification and regression, the former inherits that applicability without needing it to be stated separately. This kind of reasoning over hierarchical structure is something an ontology supports naturally through inheritance, but is difficult to achieve in a flat list or a conventional database schema.

Beyond hierarchical structure, ML concepts are connected through typed semantic relations. Methods are suited to certain tasks and datatypes, belong to certain learning paradigms, and have different performance characteristics. These associations are not incidental but reflect the structure of the domain itself, and making them explicit is a prerequisite for structured method selection. Keet et al. (2015) motivate their ontology for data mining precisely on these grounds: traditional approaches treated learning algorithms as black boxes, correlating observed performance with data characteristics alone, while an ontology allows the internal characteristics of algorithms, their assumptions, optimisation strategies, and decision mechanisms, to be represented explicitly and reasoned over.

2.3.3 Semantic Web Standards

Ontologies, both in the ML domain and in general, are commonly implemented using standardised languages from the Semantic Web, a set of specifications developed by the World Wide Web Consortium (W3C) to make information machine-readable and interoperable across systems (World Wide Web Consortium 2026).

The foundation is the Resource Description Framework (RDF), the standard knowledge representation language for the Semantic Web (Gibbins and Shadbolt 2009). RDF represents knowledge as triples, each consisting of a subject, a predicate, and an object, forming a directed graph of binary relationships (Gibbins and Shadbolt 2009). RDF Schema (RDFS) extends RDF with basic constructs for defining classes, properties, and constraints (Gibbins and Shadbolt 2009). Building on RDF, the Simple Knowledge Organization System (SKOS) provides a lightweight standard for representing taxonomies and controlled vocabularies, without the formal reasoning capabilities of a full ontology language (Humm 2026).

The Web Ontology Language (OWL) builds on RDF and RDFS and adds richer modelling constructs, including logical operators, quantifiers, and property characteristics such as transitivity (Bechhofer 2009). This gives OWL well-defined semantics that support automated reasoning, making it suitable for ontologies where logical consistency and entailment are required (Bechhofer 2009).

The SPARQL Protocol and RDF Query Language (SPARQL) is used to query RDF data. It has syntax similar to Structured Query Language (SQL) and retrieves data by matching triple patterns against stored graphs (Grobe 2009). RDF data is typically stored in triplestores, database systems designed specifically for handling RDF triples (Grobe 2009).

2.3.4 Existing Machine Learning Ontologies

Several ontologies have been developed to represent knowledge in the ML and data mining domain, each with a different scope and purpose. The following is a selection of openly available ontologies identified within the ML domain.

ML-Schema (Publio et al. 2018), developed by the W3C Machine Learning Schema Community Group, is a top-level schema for representing and exchanging metadata about ML algorithms, datasets, models, and experiments. Its primary goal is interoperability. It provides a shared reference vocabulary that other ML ontologies can be mapped to, reducing fragmentation across platforms. ML-Schema defines around 25 abstract classes but contains no populated instances, meaning it describes structure rather than ML content. It is intended as a foundation for more specific ontologies, not as a standalone knowledge base.

OntoDM-core (Panov, Soldatova, and Džeroski 2014) is a formal ontology for core data mining entities such as datasets, tasks, algorithms, and their executions. It is structured across three layers. A specification layer covers abstract definitions, an implementation layer covers algorithm implementations and parameters, and an application layer covers executions and workflows. OntoDM-core is aligned with established upper-level ontologies, which enables its use across different application

domains (Panov, Soldatova, and Džeroski 2014). It has been applied in areas such as bioinformatics and drug discovery, and serves as a mid-level ontology for Exposé (Vanschoren et al. 2012), an ontology for ML experiments used in OpenML. With 856 classes and detailed taxonomies of tasks and algorithms, it provides high structural granularity, but contains few populated instances and does not cover named ML methods, metrics, or libraries.

The MEX Vocabulary (Esteves et al. 2015) is a lightweight format for exchanging provenance metadata about ML experiments. It is organised in three layers. MEX-Core handles experiment configuration and execution, MEX-Algorithm covers algorithm class and learning method, and MEX-Performance covers evaluation measures. MEX is designed for basic ML iterations only and does not aim to cover the full data mining domain. Preprocessing semantics, model internals, and named ML methods are outside its scope.

ML Ontology (Humm 2026) is an ontology for representing ML content. It covers ML areas, tasks, approaches, preprocessing methods, metrics, libraries, Automated Machine Learning (AutoML) solutions, and configuration parameters, with around 700 individuals and 5000 RDF triples. Unlike the other ontologies, it is not designed for experiment annotation but for integrating different ML tools and frameworks through a shared vocabulary. It supports optional forward-chaining inference via SPARQL and includes built-in quality assurance checks.

Figure 2.3.1 compares the four ontologies across scope, size, and intended use.

Ontology	Purpose	ML Coverage	Size	Representation	Year
ML Ontology	Shared ML vocabulary for tool integration	Tasks, approaches, metrics, libraries, AutoML solutions, config. items	~10 classes, ~700 individuals, ~5000 triples	RDF/RDFS, SKOS, SPARQL, SHACL	**2026**
ML-Schema	Schema for ML experiment metadata and interoperability	Algorithms, tasks, datasets, models, evaluations.	~25 classes, no instances	OWL/RDF	**2016**
OntoDM-core	Semantic annotation of DM algorithms and processes	DM tasks, algorithms, datasets, generalizations, workflows	760 classes, 221 individuals, 1419 axioms	OWL-DL	**2014**
MEX Vocabulary	Lightweight format for ML experiment metadata	Algorithm class, learning method, hyperparameters, performance measures	~402 classes	RDF, PROV-O	**2015**

Figure 2.3.1: Comparison of existing ML ontologies, adapted from Humm (2026).

As Figure 2.3.1 shows, these ontologies have different aims. ML-Schema and MEX are built for annotating and exchanging experiment metadata. OntoDM-core is a broader reference ontology for data mining entities such as datasets, tasks, and algorithms. None of these ontologies provide support for reasoning about which methods fit a given ML problem. ML Ontology differs in this respect, since it links tasks, methods, and metrics in a form that can be queried and reasoned over.

2.3.5 ML Ontology Structure and Content

Among the ontologies reviewed, ML Ontology by Humm (2026) is examined in more detail here because it provides the broadest coverage of ML concepts and methods, making it the most promising foundation for the system developed in this thesis.

ML Ontology is organised into five modules: classes, instances, inference rules, quality assurance checks, and predefined queries. Figure 2.3.2 shows this overall structure.

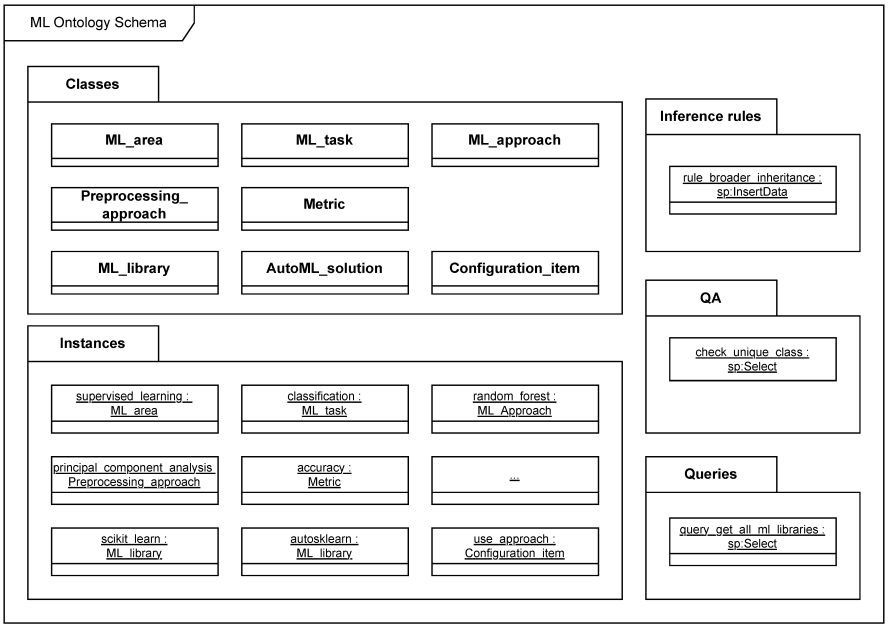

Figure 2.3.2: ML Ontology schema. Reproduced from Humm (2026), licensed under CC BY 4.0.

The classes are divided into two groups: ML concepts, which cover the domain vocabulary, and ML implementations, which cover libraries, AutoML solutions, and their configuration options. Figure 2.3.3 shows the main classes and the relations between them.

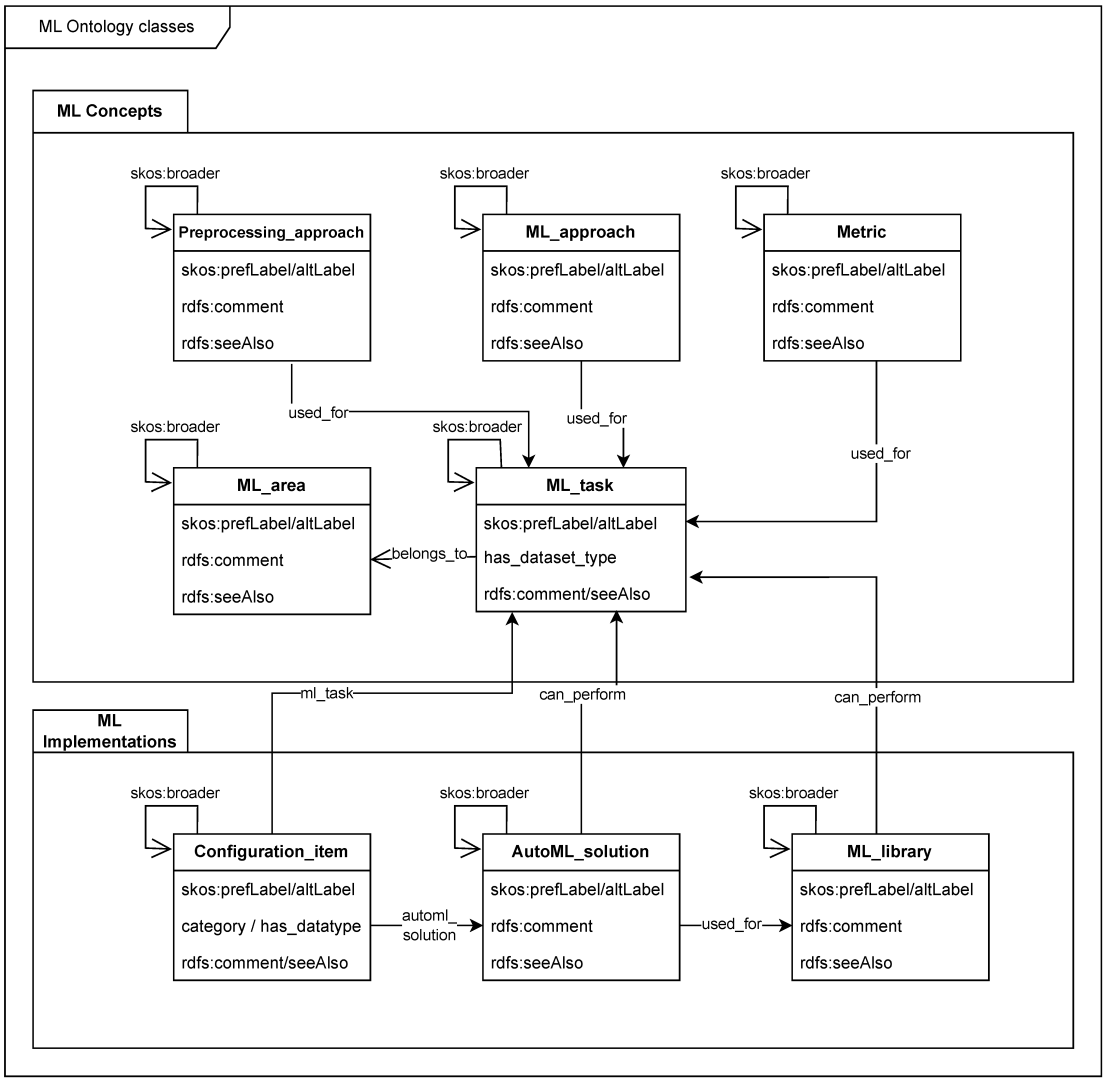

Figure 2.3.3: ML Ontology classes. Reproduced from Humm (2026), licensed under CC BY 4.0.

Some key relations are presented in Figure 2.3.3, among them are `used_for`, which links approaches and metrics to the tasks they apply to, `belongs_to`, which links tasks to learning areas, and `can_perform`, which links libraries and AutoML solutions to tasks. Instances are also linked to corresponding entries in Wikidata (Wikimedia Foundation 2012) via `rdfs:seeAlso`.

Since ML methods are central to the method selection task in this thesis, the properties of `ML_approach` instances are examined in more detail. Table 2.3.1 lists all properties associated with `ML_approach` instances in ML Ontology, as found in the published Turtle file (Humm 2026, Section 6.7).

Table 2.3.1: Properties of `ML_approach` instances in ML Ontology.

Property	Description
`skos:prefLabel`	The preferred display name of the approach.
`skos:altLabel`	Alternative names or abbreviations, for example *ANN* for artificial neural network.
`skos:broader`	Links the approach to a more general parent approach in the hierarchy.
`rdfs:comment`	A short human-readable description of the approach.
`rdfs:seeAlso`	A link to an external resource, typically a Wikidata entry.
`used_for`	The ML tasks this approach can be applied to.
`performance`	Performance characteristics, for example *fast_ training* or *accurate*.
`possible_if`	Conditions under which the approach is applicable, for example *numerical_features*.
`not_recommended_if`	Conditions under which the approach should be avoided, for example *many_features*.

Coverage varies considerably across properties. Table 2.3.2 shows how many of the 122 `ML_approach` instances have each property populated, based on an analysis of the published Turtle file (Humm 2026, Section 6.7). This coverage of the original ontology serves as a baseline for the extensions made in this thesis.

Table 2.3.2: Coverage of `ML_approach` properties across 122 instances in ML Ontology.

Property	Coverage
`skos:prefLabel`	122/122 (100%)
`used_for`	103/122 (84%)
`rdfs:comment`	97/122 (79%)
`skos:altLabel`	64/122 (52%)
`skos:broader`	56/122 (45%)
`rdfs:seeAlso`	56/122 (45%)
`performance`	23/122 (18%)
`possible_if`	9/122 (7%)
`not_recommended_if`	3/122 (2%)

The low coverage of some properties reflects that they must be populated explicitly for each instance. However, ML Ontology includes a forward-chaining inference rule that propagates selected properties along `skos:broader` hierarchies. Properties marked as *broader inherited* are automatically assigned to more specific

concepts based on what is defined for their parent. For the properties in Table 2.3.2, this applies to `used_for`, `belongs_to`, `performance`, `possible_if`, and `not_recommended_if`. The rule must be executed after loading the ontology, and is not applied automatically.

ML tasks are also organised in hierarchies using `skos:broader`. For tasks, the inference rule means that if `classification` belongs to `supervised_learning`, then `text_classification` inherits that relation without it needing to be stated explicitly. Each `ML_task` instance may also carry a `has_dataset_type` property that specifies which data types the task applies to, for example *tabular* or *image*. Since `ML_approach` instances are linked to tasks via `used_for`, dataset type is accessible indirectly through this relation. This means that an approach applicable to `image_classification` is implicitly associated with image data through the task it is linked to. Figure 2.3.4 illustrates the task hierarchy for supervised and unsupervised learning.

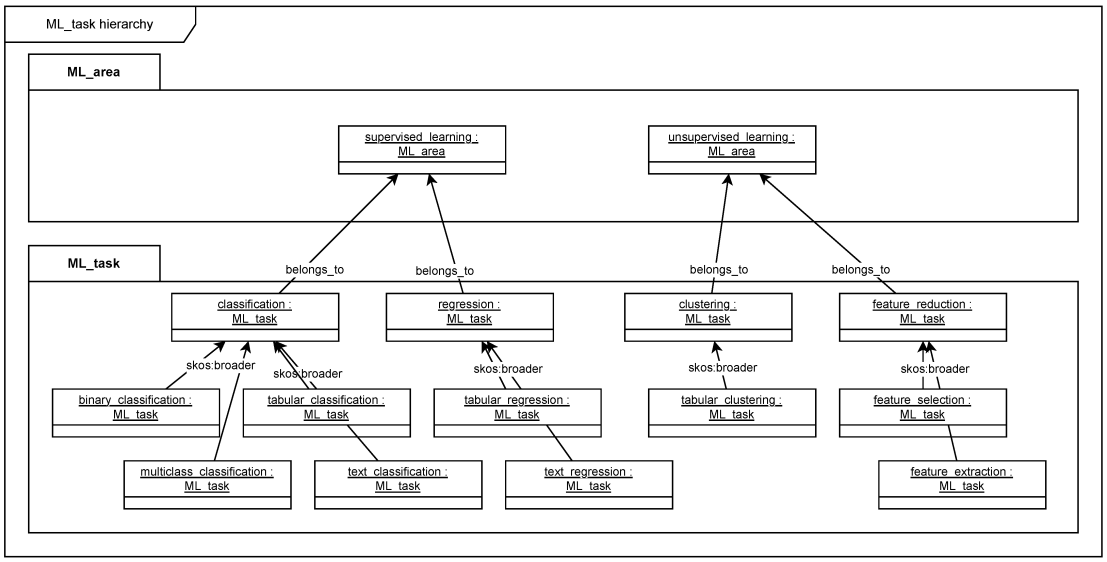

Figure 2.3.4: ML task hierarchy in ML Ontology. Reproduced from Humm (2026), licensed under CC BY 4.0.

Reinforcement learning is defined as an ML area in the ontology but does not appear in the hierarchy since no tasks and no methods have been assigned to it. Coverage of `ML_approach` instances is also incomplete. Humm (2026) report a recall of 0.84 for this class, with graph neural networks, variational autoencoders, and genetic algorithms identified as examples of missing concepts.

2.4 Semantic Retrieval and Text Similarity

Ontologies provide structured, explicit representations of domain knowledge, but they depend on manually defined concepts and relations. This makes them well-suited for reasoning over known entities, yet limited in their ability to handle descriptions expressed in natural language or to retrieve knowledge that was not explicitly encoded. Semantic retrieval addresses this gap by operating on the

meaning of text rather than on predefined structure. This section introduces the core concepts underlying this approach: how text can be represented as numerical vectors, how similarity between texts can be measured in that representation, and how these techniques support ranking and matching against a corpus of scientific literature.

2.4.1 Text Embeddings and Semantic Similarity

In vector semantics, text is represented as a numerical vector, a point in a high-dimensional space referred to as the embedding space (Jurafsky and Martin 2026, p. 116). The position of a vector in this space reflects the semantic content of the text it represents. A central property of this representation is that semantically similar texts are mapped to nearby points in the embedding space, while unrelated texts are placed further apart. Figure 2.4.1 illustrates this principle for a small set of example words.

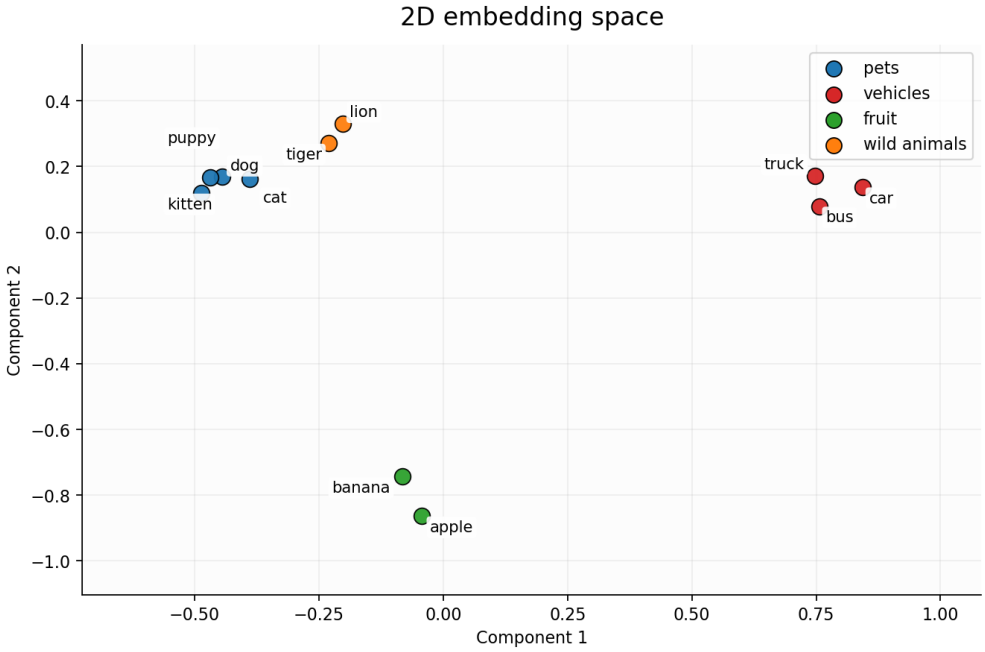

Figure 2.4.1: A two-dimensional projection of example word embeddings. Semantically similar words cluster together in the embedding space, while unrelated words are positioned further apart.

To compare two vectors $\mathbf{u}$ and $\mathbf{v}$, cosine similarity is the most common metric used in natural language processing (Jurafsky and Martin 2026, Chapter 5.4). Rather than measuring absolute distance between two points, it measures the angle between two vectors, normalised for their length:

$$\cos(\mathbf{u}, \mathbf{v}) = \frac{\mathbf{u} \cdot \mathbf{v}}{\|\mathbf{u}\|\|\mathbf{v}\|} \tag{2.1}$$

The result ranges from 1, indicating identical direction and maximum similarity, to 0 for orthogonal vectors with no similarity (Jurafsky and Martin 2026, Chap-

ter 5.4). Normalising for vector length ensures that the similarity score reflects semantic content rather than vector magnitude. In practice, cosine distance, defined as $1 - \cos(\mathbf{u}, \mathbf{v})$, is often used when a distance metric is required, where lower values indicate greater similarity.

2.4.2 Transformer-Based Sentence Embedding Models

Transformer-based language models represent text by processing input tokens through multiple layers of self-attention, producing contextual representations that capture relationships between words across the full input sequence (Vaswani et al. 2023). Unlike earlier embedding approaches that assigned a fixed vector to each word regardless of context, transformer models produce representations that reflect the meaning of a word given its surrounding text.

However, standard transformer models such as Bidirectional Encoder Representations from Transformers (BERT) are not directly suited for computing sentence-level similarity. As noted by Reimers and Gurevych (2019), comparing two sentences with BERT requires feeding both into the network simultaneously, which becomes computationally infeasible at scale. Finding the most similar pair in a collection of 10,000 sentences would require approximately 50 million inference computations.

Sentence-BERT (SBERT) addresses this by fine-tuning BERT using a siamese network structure, producing fixed-size sentence embeddings that can be compared efficiently using cosine similarity (Reimers and Gurevych 2019). Figure 2.4.2 shows this architecture at inference time. The key property is that semantically similar sentences are placed close together in the resulting embedding space, making SBERT suitable for tasks where embeddings for a large corpus can be precomputed and a new query compared against all of them in milliseconds.

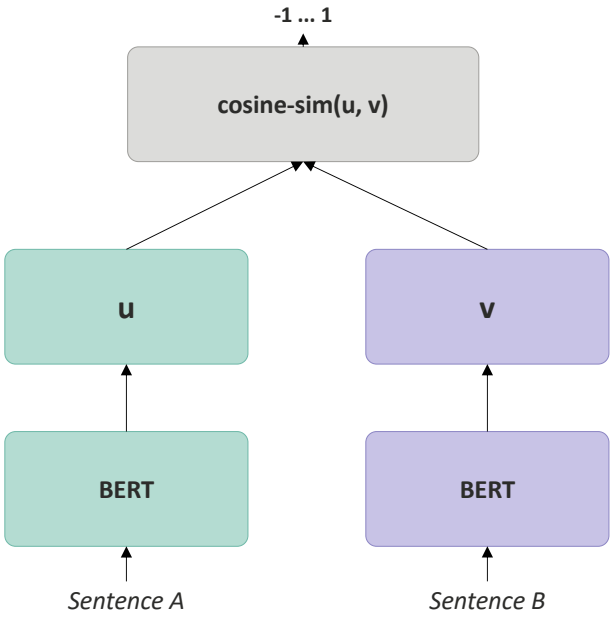

Figure 2.4.2: SBERT architecture at inference time. Each sentence is encoded independently through a shared BERT network, producing vectors **u** and **v** that are compared using cosine similarity. Reproduced from Reimers and Gurevych (2019) under CC BY-SA 4.0.

The sentence-transformers library provides pretrained models based on this approach for encoding text into fixed-size vectors (*Sentence-Transformers Documentation* 2026).

2.4.3 Semantic Retrieval

Flere kilder her!

Information retrieval (IR) is the task of identifying and ranking documents from a collection according to their relevance to a user query (Jurafsky and Martin 2026, Chapter 11.1). Traditional approaches represent both documents and queries as sparse vectors of word counts, weighted by schemes such as term frequency-inverse document frequency (tf-idf) or Best Match 25 (BM25), and compute similarity based on shared terms (Jurafsky and Martin 2026, Chapter 11.1). While effective for many tasks, this approach treats documents as bags of words, meaning that word order and meaning are ignored, and only exact term matches contribute to the relevance score.

Dense retrieval addresses this limitation by representing documents and queries as embeddings computed from language models, rather than as word count vectors (Jurafsky and Martin 2026, Chapter 11.1). Because semantically similar texts are mapped to nearby points in the embedding space, a query can retrieve relevant documents even when they do not share exact terms with the query. This makes dense retrieval more flexible than keyword-based approaches for tasks where the

user's information need is expressed in natural language rather than precise search terms.

Figure 2.4.3 illustrates the distinction with a constructed example. The same query returns only Document A under keyword-based retrieval, as it is the only document sharing exact terms with the query. Dense retrieval ranks all three documents by semantic similarity, including Documents B and C despite the absence of exact term overlap in these documents.

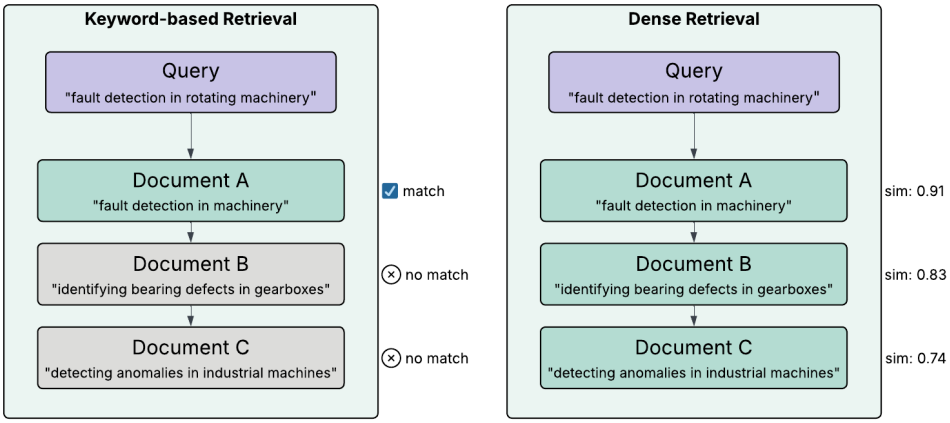

Figure 2.4.3: Keyword-based versus dense retrieval for the same query.

2.4.4 Cross-Encoder Reranking

The sentence embedding approach described in Section 2.4.2 is an example of a bi-encoder, where query and document are encoded independently into fixed-size vectors and similarity is measured using cosine similarity (Reimers and Gurevych 2019). Because the representations are computed independently, document embeddings can be precomputed and stored, making bi-encoders efficient for large-scale retrieval (Thakur, Reimers, Daxenberger, et al. 2021).

A cross-encoder takes a different approach, concatenating query and document into a single input and processing both jointly, which allows the model to apply attention across both texts simultaneously (Reimers and Gurevych 2019). This produces a direct relevance score rather than a similarity between independent vectors. Cross-encoders are generally more accurate than bi-encoders, but since no independent representations are computed, every query-document pair must be processed from scratch at query time, making cross-encoders impractical for retrieval over large document collections (Humeau et al. 2020; Thakur, Reimers, Daxenberger, et al. 2021).

Figure 2.4.4 illustrates the architectural difference between the two approaches.

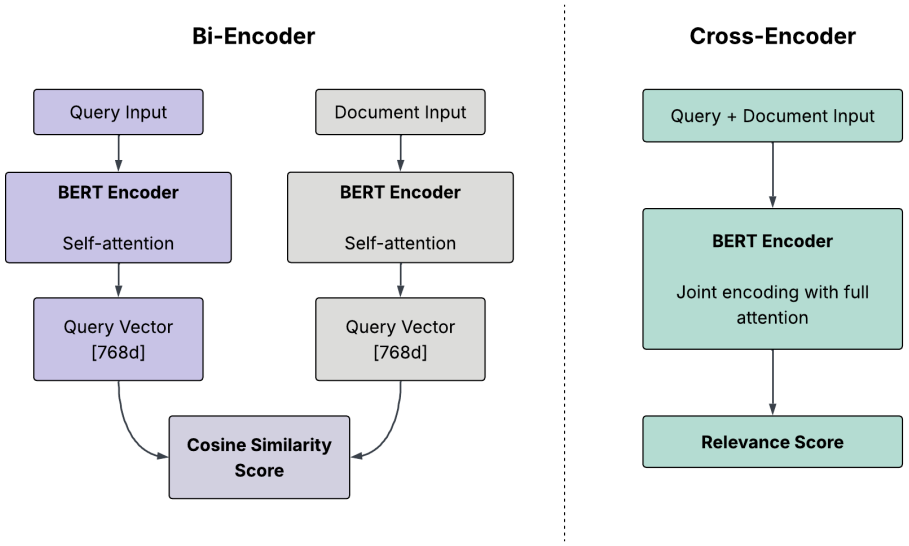

Figure 2.4.4: Bi-encoder and cross-encoder architectures. The bi-encoder encodes query and document independently, producing vectors that are compared using cosine similarity. The cross-encoder concatenates both inputs and processes them jointly, enabling full attention between query and document inputs and producing a direct relevance score. Inspired by Kumar (2024).

A common approach to combining the strengths of both architectures is a two-stage retrieval pipeline (Nogueira and Cho 2020). In the first stage, a retrieval system narrows the full document collection down to a smaller set of candidates. In the second stage, a cross-encoder reranks these candidates by scoring each one against the query directly, producing a more precise relevance ranking. Bi-encoders are well suited for the first stage for the same reason described above: document embeddings can be precomputed, making large-scale candidate retrieval efficient (Thakur, Reimers, Daxenberger, et al. 2021; Dorkin and Sirts 2025; Verma, Jiang, and Xue 2025).

This pipeline has been applied across diverse retrieval tasks, including ranking text passages by relevance (Nogueira and Cho 2020), biomedical question answering (Verma, Jiang, and Xue 2025), and subject tagging (Dorkin and Sirts 2025). Cross-attentional reranking has been shown to generalise well across diverse retrieval tasks (Thakur, Reimers, Rücklé, et al. 2021), and in several settings the two-stage pipeline has shown substantially better retrieval performance than using a bi-encoder alone (Dorkin and Sirts 2025; Verma, Jiang, and Xue 2025).

2.5 User Feedback in Recommendation Systems

Recommender systems collect information on user preferences, either explicitly through numerical ratings, or implicitly by monitoring user behaviour (Bobadilla et al. 2013, Chapter 3.1). This feedback can be stored and used to improve recommendations in subsequent interactions, so that the longer users engage with the system, the more the output can be refined to match their preferences (Ricci, Rokach, and Shapira 2010, Chapter 1).

A practical problem with explicit ratings is that some items receive few ratings, making their average score unreliable. This is related to the sparse data problem, which frequently occurs when estimating counts from small amounts of data (Murphy 2012, Chapter 3.3). Bayesian smoothing addresses this by pulling the estimated score towards a neutral prior when little data is available. The posterior is a compromise between what was previously believed and what the data indicates (Murphy 2012, Chapter 3.3). As more data accumulates, the observed average comes to dominate the estimate, since the data has overwhelmed the prior (Murphy 2012, Chapter 3.3).

When a rating-based signal must be combined with a score from a separate ranking process, the two may not be directly comparable in scale. Reciprocal Rank Fusion addresses this by converting rank positions into scores rather than using the underlying score values directly. Highly-ranked items receive higher scores, but the importance of lower-ranked items does not vanish entirely, as it would with an exponential function (Cormack, Clarke, and Buettcher 2009). This approach combines ranks without regard to the arbitrary scores returned by particular ranking methods, making it robust to differences in scale between sources (Cormack, Clarke, and Buettcher 2009).

The effectiveness of explicit feedback depends on having sufficient ratings. The cold start problem occurs when it is not possible to make reliable recommendations due to an initial lack of ratings (Bobadilla et al. 2013, Chapter 3.2). A particular variant is the new community problem, which refers to the difficulty of obtaining a sufficient amount of rating data when a system is first deployed (Bobadilla et al. 2013, Chapter 3.2). Under these conditions, enough ratings have to be collected before the system can understand user preferences and provide reliable recommendations (Ricci, Rokach, and Shapira 2010, Chapter 3).

2.6 Related Work and Existing Tools

The work reviewed in this section falls into three groups. The first is approaches to method selection: frameworks and knowledge representations that help a user identify a suitable method for a problem. The second is semantic retrieval from scientific literature, which is central to how method selection is approached in this thesis. The third is existing tools and platforms that provide access to machine learning methods or automate parts of the workflow. The three groups are reviewed in turn, followed by a summary of the gaps that motivate the work.

2.6.1 Approaches to Method Selection

Several simple and accessible decision flowcharts for method selection have been developed. Scikit-learn provides a publicly available flowchart that guides users through questions about data size and task type to suggest candidate methods (Scikit-learn developers n.d.), and Microsoft Azure offers a similar cheat sheet for its machine learning platform (Microsoft n.d.). Both rely mainly on accuracy and prediction speed as selection criteria, and do not consider factors such as interpretability, data characteristics, or domain context (Sala et al. 2018).

More structured frameworks characterise methods against a set of criteria and match a described problem to them. Riesener et al. (2020) present an evaluation method for selecting a method in product development. From a large set of methods found in the literature, the eleven most frequently described were selected and each rated on nine criteria covering the learning task, the form of learning, and seven performance properties such as accuracy, training duration, computational effort, and transparency. The result is a matrix of method strengths and weaknesses. The user describes a problem in the same criteria, and the method with the smallest Euclidean distance to that description is suggested. Lickert et al. (2021) take a similar approach for supervised methods, describing seven methods against criteria grouped into input, implementation, and output, and matching a problem to them through a single overview table. The approach supports a preselection of candidate methods for later testing rather than the choice of a single optimal method, and is demonstrated on a case from reverse logistics. Sala et al. (2018) propose a two-layer selection framework that incorporates method characteristics such as scalability and robustness to outliers alongside learning type and response type. Arena et al. (2024) develop a structured set of decision variables for method selection in predictive maintenance.

Other frameworks introduce more elaborate matching mechanisms. Sonntag, Luttmer, et al. (2023) apply a pattern language to method selection in product development. Each method is characterised through a pattern with fixed attributes, and the user formalises a problem using the same attributes, so that problem and method can be matched directly. The concept was demonstrated for two methods and tested with seven engineers without machine learning knowledge. The correct method was identified in 64% of cases, and the errors arose mainly from incorrect formalisation of the problem rather than from the matching itself. Sonntag, Pohl, et al. (2024) later proposed a multi-criteria decision analysis framework, in which a reference process first maps a task to a class of methods, and the remaining methods are then ranked against qualitative criteria using the PROMETHEE outranking method, weighted by the user's stated preferences.

A related class of approaches is AutoML, which automatically selects, composes, and parametrises models for a given task or dataset (Waring, Lindvall, and Umeton 2020). AutoML automates parts of the pipeline, including data preparation, feature engineering, hyperparameter optimisation, and model generation, by searching over candidate models and configurations and evaluating their performance (He, Zhao, and Chu 2021). Most systems treat this process as a black box, with little explanation of why a particular configuration was chosen (Barbudo, Ventura, and Romero 2023). The technical phases of the pipeline are the main focus, while the the human-dependent phases, such as the interpretation of results receives little attention (Barbudo, Ventura, and Romero 2023).

A separate line of work represents method knowledge in a structured, queryable form. Gerschütz, Schleich, and Wartzack (2021) present a semantic knowledge base for data-driven methods in product development. Each method is described by properties such as its potentials, limits, algorithm, product development use cases, and the tool that implements it, and the knowledge is realised as a demonstrator in Semantic MediaWiki, where it can be retrieved through queries. The

method properties are populated manually from literature review and industrial use cases. This work was later developed into a formal ontology. AI4PD (Gerschütz, Goetz, and Wartzack 2023) links data-driven methods to product development processes through an OWL ontology, modelled in Protégé and queried with SPARQL. It introduces concepts for processes, methods, tools, and use cases, and uses a task cluster concept to connect company-specific processes to the methods that can support them. A problem is described formally, in terms of the task, the available data, and the hardware, and a SPARQL query returns the matching methods and tools along with documented use cases. The authors note that populating the ontology with method knowledge and use cases is done manually and that the effort required is high.

A related ontology with a different purpose is the Intelligence Task Ontology and Knowledge Graph (ITO) (Blagec et al. 2022), a large knowledge graph of Artificial Intelligence (AI) tasks, benchmark results, and performance metrics. It is built on RDF and OWL and queried with SPARQL, and is structured around a hierarchy of around 1,100 task classes such as natural language processing or image classification. Models and datasets are not described by their own properties, but through the benchmark results they achieved on different tasks, and each result is linked to the article that reported it. The data is imported automatically from Papers with Code, while the task hierarchy and the structure of performance metrics are manually curated. The aim is to study the AI task landscape and track progress over time.

2.6.2 Semantic Retrieval from Scientific Literature

The second group of related work concerns retrieving relevant documents from scientific literature based on the meaning of a query rather than on exact keywords. The underlying techniques are described in Section 2.4.

Berenguer, Mazón, and Tomás (2024) retrieve data tables from publications based on a research problem stated as an abstract. The abstract serves as a representation of the research problem, and the semantic similarity between the abstract and the indexed tables is computed from text embeddings to return a ranked list of relevant tables. The authors argue that a rich natural-language formulation of a research problem gives better retrieval than isolated keywords. A comparison of nine language models finds that contextual models, and in particular models pre-trained on scientific text, clearly outperform non-contextual ones, and that surrounding context is important for good results.

Schopf and Matthes (2024) present NLP-KG, a system for exploratory search of natural language processing literature. It is built around a knowledge graph that links fields of study, publications, authors, and venues. The structural knowledge graph and the publication embeddings are stored separately, the former in a graph database and the latter in a vector database, so that the graph handles relations and the vector database handles semantic similarity search. The user submits a text query, which is matched against the publications through a retrieval pipeline that combines sparse and dense representations, merged using Reciprocal Rank Fusion and then reranked using metadata such as citation count. The system is intended to help researchers explore an unfamiliar field.

Park et al. (2026) address the task of finding related research papers given a query paper, evaluated on a benchmark of articles from the major machine learning and NLP venues. They decompose the query paper into aspect-specific subqueries covering its motivation, method, and experiments, use each subquery to retrieve candidate papers, and expand the search iteratively. They report that relying only on abstracts is suboptimal, because abstracts offer only sparse, high-level summaries of a paper, and that using the full content gives better results.

Ostendorff et al. (2022) also work on paper recommendation, but focus on the notion of similarity itself. They argue that two papers can be alike in different ways, by addressing the same task, using the same method, or using the same dataset, and that a single embedding per paper hides this distinction. They therefore represent each paper with several embeddings, one for each aspect, computed from the title and abstract and trained on the Papers with Code corpus of machine learning publications. They find that embeddings of titles and abstracts capture the dataset of a paper well but its method poorly, because methodological detail is mentioned only marginally in titles and abstracts.

2.6.3 Tools and Platforms

Ontology-Based Meta AutoML (OMA-ML) (Zender and Humm 2022) combines multiple AutoML solutions through the ML Ontology, described in Section 2.3.5. The ontology is used to filter configuration options and select AutoML strategies, and pipelines are generated across several machine learning libraries rather than a single one. The user uploads a dataset and starts training, either through a guided wizard for domain experts or detailed configuration for users with ML expertise. The generated models are shown in a leaderboard, where they are compared on prediction quality, prediction time and environmental impact, and a model is selected by the user.

Traingenerator (Rieke 2021) is a web-based tool that generates template Python code and Jupyter notebooks for machine learning tasks. A task and framework are selected from a menu, and a code template is produced that covers data loading, model setup, training, and evaluation. The generated code can be downloaded as a script or notebook, or opened directly in Google Colab. As of May 2026, the hosted version produces a runtime error and does not render the output correctly, although it appeared to function as intended during the development period of this thesis, and the tool can still be inspected through the GitHub repository.[1]

The European AI-on-Demand Platform (AIoDP) and its DeployAI implementation offer a marketplace of pre-trained models, reusable AI modules, and domain-specific applications, supported by cloud and high-performance computing infrastructure (Troumpoukis et al. 2025). The platform targets Small and Medium-sized Enterprises (SMEs), researchers, and public sector organisations, and provides tools for building, training, and deploying AI solutions at scale. Available solutions can be browsed and filtered by domain or use case.

[1] `https://github.com/jrieke/traingenerator`

2.6.4 Reseach Gap

Across the work reviewed here, structured frameworks and knowledge representations for method selection depend on method knowledge populated manually and require the user to formalise the problem in a fixed structure. Semantic retrieval from scientific literature can match a free-text problem description against a large body of methods without such formalisation, but existing systems apply it to other tasks than method selection. None of the reviewed approaches combine an ontology of methods with semantic retrieval from scientific literature to support method selection, and none extend the selection into support for implementing the chosen method. This thesis addresses these gaps together, and the approach is described in the following chapter

CHAPTER

THREE

METHODOLOGY AND SYSTEM OVERVIEW

This chapter presents the methodological framework and describes the design and development of MLguide, a web-based decision-support system for machine learning method selection. Section 3.1 introduces Design Science Research as the overarching framework. Section 3.2 gives a high-level overview of MLguide and its main components. The following sections describe the design goals, the development process, and the evaluation approach.

3.1 Design Science Research

This thesis uses Design Science Research (DSR) as its methodological framework, following Peffers et al. (2007) and Hevner et al. (2004). DSR is a well-established approach for developing and evaluating decision support systems (Arnott and Pervan 2012), and has been applied in recent work combining machine learning with user-facing decision support (Landwehr et al. 2022; Dellermann et al. 2018). In DSR, knowledge is gained by building and evaluating an artifact, producing both a usable system and knowledge that can inform future work of a similar kind (Landwehr et al. 2022; Hevner et al. 2004). The DSR approach is therefore suited for this work, where the research questions are investigated through the design and evaluation of the artifact, MLguide.

The DSR process consists of six activities: problem identification, defining objectives, design and development, demonstration, evaluation, and communication (Peffers et al. 2007). These activities are nominally sequential, but iteration is expected. After evaluation, the researcher may return to design and development to improve the artifact before proceeding (Peffers et al. 2007). Figure 3.1.1 shows how these activities map to the structure of this thesis.

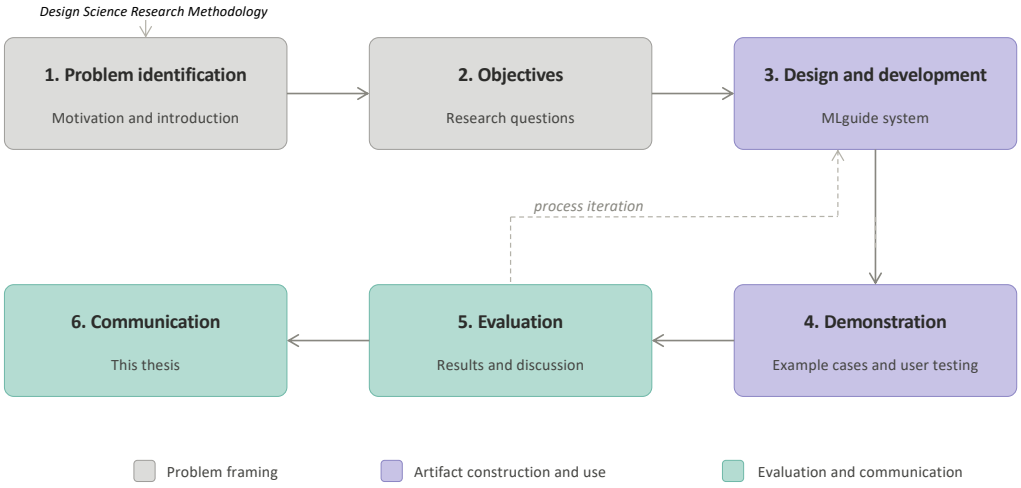

Figure 3.1.1: Design Science Research Methodology adapted from Peffers et al. (2007).

The process in this thesis followed a design-centered initiation, where development of the system started before the research questions were fully formalised (Peffers et al. 2007).

3.2 System Overview

MLguide is a web-based decision-support system designed to help users select machine learning methods for a given problem. The user starts by describing their problem with text input, and can provide filters to narrow the scope of the problem, such as ML area (earlier referred to as paradigm in this thesis) and task type. Figure 3.2.1 gives a high-level overview of MLguide and its main components.

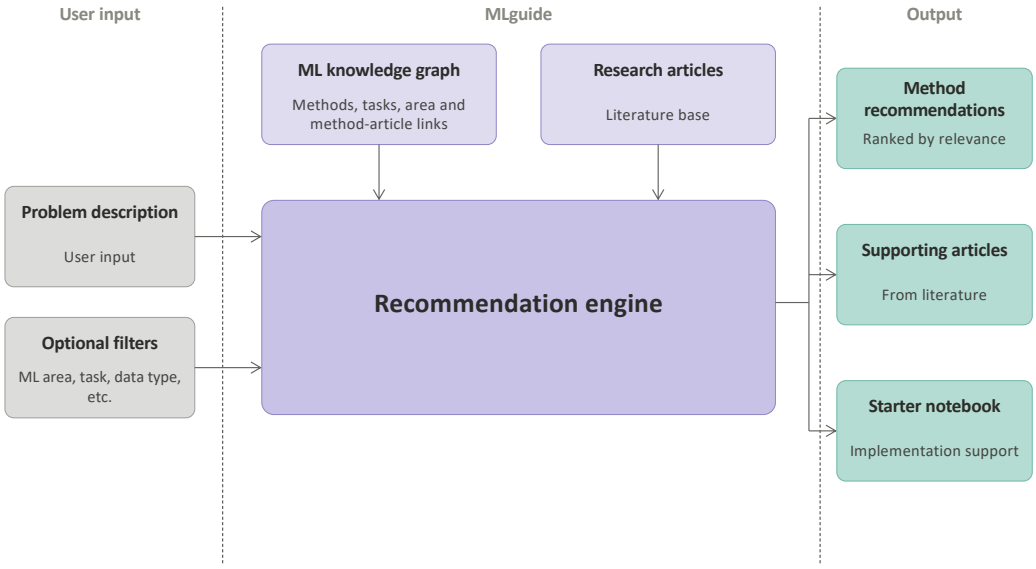

Figure 3.2.1: System Overview of MLguide.

The system uses two knowledge sources. The first is a structured ML knowledge graph containing information about machine learning methods and tasks, including links between methods and the research articles they appear in, among other things. The second is a database of research articles from the literature. With the help of these knowledge sources, the recommendation engine produces three outputs: a ranked list of ML methods, a set of supporting articles, and a starter notebook, a Jupyter notebook file with code generated by MLguide as a starting point for implementation.

3.3 Design Goals

MLguide was developed as a direct continuation of the work done in the pre-master project (Bergstø 2026), which focused on collecting and analysing research literature on ML in product development and production. Each article in the dataset was categorised by ML area, product lifecycle phase, and application area, and ML methods were extracted from the articles, as described in Section 2.2.3. Based on this, a roadmap was developed that filtered and ranked articles and methods according to the user's selected ML area, PLC phase, and application area. A simplified version of this roadmap is shown in Figure 3.3.1.

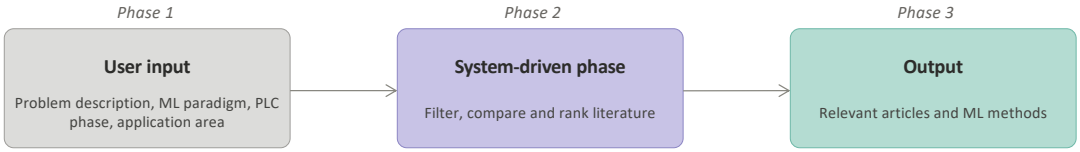

Figure 3.3.1: Simplified ML method selection roadmap from the pre-master project (Bergstø 2026).

Several directions for future development were identified, including better support for problem formulation, ontology-based ML knowledge representation, support for translating recommendations into an ML implementation, and incorporating user feedback to improve method recommendations over time (Bergstø 2026). These directions correspond closely to the gaps identified in Section 2.6, in particular the need to combine structured method knowledge with literature-based evidence and to extend method selection into support for implementation. MLguide addresses several of these by extending the original roadmap into a web-based decision-support system. The main design goals are to support users in formulating their problem, to recommend relevant ML methods based on both structured knowledge and research literature, and to provide a starting point for implementation through generated notebooks. The system is primarily aimed at engineers and students who are new to machine learning and need structured guidance in choosing a suitable method. A web-based interface was chosen to lower the threshold for use, making it easy for other users to access and test the system without installation.

3.4 Incremental and Iterative Development

MLguide was developed incrementally over the course of the master's project. New features and components were built and deployed one at a time, each adding to the working system. This approach is consistent with incremental software development, where the system grows through a series of versions rather than being built all at once (Sommerville 2011, p. 33).

Figure 3.4.1 illustrates this process. Planning, development, and self-evaluation were carried out as concurrent activities, producing successive versions of the system. Throughout development, individual features and the overall system output were continuously tested to check that behaviour was correct and results were useful.

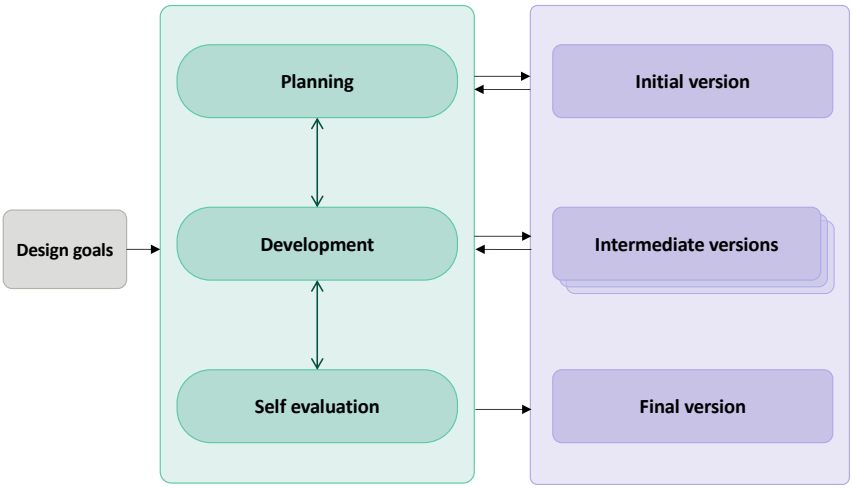

Figure 3.4.1: Illustration of the incremental development process, based on Sommerville (2011, p. 33).

The development also had an iterative character. Feedback from the thesis supervisor informed adjustments to the system design and priorities throughout the development period. In addition, two rounds of user testing were conducted, and the findings were used to evaluate and adjust the system. This iterative cycle is consistent with the DSR process described by Peffers et al. (2007), where evaluation results inform a return to design and development. The user testing and evaluation method is described in the following section.

3.5 Evaluation

MLguide was evaluated in two ways. The first is user testing with a part of the intended user group, which shows how the system is received in real use. The second is an example-based evaluation, where the output of the system is examined for a set of constructed problem cases. The two parts have different strengths. User testing gives independent use of the system, but the problems and the way the system is used cannot be controlled. The example-based evaluation allows the

input to be controlled, so the output can be examined more systematically. The two parts therefore complement each other.

3.5.1 User Testing

User testing was conducted in collaboration with the course TMM4128 Machine Learning for Engineers at the Norwegian University of Science and Technology (NTNU). The course provided a natural opportunity to test the system with its intended user group, since the participants were engineering students with limited prior experience with machine learning. Student groups used MLguide for two of their course projects. The two course projects are summarised in Table 3.5.1.

Table 3.5.1: Description of the two user testing sessions conducted in collaboration with the course TMM4128 Machine Learning for Engineers at NTNU.

	Project 1	**Project 2**
Topic	Anomaly detection in time series data	Reinforcement learning in product lifecycle
ML area	Unsupervised learning	Reinforcement learning
Task	Detect anomalies using unsupervised learning techniques and compare performance	Analyse RL applications across lifecycle phases and implement a sustainability agent
MLguide task	Test MLguide on their problem and report on relevance and usefulness of results	Assess the applicability of MLguide to their problem
Deadline	March 2026	April 2026

The two projects differed in scope and focus. The first project had a well-defined problem: detecting anomalies in time series data using unsupervised learning. The groups were asked to compare at least three ML methods.

The second project was more open-ended. The groups were asked to analyse applications of reinforcement learning across four product lifecycle phases, product design, manufacturing, product use, and recycling, and to implement a simulated RL environment and a sustainability agent.

MLguide provides two channels for user feedback. The first is the students' project reports. As part of these course deliverables, the students reported on how MLguide was used and its applicability to the project work. The second is a feedback form, available on the home page, where users can rate their overall satisfaction, report any technical problems, and note what worked well and what should be improved.

The analysis of feedback has two distinct parts, one quantitative and one qualitative. The quantitative part looks at numbers regarding the use of the system, such as how many groups used MLguide and at what stage of the project. For

this, each group response is coded into a structured form that records whether it used the system, how it was used, and how the recommended methods compare to the methods the group used. The qualitative part examines the feedback from the form text answers and the report free text. Comments about the usefulness of the system, and about any problems or criticism, are sorted into themes, so that points raised by one or more groups can be collected and compared systematically.

3.5.2 Example-Based Evaluation

In addition to the user testing, the system was evaluated using a set of constructed examples. Each example is a problem description created to represent a typical use case for machine learning in product development and production. The examples were chosen to cover different ML areas and task types.

Each example was submitted to MLguide, and the output was examined, including the ranked methods, the supporting articles, and the generated notebook. The aim was to check whether the output is reasonable and relevant for the given problem. Because the examples were both written and assessed as part of this work, this evaluation is a structured review of the system's behaviour rather than an independent test, and it is used to complement the user testing.

3.6 Development Timeline

Figure 3.6.1 summarises the development timeline of MLguide. It marks the two user tests and the example-based evaluation, and divides the development into three main system versions. Version A is the version used in the first user test, version B the version used in the second, and version C the current and latest version used in the example-based evaluation. The main technical differences between the versions are described in Section 4.8 in the Implementation chapter.

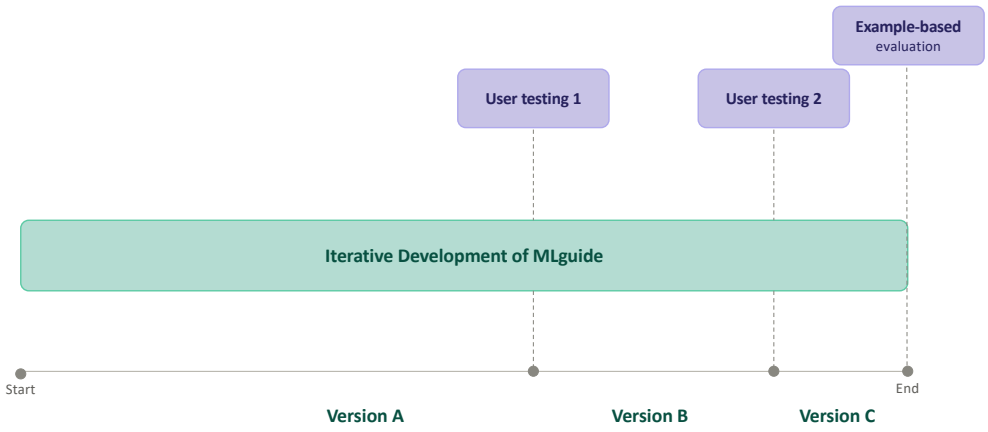

Figure 3.6.1: Development timeline of MLguide, showing the three system versions, the two user tests, and the example-based evaluation.

CHAPTER

FOUR

IMPLEMENTATION

This chapter describes the implementation of MLguide, a web-based decision-support system for machine learning method selection. The system combines an ontology-based knowledge graph with semantic retrieval from scientific literature to produce ranked method recommendations grounded in research literature. It also provides starter notebook templates to support the first steps of implementation.

The chapter is organised by component. The overall architecture is presented first, followed by the knowledge graph and database, the recommendation process, notebook generation, the article retrieval pipeline, and the user interface. Alongside the description of each component, the main design choices are stated together with the reasoning behind them. The chapter closes with a summary of the changes made across the evaluated system versions.

The implementation of MLguide is published as open source:

`https://github.com/mbergsto/MLguide.git`

The main `README` covers the most important details, such as the repository structure, requirements, and how to set up and run the system. Each main component also has its own `README` with further detail. These practical details are not repeated in this chapter.

A running instance of MLguide is available at:

`https://mlguide.it.ntnu.no/`

The deployed instance reflects the state of the system at the end of the active development period in spring 2026. As a research prototype, its later availability cannot be guaranteed.

4.1 System Architecture

MLguide is structured as a three-tier web application consisting of a frontend, a backend, and a database layer, deployed as Docker containers (Docker Inc. 2026). Docker was chosen because containerisation makes the system easy to run across different environments without managing dependencies manually, which is important for a research artefact that needs to be shared and tested by others.

Figure 4.1.1 shows the main components and how they communicate. The frontend is implemented in Streamlit (Streamlit Inc. 2026), which was chosen because it allows interactive web applications to be built directly in Python, without writing HTML or JavaScript. This made it well suited for rapid development of a data-driven prototype. The frontend communicates with the backend through an Application Programming Interface (API) over the Hypertext Transfer Protocol (HTTP). The backend is implemented in FastAPI (Ramírez 2026), which provides automatic API documentation and built-in data validation, and contains the core application logic.

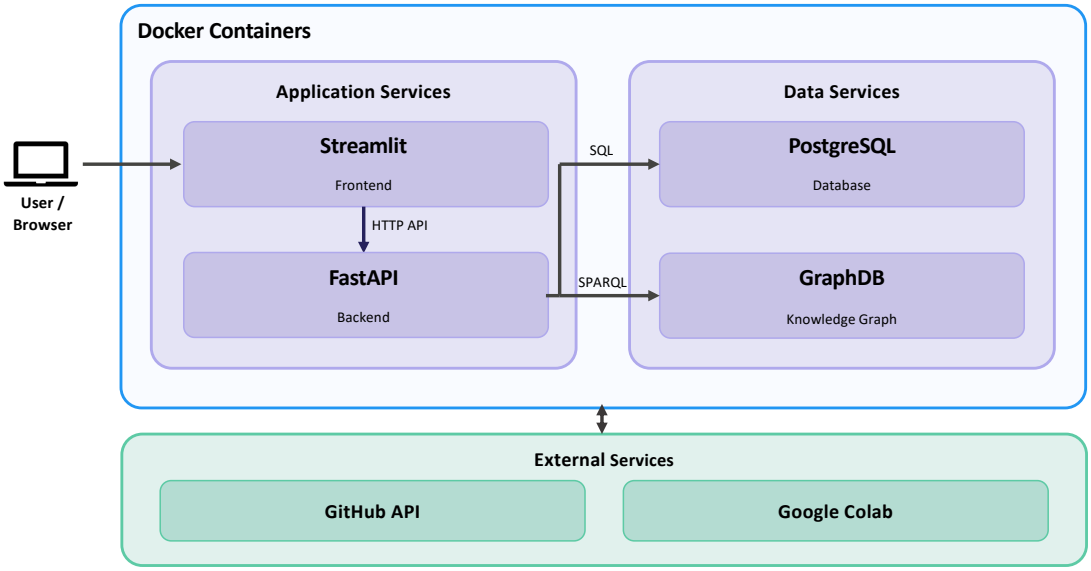

Figure 4.1.1: System Overview of MLguide.

The backend reads from two data stores. GraphDB (Ontotext 2025) stores the ML ontology and article links, and is queried using SPARQL. It was chosen because it is designed specifically for RDF data and supports SPARQL, making it a natural fit for an RDF-based knowledge graph. PostgreSQL (The PostgreSQL Global Development Group 2025) stores article abstracts and vector embeddings using the pgvector extension (Kane 2025). This allows embeddings and relational data to be stored and queried in the same database, avoiding the need for a separate vector database. The frontend also integrates with two external services: the GitHub API and Google Colab, which are used to upload and open generated notebooks.

MLguide follows a layered architecture, in which each layer only depends on the layer immediately beneath it (Sommerville 2011, pp. 157–158). This keeps con-

cerns separated across the system, so that individual layers can be modified or replaced without affecting the rest of the application. The layered approach also supports incremental development, because each layer can be made available as it is completed without waiting for the full system to be built (Sommerville 2011, pp. 157–158). This is consistent with the development process described in Section 3.4.

Figure 4.1.2 shows the internal layer structure of the frontend and backend. Both are organised into distinct layers with clear responsibilities. In the backend, the router layer handles incoming API requests and passes them to the service layer, which contains the application logic. The persistence layer is responsible for all database access and isolates data retrieval from the rest of the application. In the frontend, the presentation layer handles rendering and user interaction. The application logic layer coordinates calls to the backend and external services. Because the frontend is a Streamlit application that delegates most logic to the backend, the majority of modules in this layer are thin wrappers around the integrations layer. The clearest exceptions are the notebook template services, which contain local logic for selecting and rendering starter notebooks.

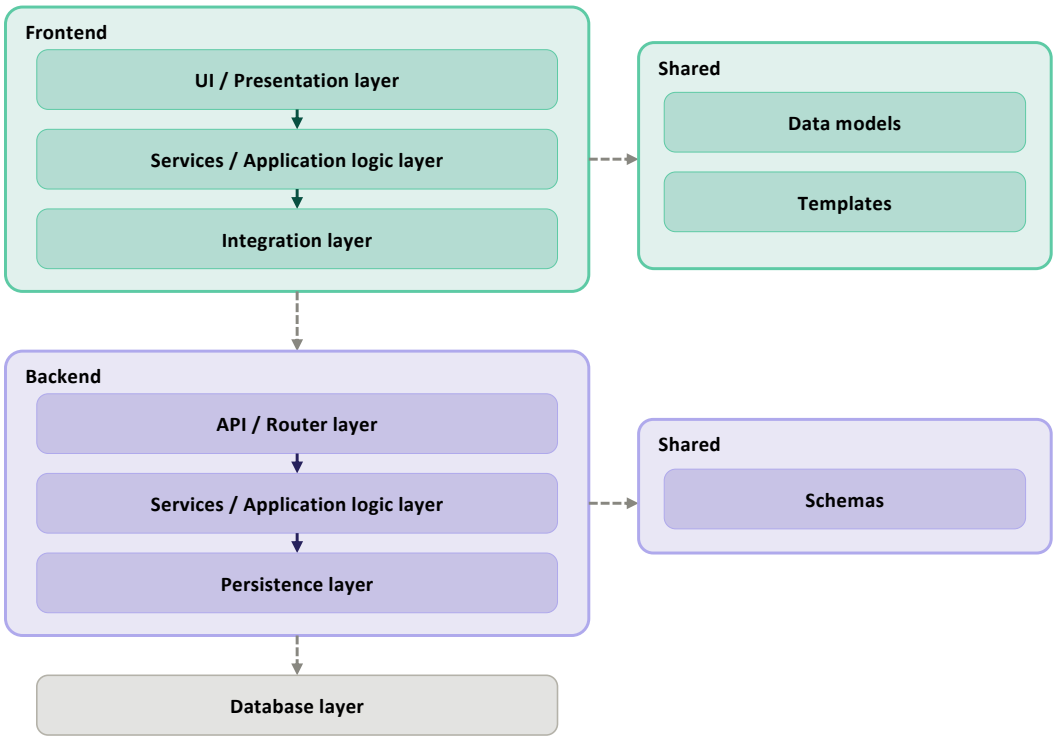

Figure 4.1.2: Layered Architecture of MLguide.

Figure 4.1.3 shows the full package structure of both the frontend and backend, including the main modules within each layer. In addition to the recommendation logic, the system handles two other concerns that run through most layers. User services is a lightweight feature covering authentication and saved searches, allowing users to store and reload previous searches with problem descriptions. Feedback handling allows users to rate recommended methods, with ratings stored in PostgreSQL and used to adjust method ranking over time. Both concerns follow

the same layered structure as the recommendation logic, with separate modules in the router, service, and persistence layers of the backend, and corresponding wrappers in the frontend application logic layer. The feedback logic is further described as part of the recommendation process in Section 4.3.

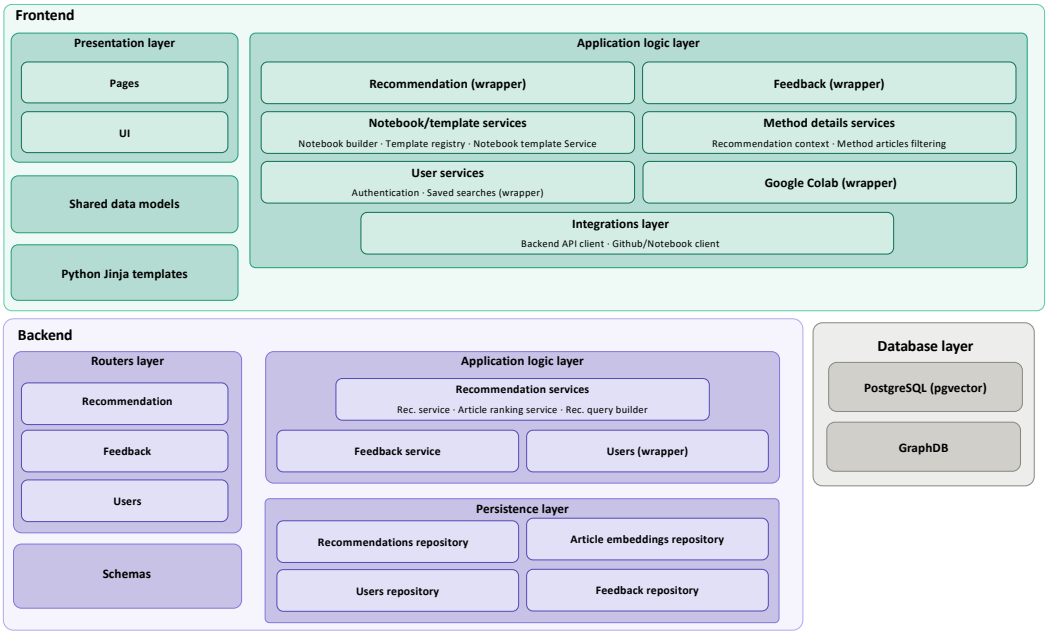

Figure 4.1.3: Package Diagram of MLguide.

4.2 Knowledge Graph and Database

This section describes the two data stores used in MLguide. The first is the ML knowledge graph stored in GraphDB, which is based on ML Ontology and extended for use in MLguide. The second is the PostgreSQL database, which stores article abstracts and vector embeddings used for semantic retrieval.

4.2.1 ML Ontology Extension

ML Ontology was selected as the knowledge base for MLguide because it provides the broadest coverage of ML concepts among the ontologies reviewed in Section 2.3.5, and because its structure directly supports the kind of structured method retrieval that MLguide requires. It is open source and published under an MIT licence (Humm 2026), which allows for use and modification as needed. The limitations identified in Section 2.3.5, including the absence of reinforcement learning instances and incomplete coverage of `ML_approach`, required extension before the ontology could be used in MLguide. Some extensions were also made to support better knowledge structure and specific MLguide functionality. The extension was carried out gradually across the system versions. This section describes the extension as a whole.

Three new classes were added under a separate namespace (`mla:`), short for *machine learning articles*, to make clear that they are not part of the original ML Ontology. Table 4.2.1 describes these classes.

Table 4.2.1: New classes added under the `mla:` namespace.

Class	Description
`mla:Article`	Represents a research article from the literature dataset. Linked to one or more `mla:Method` instances that are mentioned in the article.
`mla:Method`	Represents an ML method as extracted from article abstracts. Linked to a corresponding `ML_approach` instance via `skos:exactMatch`.
`mla:ML_task_family`	Groups related `ML_task` instances under a common family label, helps selecting notebook templates during notebook generation.

`mla:Method` and `ML_approach` represent the same concept but serve different roles. The reason for keeping them separate is practical: `ML_approach` instances carry relations to other ML concepts in the ontology, such as which tasks a method can be used for and which learning area it belongs to. Adding article relations directly to these instances would mix two different kinds of knowledge on the same node, making the ontology harder to navigate during development. Instead, article relations are kept on `mla:Method`, and the two classes are connected via `skos:exactMatch`. If needed, this separation could be removed by transferring the article relations directly to the corresponding `ML_approach` instances.

Figure 4.2.1 shows the class structure of the extended ontology. The upper part shows the classes from the original ML Ontology that are directly connected to the three new `mla:` classes: `ML_approach`, `ML_task`, and `ML_area`. The lower part shows the new `mla:` classes and how they connect to these. `mla:Method` is linked to `ML_approach` via `skos:exactMatch`, and to `mla:Article` via `mentionsMethod`. `mla:ML_task_family` is linked to `ML_task` via `has_family`, and `mla:Article` is linked to `ML_area` via `has_ml_area`.

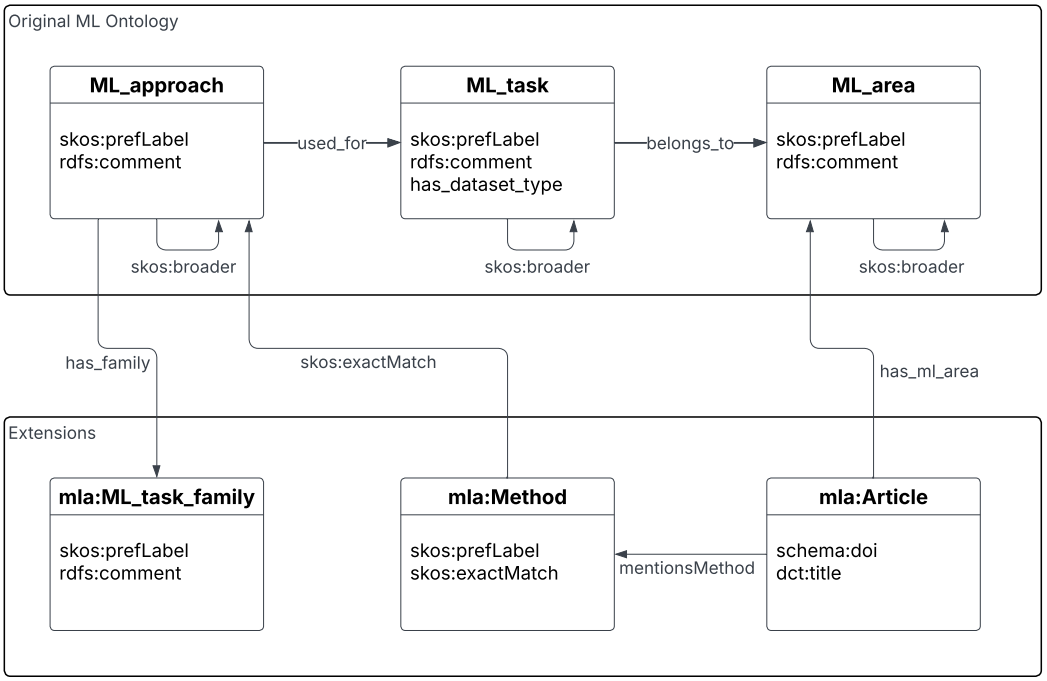

Figure 4.2.1: Unified Modeling Language (UML) class diagram of the ML Ontology, including added classes and properties for MLguide.

Figure 4.2.2 shows a concrete example of how the extended ontology represents an article and its relations. The `mla:Article` instance represents a research article identified by its Digital Object Identifier (DOI) and title. It is linked to `supervised_learning` via `has_ml_area`, and to `method_linear_regression`, an `mla:Method` instance, via `mentionsMethod`. The method instance is in turn linked to the `ML_approach` instance `linear_regression` via `skos:exactMatch`, which carries the ontology relations: `used_for` links it to the `ML_task regression`, which belongs to `supervised_learning` via `belongs_to`, and `has_family` links it to the `mla:ML_task_family` instance `regression_family`.

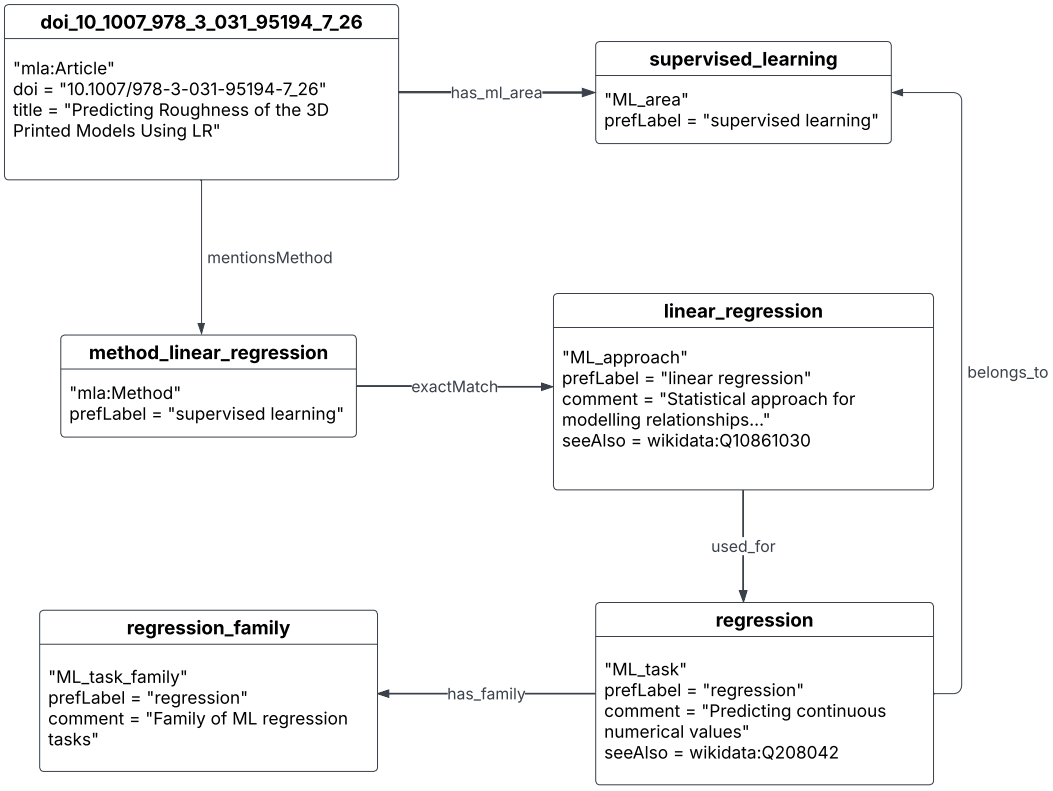

Figure 4.2.2: Example of an Article instance in the ML Ontology.

New `ML_approach` instances were added to address the coverage gaps identified in Section 2.3.5. A first set was derived from the method extraction results of the pre-master project (Bergstø 2026), where ML methods were identified from article abstracts in the literature dataset. Methods that appeared in the dataset but had no corresponding `ML_approach` instance in ML Ontology were added. Reinforcement learning methods and tasks were added based on established taxonomies of RL algorithms (Zhang and Yu 2020; Geron 2019), following the same classification used in Section 2.1.2. In total, 65 new instances were added, grouped by theme in Table 4.2.2.

Table 4.2.2: New `ML_approach` instances added to ML Ontology, grouped by theme.

Theme	New instances	n
Reinforcement	deep reinforcement learning, Q-learning, REINFORCE, deep Q-network, policy gradient, proximal policy optimization, soft actor-critic, actor-critic, deep deterministic policy gradient, twin delayed DDPG, A3C	11
Deep learning	deep neural network, attention mechanism, graph attention network, graph neural network, graph convolutional network, GraphSAGE, U-Net, variational autoencoder, normalizing flow, diffusion model, DeepAR	11
Evolutionary	genetic algorithm, particle swarm optimization, simulated annealing, differential evolution, ant colony optimization, firefly algorithm, harmony search, evolutionary strategy, CMA-ES	9
Anomaly detection	isolation forest, local outlier factor, one-class SVM, HBOS, Hotelling T^2, squared prediction error, minimum covariance determinant, elliptic envelope	8
Time series	Kalman filter, temporal convolutional network, N-BEATS, seasonal ARIMA, state space model, VAR, Holt-Winters	7
Explainability	explainable AI, SHAP, Granger causality, DoWhy, CausalImpact, causal forest	6
Probabilistic	Bayesian method, Gaussian process regression, partial least squares, elastic net, robust regression	5
Clustering	k-medoids, Gaussian mixture model clustering, t-SNE, UMAP	4
Ensemble	gradient boosting, extra trees, bagging	3
Other	naive Bayes	1
Total		**65**

For methods that form hierarchical families, `skos:broader` relations were added to preserve the inheritance structure of the ontology. For example, `deep_q_network` was linked as broader to `q_learning`, and `proximal_policy_optimization` to `policy_gradient`. After adding new instances, the inheritance rule described in Section 2.3.5 was applied to propagate `used_for` and `belongs_to` relations to the new instances through their `skos:broader` hierarchies.

Where available, links to corresponding Wikidata entities were added to new instances via `rdfs:seeAlso`, following the same approach used in the original ontol-

ogy (Humm 2026). Candidates were retrieved using the Wikidata search API and scored by string similarity between labels and whether the Wikidata description contained predefined ML-related terms. Results were reviewed manually before being added.

Table 4.2.3 shows the resulting coverage across all 187 `ML_approach` instances after the extension and applying the inference rule. Coverage here means the share of `ML_approach` instances that have a value for a given property. For reference, the original coverage across 122 instances is shown in Table 2.3.2.

Table 4.2.3: Coverage of `ML_approach` properties across 187 instances after ontology extension and inference rule application.

Property	Coverage
`skos:prefLabel`	187/187 (100%)
`used_for`	180/187 (96%)
`rdfs:comment`	130/187 (70%)
`performance`	95/187 (51%)
`rdfs:seeAlso`	90/187 (48%)
`skos:altLabel`	66/187 (35%)
`skos:broader`	57/187 (30%)
`possible_if`	28/187 (15%)
`not_recommended_if`	5/187 (3%)

Note: `has_dataset_type`, which relates to `ML_approach` indirectly through `ML_task`, is populated for 29 of the 43 `ML_task` instances that exist after the extension.

Populating instances with properties requires manual work, and gathering the knowledge to do so takes effort. The knowledge used in this work with updating the graph, was based on the established ML sources and textbooks also used in Section 2.1. When updating the knowledge, new instances were prioritised, since they had no properties at all. Reinforcement learning is one example, as it had no instances in the original ontology. Clear gaps found during testing of the recommendation process, or through user testing feedback, were also prioritised. Among the properties, `used_for` and `performance` were prioritised, as these are central to the candidate retrieval and ranking described in Section 4.3.1. As a result, coverage is improved but remains incomplete for several of the remaining properties, as Table 4.2.3 shows.

`mla:ML_task_family` instances were also added to group related `ML_task` instances under a common label. This was motivated by the large amount of similar tasks, especially for classification and regression. Grouping them into families provides a clearer structure. Task families also support notebook generation by providing a better structure for selecting the correct template for a given method and task combination, since many methods can be applied to more than one type of task. This is described in more detail in Section 4.4.

Families were defined by inspecting the `skos:broader` hierarchy to identify which tasks were already grouped under common parent tasks, and also by grouping

tasks of similar character where no such hierarchy existed. Each family was given a name that reflects the shared nature of its tasks. Table 4.2.4 lists the task families and their assigned tasks.

Table 4.2.4: ML_task_family instances and their assigned ML_task instances.

ML_task_family	Assigned ML_task instances	n
classification_family	classification, binary_classification, multi-class_classification, tabular_classification, text_classification, image_classification, audio_classification, time_series_classification, video_classification, vertex_classification	10
regression_family	regression, tabular_regression, text_regression, image_regression, audio_regression, video_regression	6
clustering_ unsupervised_ discovery_family	clustering, tabular_clustering, association_rule_learning, community_detection, topic_modeling	5
detection_ localization_family	object_detection, anomaly_detection, action_recognition	3
feature_engineering_ family	feature_extraction, feature_reduction, feature_selection	3
structured_ prediction_family	image_segmentation, named_entity_recognition, translation	3
graph_tasks_family	graph_matching, link_prediction, vertex_classification	3
nlp_analysis_family	sentiment_analysis, text_analytics	2
ranking_ recommendation_ similarity_family	ranking, recommendation, sentence_similarity	3
time_series_family	time_series_analysis, time_series_forecasting, time_series_classification	3
reinforcement_ learning_family	policy_learning, value_estimation, policy_optimization	3
tracking_family	video_object_tracking	1

4.2.2 Database Structure

Article data is stored across several tables in PostgreSQL, shown in Figure 4.2.3. The `article_abstracts` table stores the raw abstract text for each article, used during reranking. The `abstract_embeddings` and `title_embeddings` tables store 768-dimensional vector embeddings of the abstract and title respectively, used for semantic similarity search via pgvector (Kane 2025). The `users` table stores

a minimal user record created on first login. The `saved_searches` table stores previously saved searches with problem descriptions, allowing users to reload them without re-entering their inputs. The `method_feedback` table stores user feedback on recommended methods, capturing all context related to the user's problem description and the specific method. The `skipped_articles` table used in the process retrieving new articles for the knowledge graph and article database. It is further explained in Section 4.5.5.

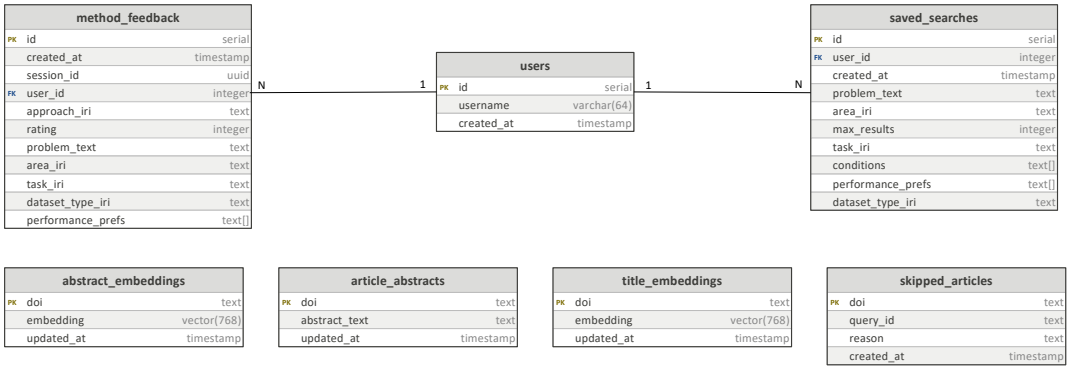

Figure 4.2.3: Entity-Relationship Diagram of the PostgreSQL database used in MLguide.

4.3 Recommendation Process

The recommendation process takes the user's input and produces a ranked list of ML methods. The process has two main stages. In the first stage, candidate methods are retrieved from the ontology based on the parameters the user has selected. In the second stage, if a problem description is provided, the candidates are reranked using semantic similarity to relevant research articles. A feedback boost is applied as a final adjustment to the ranking.

Figure 4.3.1 shows the sequence of steps in the recommendation process, covering ontology-based candidate retrieval and semantic similarity ranking. Feedback is not included in detail in the diagram. It is described separately in Section 4.3.4.

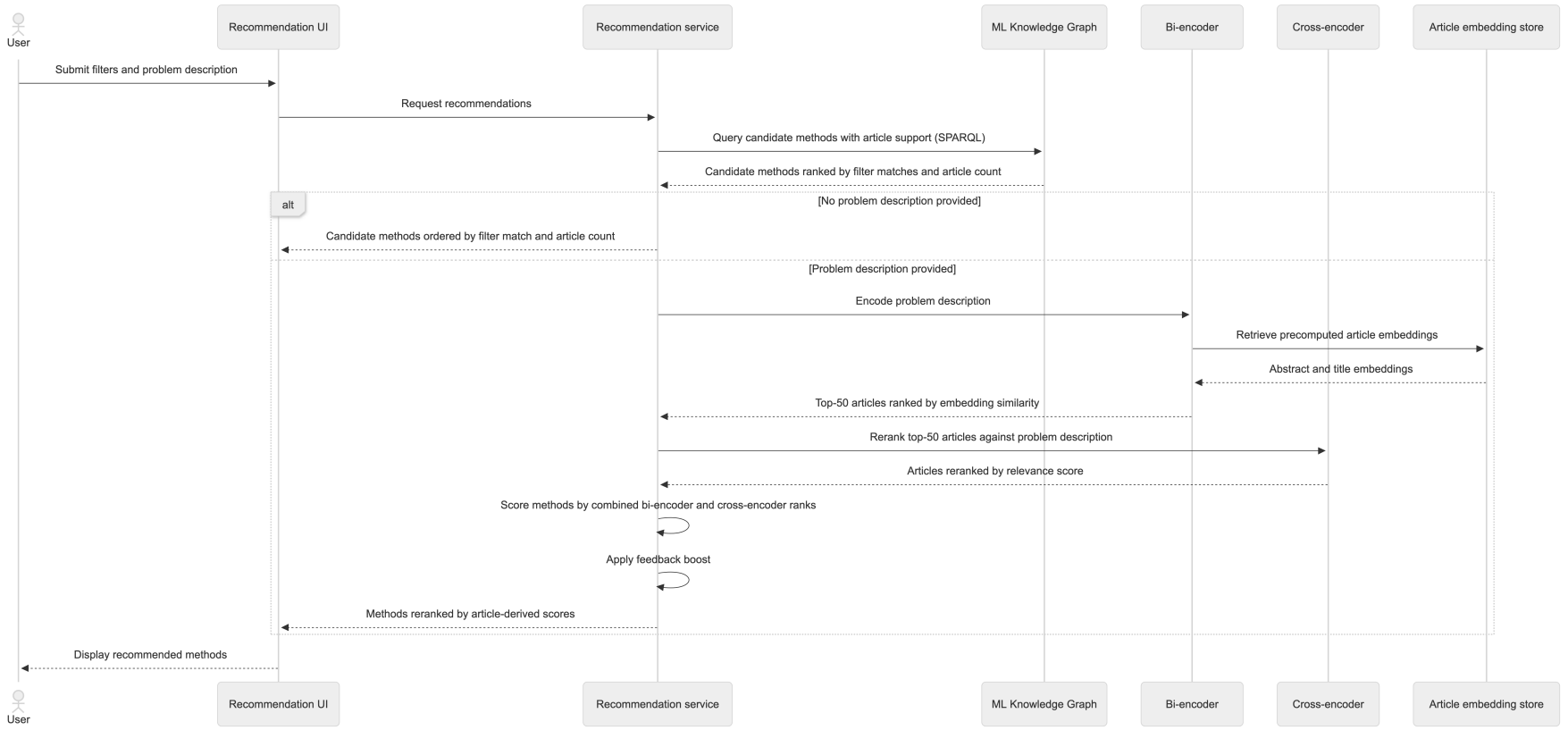

Figure 4.3.1: Sequence diagram of the recommendation process in MLguide, showing ontology-based candidate retrieval and semantic similarity ranking.

4.3.1 Ontology-Based Candidate Retrieval

The first step is to retrieve candidate methods from the knowledge graph using a SPARQL query against GraphDB. The query goes through the graph from articles to methods to approaches. It finds articles related to the selected ML area, follows the `mentionsMethod` relation to `mla:Method` instances, and then follows `skos:exactMatch` to the corresponding `ML_approach` instances. A candidate approach is only included if at least one article in the knowledge graph mentions a method that maps to it, which grounds the candidate set in the research literature.

The properties used to filter and rank candidates are listed in Table 4.3.1. They were selected because they describe characteristics of an ML method that can be matched against the user's problem context. ML area is the only hard filter, since all articles in the knowledge graph are related to an ML area, restricting to the selected area does not risk excluding relevant candidates. The remaining properties have incomplete coverage, as shown in Table 4.2.3, and are therefore used as soft signals that boost the ranking of matching methods without excluding non-matching ones.

Referere til 2.1.2 ? Og den manglende delen om attributter på metoder

Table 4.3.1: Ontology properties used for filtering and ranking in candidate retrieval.

Property	Source	Example values
`has_ml_area`	`mla:Article`	`supervised_learning`, `reinforcement_learning`
`used_for`	`ML_approach`	`classification`, `regression`, `anomaly_detection`
`performance`	`ML_approach`	`fast_training`, `accurate`, `explainable`
`has_dataset_type`	`ML_task` via `used_for`	`tabular`, `image`, `text`

Task, performance, and dataset type filters are applied as optional matches. For each candidate method, the query counts how many of the user's selected values match the corresponding ontology properties. These counts are used to sort candidates, so methods that match more of the user's filters rank higher. Dataset type is accessible indirectly through the tasks linked via `used_for`, as shown in the table.

The other properties of `ML_approach`: `possible_if` and `not_recommended_if` are retrieved and returned alongside each method as informational notes, but are not used for filtering or ranking. This is due to the low coverage of these properties shown in Table 4.2.3. Therefore using them for filtering would risk excluding valid candidates.

In the end, candidates are sorted by how well they match the user's filters, with the number of supporting articles as a tiebreaker. If no problem description is pro-

vided, this ordered list is returned directly, only adjusted for feedback as described in Section 4.3.4.

4.3.2 Article Similarity Scoring

When a problem description is provided, articles in the database are ranked by their semantic similarity to the query, using the two-stage retrieval pipeline combining a bi-encoder and a cross-encoder, as described in Section 2.4.4. Similarity is measured using cosine similarity, which is described as the most common metric in Section 2.4.1. It compares the angle between two vectors regardless of their magnitude (Jurafsky and Martin 2026, Chapter 5.4).

In the first stage, the problem description is encoded using `all-mpnet-base-v2` (Reimers and Gurevych n.d.[b]), a sentence-transformer model based on MPNet (Song et al. 2020). This model was used in the pre-master project (Bergstø 2026) to compute the article embeddings stored in the database, so the same model must be used here to ensure the problem description is encoded in the same embedding space. The model was also selected for its quality. It produces 768-dimensional embeddings and has been shown to perform well on abstract-level semantic similarity tasks in scientific literature (Galli, Donos, and Calciolari 2024; Colangelo et al. 2025). Article scores are computed as a weighted sum of cosine similarities between the query embedding and the precomputed abstract and title embeddings:

$$\text{score}_a = w_{\text{abs}} \cdot \text{sim}_{\text{abs}}(a) + w_{\text{title}} \cdot \text{sim}_{\text{title}}(a) \tag{4.1}$$

where $w_{\text{abs}} = 0.7$ and $w_{\text{title}} = 0.3$. The higher weight on abstracts reflects that abstracts contain more information about the method and application context than titles. These weights are set as defaults, but can be adjusted. The top $k = 50$ articles are retained for the reranking step.

In the second stage, the top-k articles are reranked using `ms-marco-MiniLM-L6-v2` (Reimers and Gurevych n.d.[a]), a cross-encoder fine-tuned on the MS MARCO passage ranking dataset (Bajaj et al. 2018). The model was chosen because it achieves strong ranking performance while being lightweight enough for query-time inference over a small candidate set. Among 14 publicly available cross-encoder models evaluated on MS MARCO, it was found to offer the best performance (Thakur, Reimers, Rücklé, et al. 2021). As described in Section 2.4.4, a cross-encoder processes the query and each article jointly, producing more accurate relevance scores than the bi-encoder alone.

4.3.3 Method Scoring

After article reranking, each candidate method is scored based on the articles that mention it. Combining rank positions from both stages gives weight to both broad semantic similarity from the bi-encoder and precise relevance scoring from the cross-encoder. Using rank positions rather than the raw similarity and relevance scores from each model also avoids differences in scale between the two sources, consistent with the principle behind Reciprocal Rank Fusion described in Section 2.5:

$$\text{score}_m = \sum_{\substack{a \in \text{top-}k \\ m \in a}} \left(\frac{w_b}{r_b(a) + 1} + \frac{w_c}{r_c(a) + 1} \right) \tag{4.2}$$

where $r_b(a)$ and $r_c(a)$ are the zero-based rank positions of article a in the bi-encoder ranking and cross-encoder ranking respectively. The weights w_b and w_c are normalised with $w_b + w_c = 1$, and both default to 0.5. The choice to combine both signals was made by manually checking output for queries of different types and levels of detail, since no ground-truth ranking data exists. Using only the cross-encoder score directly was the initial approach and is a possibility. Both the signal weights and this choice are configurable. The score calculation ensures that articles ranked higher contribute more to the method score, and the contribution decreases gradually for lower-ranked articles.

4.3.4 Feedback Boost

After methods are ranked by similarity score, user feedback can adjust the final ordering. When a user rates a method on a 1–5 scale, the rating is stored and used in subsequent recommendations. As discussed in Section 2.5, rating estimates based on few observations are unreliable. A smoothed score is therefore computed using Bayesian smoothing towards a neutral prior (Murphy 2012, Chapter 3.3):

$$\hat{r}_m = \frac{s \cdot \mu_0 + n \cdot \bar{r}_m}{s + n} \tag{4.3}$$

where $\bar{r}_m$ is the observed mean rating, n is the number of ratings, $s = 5$ is the prior strength, and $\mu_0 = 3$ is the prior mean on a 1–5 scale. When n is small, the estimate is pulled towards μ_0. As n grows, the observed mean dominates. The smoothed rating is normalised to $[-1, 1]$ relative to the neutral rating of 3. The prior strength of 5 means that approximately five ratings are needed before feedback has a meaningful effect on the score. This is a low threshold, set with the expectation that the system will initially have a small user base. The value is a default and can be adjusted.

The feedback score is combined with the similarity-based ranking using a rank-based score, so that the two signals are on a comparable scale:

$$\text{combined\_score}(m) = \frac{1}{\text{rank}(m) + 1} + w_f \cdot \text{feedback\_score}(m) \tag{4.4}$$

where $\text{rank}(m)$ is the zero-based position after similarity ranking, and $w_f = 0.15$ is an adjustable weight value. The low weight means that feedback can adjust a method's position by a few places, but cannot override the similarity-based ranking entirely.

Figure 4.3.2 illustrates the full ranking process, from initial retrieval of candidate methods, through similarity scoring and feedback boost.

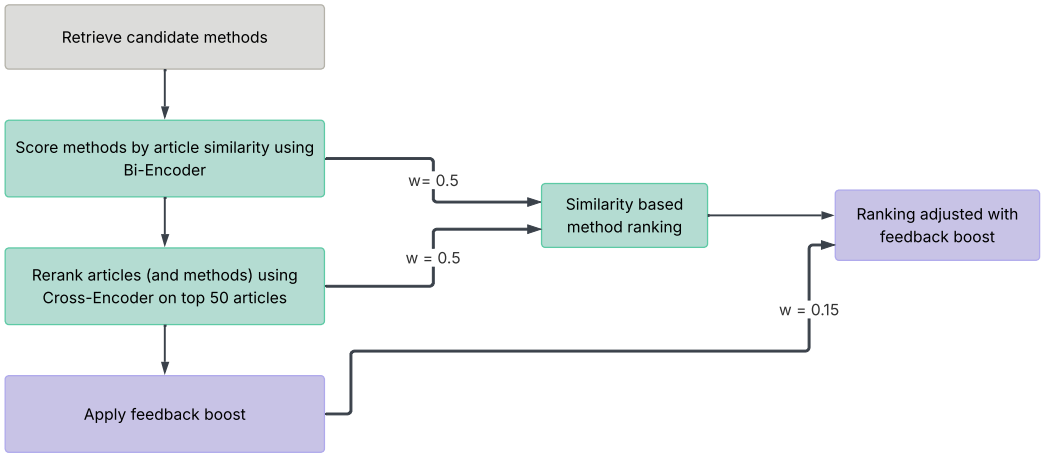

Figure 4.3.2: Flowchart of the ranking process in MLguide, including similarity scoring and feedback boost.

4.4 Notebook Generation

Jupyter notebooks (Kluyver et al. 2016) are widely used in ML development because they allow code, explanatory text, and visual output to be combined in a single document, making them well suited for experimentation and exploration. Providing a pre-structured notebook for a recommended method gives users a concrete starting point without having to write boilerplate code from scratch. A template-based approach makes it possible to generate such notebooks automatically, with content tailored to the selected method and the user's problem description.

MLguide has the possibility to generate a starter notebook for recommended ML methods. This relies on the existence of a pre-written template that defines the notebook structure and code. When a user opens a method details page, by clicking on one of the recommended methods, a notebook is generated automatically from a template and displayed as a code preview. The user can then download the notebook or open it directly in Google Colab. This section describes how templates are structured, how notebooks are generated, what the notebooks contain, and how they are made available to the user.

Notebook generation relies on the following tools:

- **Jinja2** (The Pallets Team 2026): A Python templating engine used to render notebook templates with method-specific and user-provided content.
- **jupytext** (Wouts 2026): Tool for converting Python code into the `.ipynb` format used by Jupyter and Google Colab.
- **Google Colab**: A cloud-based notebook environment that allows users to run the generated notebook directly in the browser without local setup.
- **GitHub API**: Used to upload the generated notebook so that Google Colab can load it from a publicly accessible URL.

Much of the notebook generation approach, including the use of Jinja2 templates and the GitHub-based Open in Colab mechanism, is inspired by Traingenerator (Rieke 2021), which is described in Section 2.6.3. Unlike Traingenerator, where the user selects task, framework and more parameters manually, MLguide selects the template automatically based on the recommended method that is chosen by the user.

4.4.1 Template Resolution and Rendering

The generation process has three steps. First, the template resolver maps the selected method to a Jinja2 template file. Templates are organised by task family and method key, following the path structure:
`templates/notebooks/{family}/{method_key}.py.jinja`.

Second, the template is rendered with user-specific context, including the problem description, method title, and task family. This produces a Python source file personalised to the user's input.

Third, the rendered Python source is converted to `.ipynb` format using jupytext. Colab-compatible metadata is added at this stage, including kernel specification and notebook name, so the file can be opened in Google Colab without additional configuration.

4.4.2 Colab Export

The user can open the generated notebook directly in Google Colab. The notebook is uploaded to a configured GitHub repository via the GitHub API, and a Colab URL is constructed from the repository name, branch, and file path. The browser opens this URL in a new tab, where Colab loads the notebook directly from GitHub.

The whole process of generating the notebook and opening it in Colab is described in the sequence diagram in Figure 4.4.1.

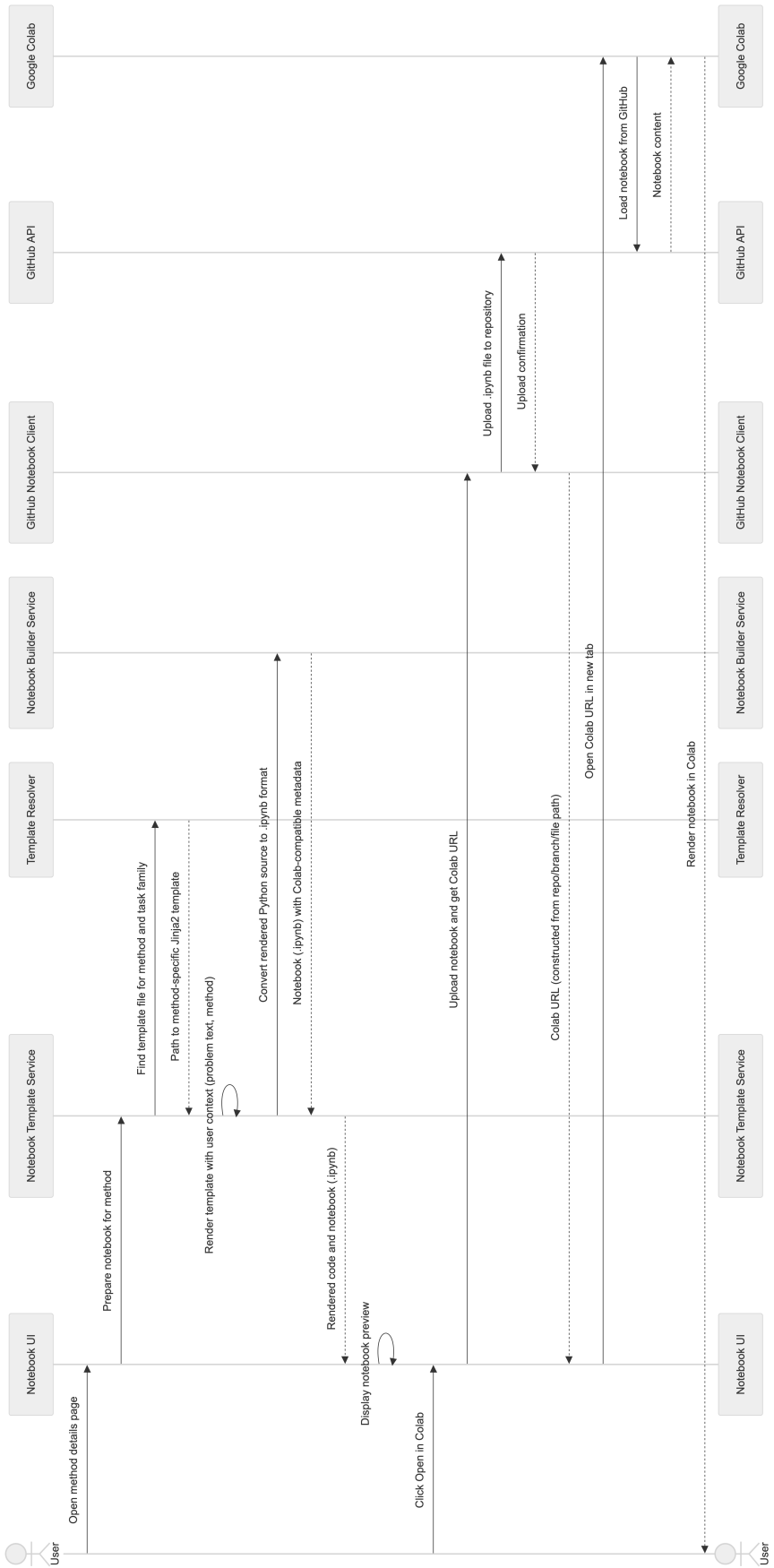

Figure 4.4.1: Sequence diagram of the notebook generation process in MLguide.

4.4.3 Template Content and Inheritance

Each notebook template follows the same fixed structure. It contains the method title and the user's problem description, followed by code cells for environment setup, imports, data loading and preparation, method configuration, and a training and evaluation step. A final markdown cell gives short notes specific to the method.

To avoid repeating the parts of the structure that are shared by methods solving the same kind of task, a base template is defined for each task family. Task families are the task groupings defined in the ontology extension (Section 4.2.1), and their role in the template organisation is described in Section 4.4.4. The base template contains the fixed cells and marks the method-specific parts using Jinja2 blocks. A specific method template can then extend the base template and override only the blocks that are specific to that method. Listing 4.1 shows a shortened version of the base template for the classification family, where the general cells are reduced to comments and the three method-specific blocks are marked.

Listing 4.1: Base notebook template for the classification task family.

```
# %% [markdown]
# {{ method_title }}

Generated by MLguide. This notebook is a starter
workflow for a classification task.

## Problem context
{{ problem_text }}

# %% [markdown]
## 1. Environment setup
# %%
# ... install dependencies ...

# %% [markdown]
## 2. Imports
# %%
# ... standard imports ...
{% block model_import %}{% endblock %}

# %% [markdown]
## 3. Load and prepare data
# %%
# ... load dataset and split into train and test sets ...

# %% [markdown]
## 4. Configure model
# %%
{% block model_init %}{% endblock %}

# %% [markdown]
## 5. Train and evaluate
```

```
# %%
# ... fit the model and print evaluation metrics ...

# %% [markdown]
## 6. Method notes
{% block method_notes %}{% endblock %}
```

A method template only needs to fill in the method specific blocks. Listing 4.2 shows the full template for random forest classification, which extends the base template and overrides the import, configuration, and method notes blocks. Adding a new method requires only a small template of this kind, since the shared structure is inherited from the base template.

Listing 4.2: The random forest template for classification, which extends the base template and overrides its blocks.

```
{% extends "classification/base.py.jinja" %}

{% block model_import %}
from sklearn.ensemble import RandomForestClassifier
{% endblock %}

{% block model_init %}
model = RandomForestClassifier(
    n_estimators=300,
    max_depth=None,
    min_samples_split=2,
    random_state=42,
)
{% endblock %}

{% block method_notes %}
- Random Forest is robust to noisy features and
  non-linear relationships.
- Start with feature importance inspection before
  aggressive tuning.
{% endblock %}
```

The template structure is inspired by Traingenerator (Rieke 2021) and covers the basic steps needed to load a dataset, configure the method, train it, and evaluate the result.

4.4.4 Template Folder Structure and Extensibility

The template folder structure matches the `mla:ML_task_family` classes defined in the ontology extension described in Section 4.2.1 and listed in Table 4.2.4. Template files are named after the method's ontology Internationalized Resource Identifier (IRI), the unique identifier used in the knowledge graph, which gives accurate mapping between the ontology and the file system.

Many ML methods can be applied to more than one type of task, as described in Section 2.1.2 and Section 2.1.3. Such methods have a separate template in each

relevant family folder, since different task types require different code structure. The correct template is selected by inspecting the task chosen in the users problem description. Table 4.4.1 shows a selection of templates to illustrate how methods are distributed across family folders. It is not the full set of templates.

Table 4.4.1: Selected examples of template organisation by task family and method key.

Family / Template file	**Method**
`classification/`	
`base.py.jinja`	*Base template*
`artificial_neural_network.py.jinja`	Artificial neural network
`random_forest.py.jinja`	Random forest
`support_vector_machine.py.jinja`	Support vector machine
`logistic_regression.py.jinja`	Logistic regression
`regression/`	
`base.py.jinja`	*Base template*
`artificial_neural_network.py.jinja`	Artificial neural network
`random_forest.py.jinja`	Random forest
`support_vector_regression.py.jinja`	Support vector regression
`linear_regression.py.jinja`	Linear regression
`clustering_unsupervised_discovery/`	
`base.py.jinja`	*Base template*
`k_means.py.jinja`	K-means

Adding a template for a new method only requires creating a template file with the correct method name in the appropriate family folder. This makes the template system straightforward to extend.

No templates were written for reinforcement learning methods. As described in Section 2.1.1, reinforcement learning involves an agent interacting with an environment, and the environment is typically specific to the problem. A generic template can provide much less of a working starting point in this case than for example for a classification or regression method. How reinforcement learning could best be supported would need to be explored separately, and is discussed further in Section **??**.

Legg til riktig seksjon her!!

4.5 Article Retrieval and Knowledge Extraction

The recommendation process in MLguide depends on a database of research articles, where problem descriptions are matched through semantic retrieval. This article base is built and maintained by a separate article retrieval pipeline, which is the subject of this section.

Article abstracts are used as the textual basis for each article. Text mining of

scientific literature has mostly been carried out on abstracts, both because they are widely available and because they consist of short sentences presenting only the main findings of a study (Westergaard et al. 2018). Full-text articles contain richer information, but also include tables, references, and discussion sections that make them less consistent as a basis for comparison across a large corpus (Westergaard et al. 2018). Abstracts therefore provide a consistently available representation of each article's core content.

Scopus was selected as the source of research articles. Scopus and Web of Science are the two main databases for citation data (Mongeon and Paul-Hus 2016), and of the two, Scopus has the broader journal coverage (Mongeon and Paul-Hus 2016). Scopus also provides an API (Elsevier 2026), which allows article metadata and abstracts to be retrieved automatically. This makes it possible to collect articles in a repeatable way and to expand the article database over time.

4.5.1 Data Collection

The article retrieval pipeline builds on the data collection and classification work from the pre-master project (Bergstø 2026). The same query-based search strategy is reused here, and the pipeline is adapted so that new articles can be added to the MLguide databases over time.

Articles are collected from Scopus through a set of queries, each targeting machine learning applications in a specific domain or application area. Each query is defined in a YAML Ain't Markup Language (YAML) configuration file with an identifier and a Scopus search string. The full set of queries is listed in Appendix A, and the search strategy behind them is described in detail in the pre-master project (Bergstø 2026). When the pipeline runs, each query is sent to the Scopus API (Elsevier 2026) through the Python package pybliometrics (Rose and Kitchin 2019), which provides a stable interface for retrieving data from Scopus. For each article, the DOI, Scopus Electronic Identifier (EID), title, and abstract are stored, together with the identifier of the query that retrieved it. The DOI and EID are used to identify articles and to skip articles that are already present in the database, to avoid duplicates.

4.5.2 Adaptations for MLguide

The pre-master project classified each article along four dimensions: ML area, product lifecycle phase, application area, and the ML methods mentioned in the abstract, as described in Section 2.2.3. In MLguide, only ML area and ML methods are retained, as these are the dimensions used in the recommendation process. ML area is used as the ML area filter, and ML methods link each article to methods in the knowledge graph.

PLC phase and application area were removed because they were not reliable enough to act as filters. As discussed in the pre-master project (Bergstø 2026), single-phase PLC labels are only an approximation, since many studies span several lifecycle phases, and the application areas, represented by clusters, did show limited natural separation. Using either as a filter would risk excluding relevant articles on uncertain grounds. Instead, the system relies on semantic similarity

between the problem description and the article abstracts, as described in Section 4.3.

4.5.3 Knowledge Extraction

After articles are retrieved, two types of information are extracted from each abstract: the ML methods mentioned in the text, and the ML area the article belongs to. Both steps work on a cleaned version of the abstract, where copyright statements, URLs, and similar noise are removed so that matching is more reliable.

ML methods are extracted using a dictionary of known methods. The dictionary was built in the pre-master project, where method phrasings were curated manually and supplemented with candidate phrases extracted from article abstracts using syntactic patterns (Bergstø 2026). It is reused unchanged in MLguide, and the complete dictionary is listed in Appendix B. Each method is linked to a set of common phrasings, and the cleaned abstract is searched for these phrasings using word-boundary matching. A method is recorded for an article if any of its phrasings is found. One case needs extra handling, since the phrase for a specific neural network often contains the general term "neural network". For example, "convolutional neural network" also contains "neural network", so the matching detects both. When a specific variant is found, the general method `neural network` is therefore removed, so that the article is linked only to the variant actually mentioned.

ML area is assigned with a keyword-based approach. Separate keyword lists are defined for supervised, unsupervised, and reinforcement learning, and the abstract is matched against each one. The lists were built in the pre-master project from standard terminology in machine learning, and refined through iterative testing to remove overly generic terms (Bergstø 2026). The complete lists are given in Appendix C. An article can be assigned more than one area if terms from several lists are found. Articles with no matches are not linked to any ML area. The pre-master project extended this keyword step with a model-based prediction step, but found that the weakly supervised training labels were noisy and that the reliability of the resulting model was difficult to assess (Bergstø 2026). In MLguide, only the keyword step is retained, since it is fast to run for every batch and its results are transparent and easy to inspect.

4.5.4 Database Upload

The final step stores the processed articles in the two MLguide databases. Abstracts and embeddings are uploaded to PostgreSQL, and the links between articles and ML methods are uploaded to GraphDB.

For PostgreSQL, each new article is stored with its cleaned abstract, together with embeddings for the title and the abstract. Before the embeddings are computed, each article is checked against the database, and articles that are already present are skipped. This check is done first because embedding generation is the expensive part of the step, and it should not run for articles that will not be added. The embeddings are produced with the `all-mpnet-base-v2` sentence-transformer model, which maps text to 768-dimensional vectors. The same model is used in

the recommendation process, so that problem descriptions and stored articles are placed in the same embedding space, as described in Section 4.3.2.

For GraphDB, the extracted methods and areas are converted to RDF triples in Turtle format and uploaded to the knowledge graph. Each article is linked to the ML methods it mentions, and, where an area was identified, to the ML area it belongs to. These links allow the recommendation process to move from articles to methods when retrieving candidates, as described in Section 4.3.

4.5.5 Flowchart and Database Updates

Figure 4.5.1 shows the flow of the article retrieval process described in the previous sections. Intermediate results are stored as artifacts along the way, so that each step has the data it needs from the previous step, and so that the output can be inspected manually when needed.

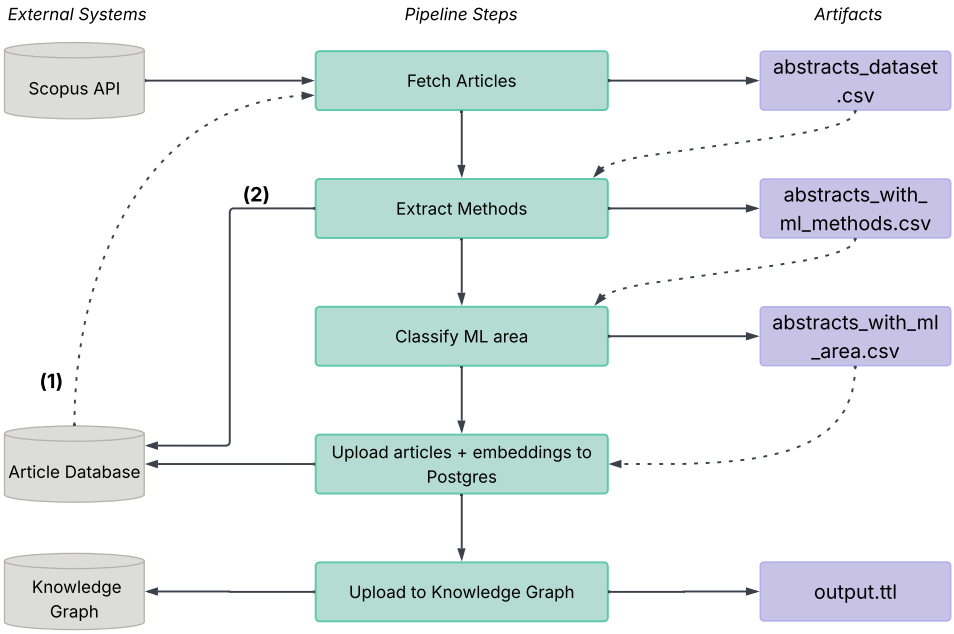

Figure 4.5.1: Flowchart of the article retrieval and knowledge extraction process in MLguide.

For MLguide to stay up to date with current research, the article retrieval pipeline should run periodically to add new articles to the database. The numbered marks in Figure 4.5.1 indicate steps that were added to the pipeline after the first run, to support these updates:

- **(1)** When fetching articles from Scopus, the process checks the `skipped_articles` table in the article database (see Section 4.2.2). This table holds articles that were skipped in earlier runs, so they can be skipped again without further processing, which speeds up the pipeline.
- **(2)** During method extraction, an article with no detected methods is not

added to the database. Its metadata is instead stored in the `skipped_articles` table, so it can be recognised and skipped in future runs.

4.6 Resulting Database

Table 4.6.1 summarises how the number of articles changes through the pipeline. These numbers come from the first run of the pipeline, which collected the articles for the MLguide databases. Articles without an extracted method are not added, since a method is needed in the recommendation process. Articles without an ML area are kept, but receive no ML area link in the knowledge graph. This left 15 651 articles after the first run. A later update run on 25 March 2026 added 2 269 new articles, giving a total of 17 920 articles in the database.

Table 4.6.1: Number of articles after each stage of the article retrieval and knowledge extraction process.

Stage	Articles
Collected from Scopus	52 290
After deduplication and removal of articles without an abstract	33 130
After removal of articles without an extracted method	15 651
Added in the update run of 25 March 2026	+2 269
Total articles in the database	**17 920**

Table 4.6.2 shows how the ML area labels are distributed across the database. An article can be assigned more than one ML area, so the labelled counts overlap and sum to more than the number of labelled articles. Articles where no area keyword was matched have no ML area link in the knowledge graph.

Table 4.6.2: Distribution of ML area labels across the articles in the database.

ML area	Articles
Supervised learning	14 185
Reinforcement learning	1 544
Unsupervised learning	1 505
No ML area	1 017

4.7 User Interface

This section describes the user interface of MLguide and the main steps a user follows when interacting with the system. The interface is built as a Streamlit application, as described in Section 4.1, and consists of two pages. The home page contains the input form, where the problem is described, and the recommendation table, with 10 possible machine learning methods, which appears below the

form once recommendations have been requested. The method details page, for a specific method, is opened when a method is clicked on in the recommendation table.

4.7.1 Input Form

The input form, shown in Figure 4.7.1, is where the user describes the problem to be solved. The problem is entered as a short free-text description (1). All fields are optional, but each one can be used to narrow the recommendations.

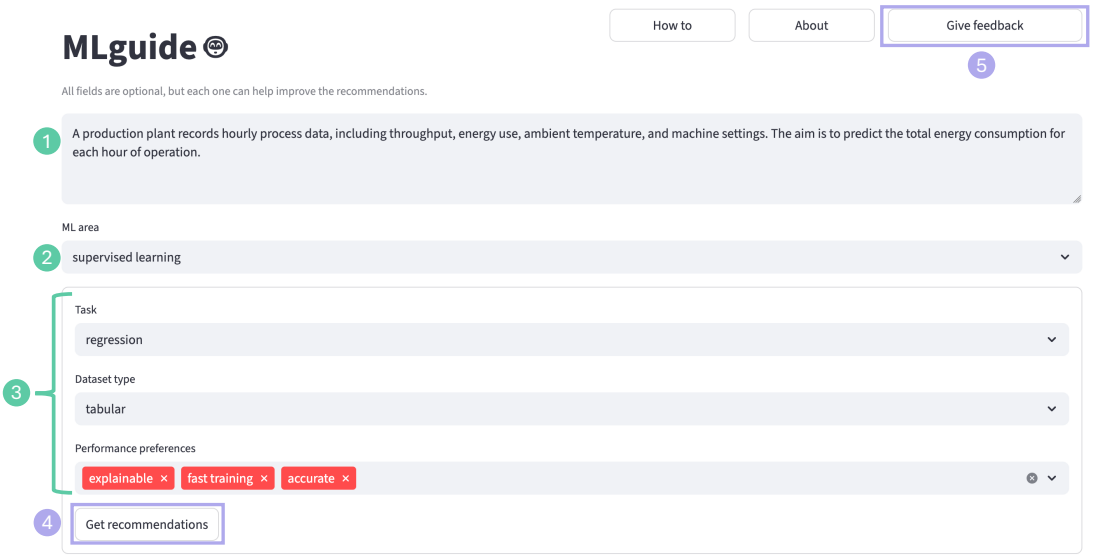

Figure 4.7.1: The input form on the home page, with an example problem description and a set of selected filters.

The ML area is selected next (2). It covers supervised, unsupervised, and reinforcement learning. The ML area is the main filter and the only hard filter applied during recommendation, as detailed in Section 4.3. The remaining filters are task, dataset type, and performance preferences (3). The selected ML area impacts which tasks are listed, since tasks with no relation to the chosen area in the knowledge graph are not relevant to display. In the example shown, supervised learning is selected, with a regression task, a tabular dataset type, and three performance preferences. Several performance preferences can be selected at the same time.

Once the problem has been described, the recommendations are requested using the "Get recommendations" button (4). A separate "Give feedback" button (5) allows the user to submit feedback on the system, as specified in Section 3.5.1.

4.7.2 Recommendation Table

Once recommendations have been requested, the methods are presented in the recommendation table, which appears below the input form on the home page. The table is shown in Figure 4.7.2. The methods are ranked according to the recommendation process specified in Section 4.3.

Figure 4.7.2: The recommendation table, listing the candidate methods for the described problem.

Each row corresponds to one method (1). The Performance, Task, and Dataset columns are shown only when the corresponding filter has been selected in the input form (2). The Performance column lists the performance preferences that the method matches, while the Task and Dataset columns indicate whether the method fits the selected task and dataset type, according to the knowledge graph. The Article count column reports how many articles in the dataset are associated with the method, counted within the selected ML area when one has been chosen (3). The Note column displays the `possible_if` and `not_recommended_if` properties from the ontology. These are shown as informational notes only and are not used for filtering or ranking (see Section 4.3.1 for the reasoning behind this) (4). Each method name is a button, and selecting it opens the method details page for that method (5).

4.7.3 Method Details Page

As mentioned above, when a method is clicked from the recommendation table, the method details page is opened for that specific method. The page is shown for Random Forest in Figure 4.7.3.

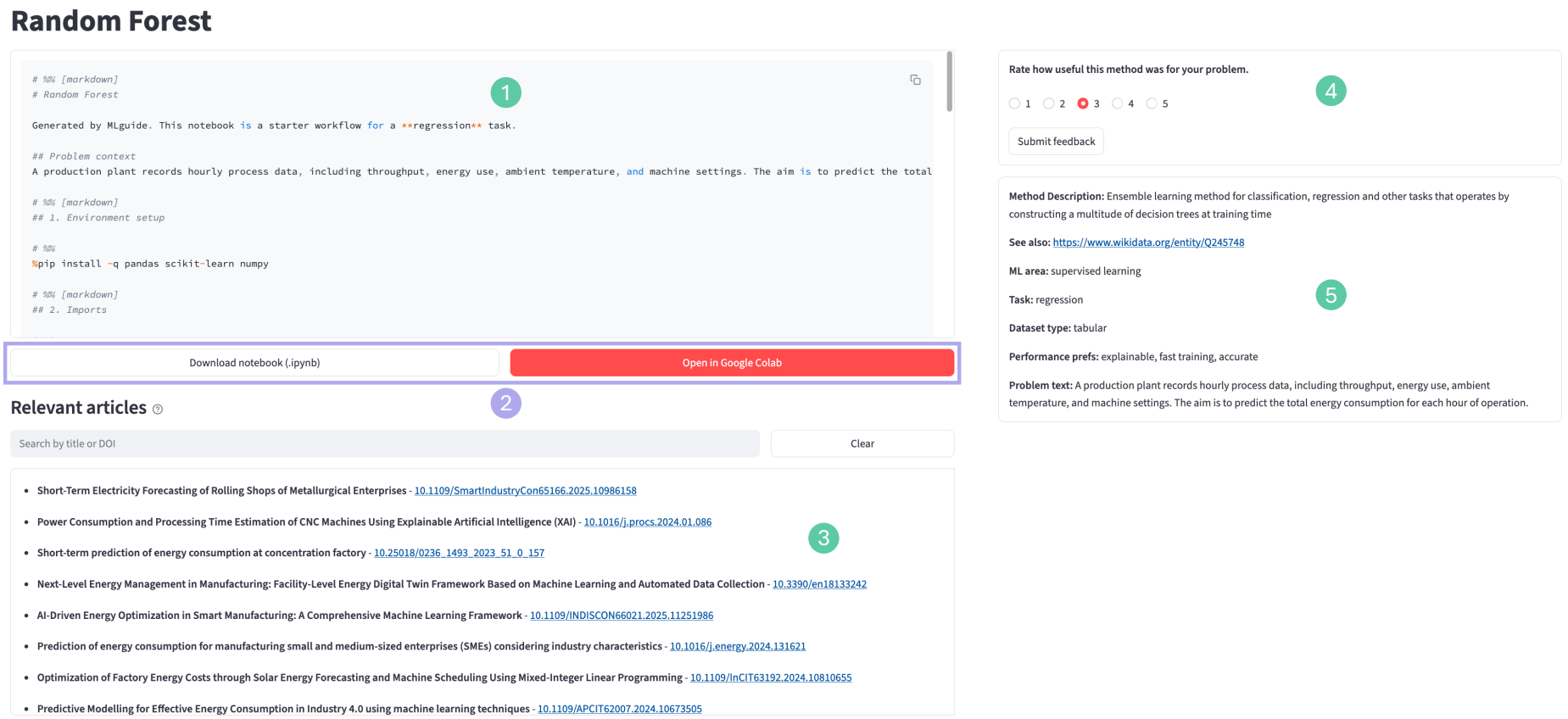

Figure 4.7.3: The method details page for the Random Forest method, showing the generated notebook, relevant articles, feedback option, and method information.

For supported methods and task families, the page provides a starter notebook (1). The notebook in this case is specific for Random Forest and regression, and is selected correctly by the process explained in Section 4.4.4. The notebook can be downloaded as an `.ipynb` file, and opened directly in Google Colab (2). A list of up to ten relevant articles is also shown, ranked by their similarity to the problem description. Each article is shown with a clickable DOI link and can be searched by title or DOI (3).

The user can rate how useful the method was for the described problem and submit this rating as feedback (4). The rating can be used to adjust method ranking over time, as outlined in Section 4.3.4. An information panel presents data about the method from the knowledge graph (5). This includes a short description from the `rdfs:comment` property and a link to an external reference. The other data is just for the user to have context available from the input form.

4.8 Changes Across System Versions

The development of MLguide was incremental and iterative, as described in Section 3.4. This form of development naturally produced different versions of the system at different points in time, as shown by the development timeline in Section 3.6. This section summarises the central differences between the three versions that were evaluated. These differences are needed to interpret the results and discussion chapters, since the two user tests and the example-based evaluation were carried out on different versions of the system.

As the development timeline in Figure 3.6.1 shows, the user testing took place some way into the development period. By the time of the first user test, all main functionality was in place: the input form, the recommendation output, the generated starter notebooks for the methods these are available for, the display of relevant articles to a specific method, and the "give feedback" button. Having this functionality running was a priority in order to reach the first user test. The differences between the three versions therefore lie mainly in the content of the knowledge graph and in smaller user interface details, rather than in the core functionality.

Version A, as described above, had the core functionality in place. It also included the extension of classes and concepts in the ontology, described in Section 4.2.1. The ML methods (`ML_approach` instances) used in article retrieval that were missing from the original ontology were added for version A. But at this point, neither the new nor the existing methods had their properties updated. The added methods carried little beyond an article link, and the existing methods had the properties of the original ontology, summarised by the property coverage in Table 2.3.2. Version A had the same user interface as the final version, as described in Section 4.7, with only minor parts not yet implemented.

Version B added the reinforcement learning methods and tasks, with properties, since reinforcement learning was entirely absent from the original ontology. It also simplified the input form and recommendation process by removing the filters for task, data type, and performance preferences, leaving only the problem text input

and the ML area filter.

Version C populated multiple methods with properties, adding 116 triples with performance attributes to ML approaches, and 248 `used_for` links between approaches and tasks, thereby updating the knowledge available to the system. This resulted in the coverage shown in Table 4.2.3. Version C also reintroduced the filters that had been removed from the input form in Version B, and added the feedback rating option on individual methods, as shown in Figure 4.7.3. The feedback can affect recommendations, as described in Section 4.3.4.

Table 4.8.1 summarises the main changes in each of the three versions.

Table 4.8.1: Summary of what was added or changed in each evaluated version of MLguide.

Version	Added or changed
Version A	• Core functionality in place. • Ontology extended with new classes and concepts. • Many methods still without properties beyond an article link.
Version B	• Reinforcement learning knowledge added. • Input form simplified to include problem description text and ML area filter only.
Version C	• Several performance attributes and `used_for` links added to methods, improving the system knowledge. • Task, data type and performance filters reintroduced in input form. • Method-level feedback added.

CHAPTER

FIVE

RESULTS

This chapter presents the results of the evaluation of MLguide. As described in Section 3.5, MLguide was evaluated in two ways: through user testing with students from the course TMM4128 Machine Learning for Engineers, referred to as Project 1 and Project 2, and through an example-based evaluation using a set of constructed problem cases.

Because MLguide was developed iteratively, the three evaluations were carried out on different versions of the system, as shown in Section 3.6. Project 1 used version A, Project 2 used version B, and the example-based evaluation used version C, the final version. The differences between the versions are summarised in Section 4.8.

Sections 5.1 and 5.2 present the results from the two user testing projects. Section 5.3 presents the results of the example-based evaluation.

Although MLguide includes a feedback form, it produced too few responses to serve as a usable basis for evaluation. The user testing results reported here are therefore only from the students' project reports. Since the reports were not written to a fixed structure, these results were obtained by interpreting each group's description of its use of MLguide.

5.1 Project 1 (P1): Unsupervised Learning and Anomaly Detection for Time Series Data

In the first project, the groups worked on detecting anomalies in time series data using unsupervised learning. As part of the assignment, the groups were instructed to try MLguide for their selected problem, and to assess the relevance and usefulness of the results in their project reports. The results below are based on these reports.

Of the 17 groups in the project, 13 used MLguide and 4 did not, as shown in Figure 5.1.1. The results that follow are based on the 13 groups that used the system.

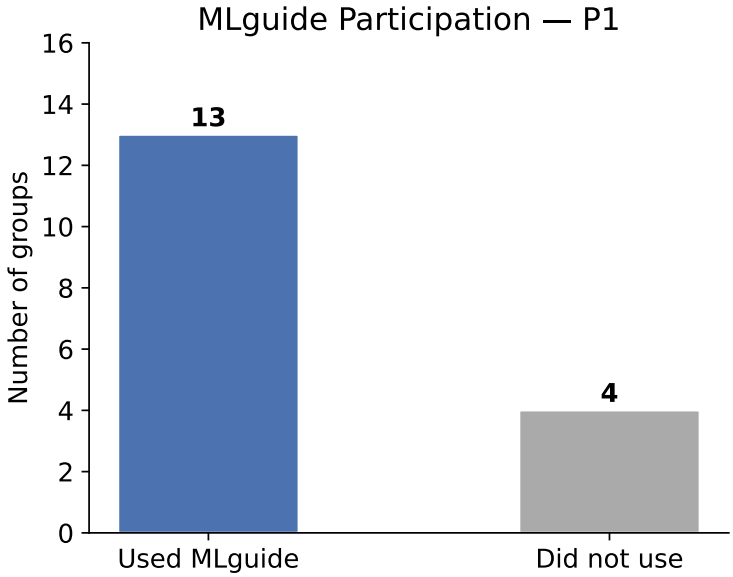

Figure 5.1.1: Number of groups in Project 1 that used MLguide and number that did not.

The reports also indicated when in the project MLguide was used. Figure 5.1.2 shows the result. Seven groups used the system late, after they had already chosen methods, and six used it early, before choosing methods.

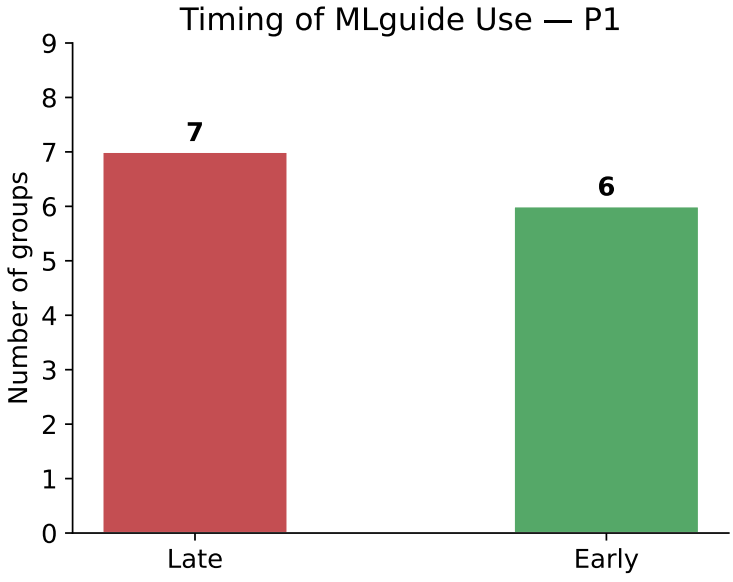

Figure 5.1.2: Number of groups that used MLguide early, before choosing methods, and late, after choosing methods.

For each group, the methods recommended by MLguide were compared with the methods the group used. The overlap was measured as the share of the group's chosen methods that MLguide also recommended, and classified as high (at least 50%), medium (33–50%), or low (below 33%). Figure 5.1.3 shows the distribution.

Of the twelve groups that could be assessed, six had high overlap, five had medium overlap, and one had low overlap.

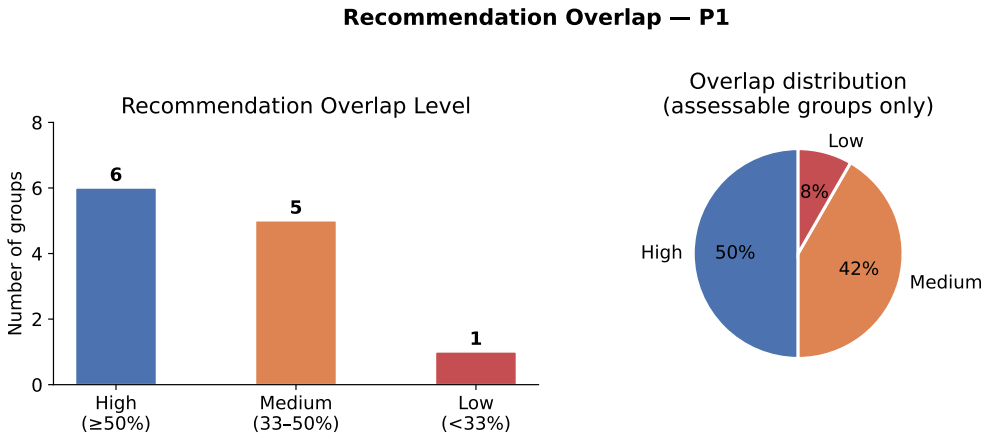

Figure 5.1.3: Distribution of the overlap between the methods recommended by MLguide and the methods used by the groups, classified as high (≥50%), medium (33–50%), and low (<33%).

Figure 5.1.4 combines the overlap level with the timing of use. Among the groups that used MLguide early, three had high overlap, two had medium overlap, and one had low overlap. Among the groups that used it late, three had high overlap and three had medium overlap.

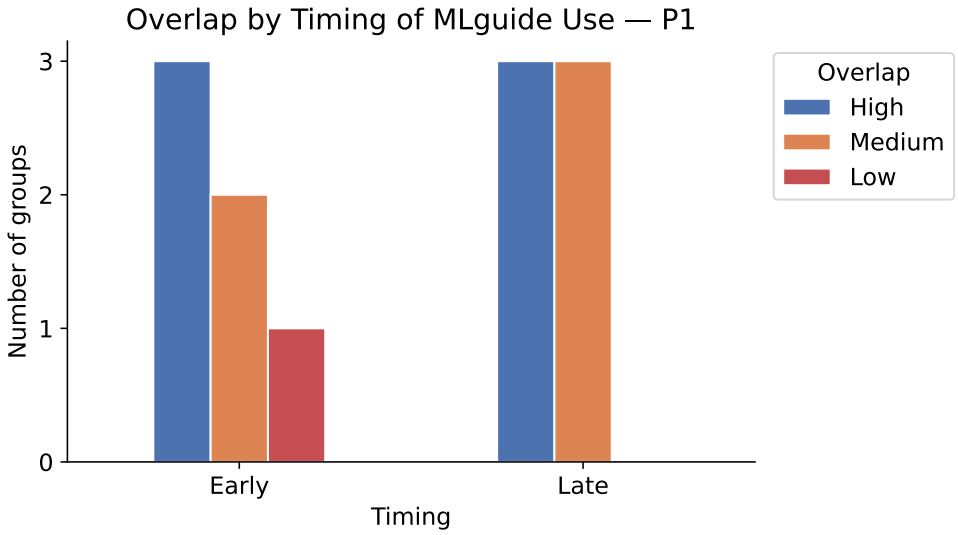

Figure 5.1.4: Overlap level between recommended and used methods, split by whether the group used MLguide early or late.

Figure 5.1.5 compares, for each method, the number of groups that were recommended the method with the number of groups that used it. Some methods were recommended more often than they were used, such as Neural Network, PCA and LSTM. Others were used more often than they were recommended, most clearly

Isolation Forest and Gaussian Mixture Model (GMM). GMM, One-Class SVM, and DBSCAN were used by several groups without being recommended to any of them.

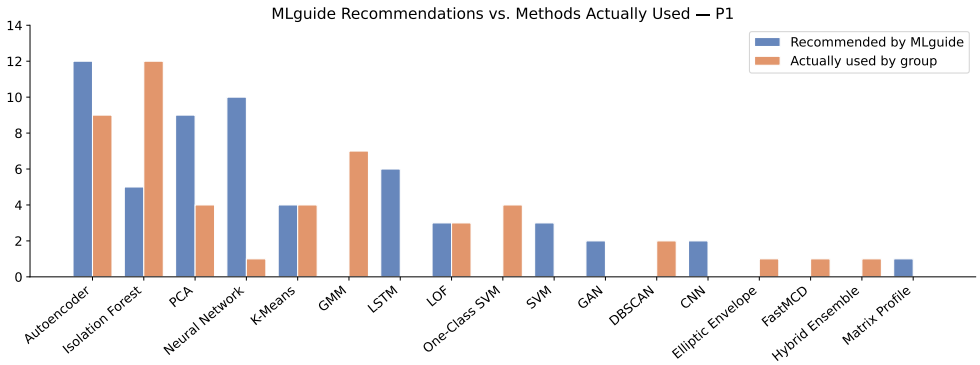

Figure 5.1.5: Number of groups recommended each method by MLguide compared with the number of groups that used it.

The reports were also examined for benefits and issues that the groups raised when describing their use of MLguide. The points were sorted into themes, and each theme was counted by the number of groups that raised it. Figure 5.1.6 shows the reported benefits. The most common were method shortlisting and early orientation, and validation or confirmation of method choices, each raised by seven groups. Code templates and implementation hints, and a narrowed search space, were each raised by two groups.

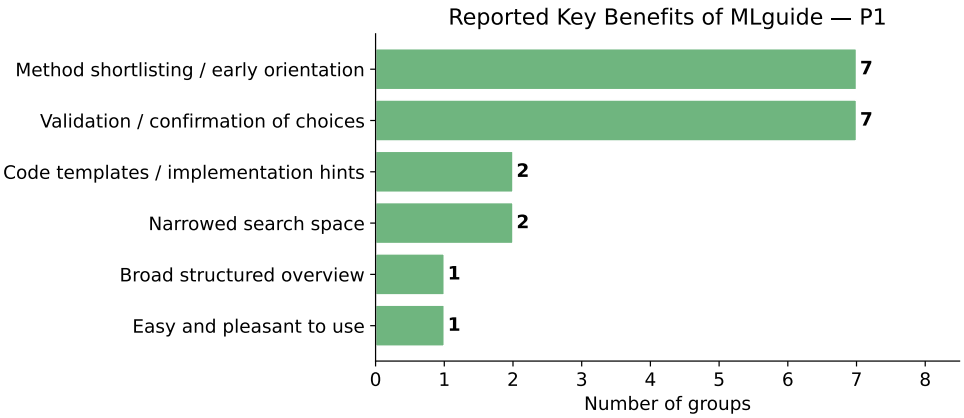

Figure 5.1.6: Benefits of using MLguide reported in the project reports, counted by the number of groups that raised each theme.

Figure 5.1.7 shows the reported issues. The most common was that MLguide was used too late to influence method decisions, raised by six groups. Five groups reported that GMM was missing from the recommendations. Three groups each reported that One-Class SVM was missing, that the recommendations were too generic or lacked ranking, and that the performance preferences were not useful in practice.

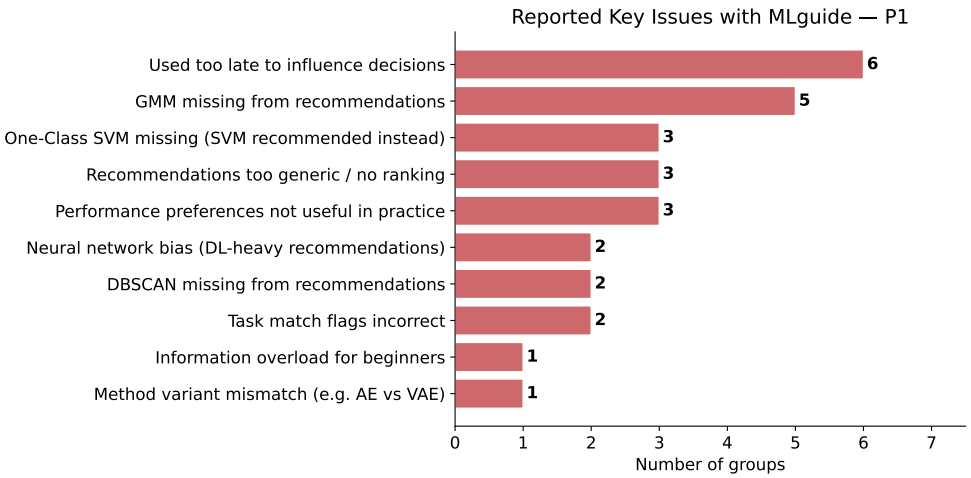

Figure 5.1.7: Issues with MLguide reported in the project reports, counted by the number of groups that raised each theme.

5.2 Project 2 (P2): Reinforcement Learning Across Product Lifecycle

The second project asked the groups to analyse applications of reinforcement learning across four product lifecycle phases, and to implement a simulated RL environment with a sustainability agent. The groups were asked to assess the applicability of MLguide to their project work and to report on it. The results below are based on the project reports.

Of the 19 groups in the project, 9 used MLguide and 10 did not. The results that follow are based on the 9 groups that used the system. Figure 5.2.1 shows the participation in Project 2 alongside Project 1, where a larger share of the groups used the system.

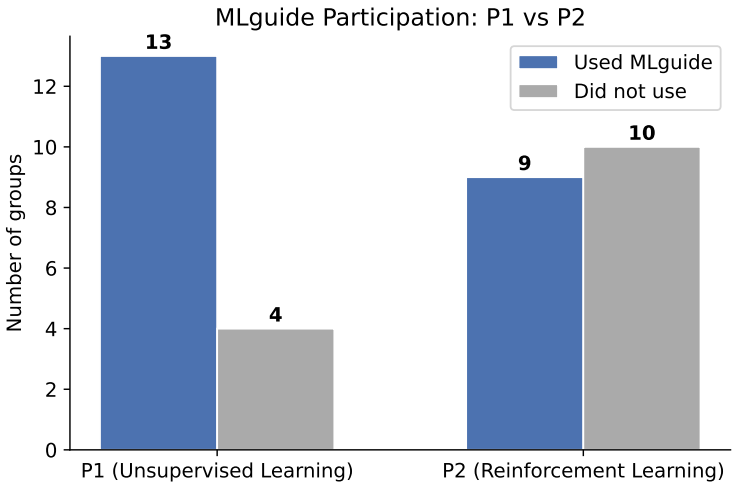

Figure 5.2.1: Number of groups that used MLguide and number that did not, in Project 1 and Project 2.

The reports were examined for how the groups used MLguide. Figure 5.2.2 shows the result. Five groups used only the recommended methods, three used both the methods and the supporting articles, and for one group the use was not specified.

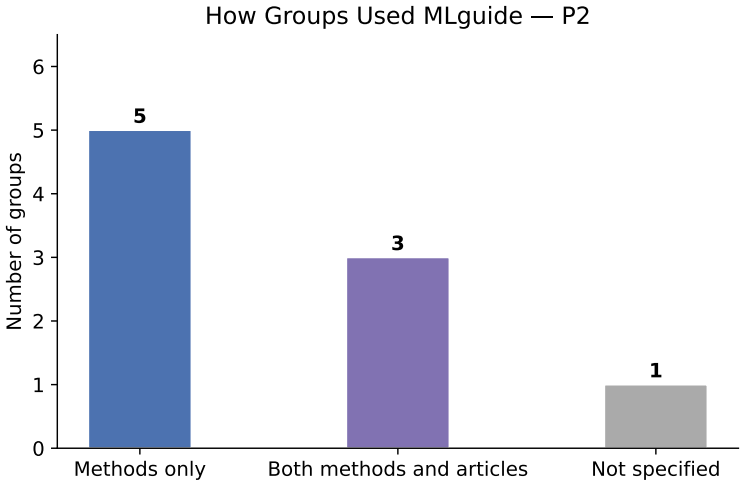

Figure 5.2.2: How the groups used MLguide, counted by the number of groups.

The reports were also examined for benefits that the groups raised when describing their use of MLguide. The points were sorted into themes, and each theme was counted by the number of groups that raised it. Figure 5.2.3 shows the result. The most common benefit was a good overview of relevant RL methods, raised by five groups. Confirmation that the tool was applicable to the problem, agreement between the recommendations and the literature, and help with finding or confirming relevant articles were each raised by three groups. One group reported that MLguide was sufficient for standard single-algorithm RL.

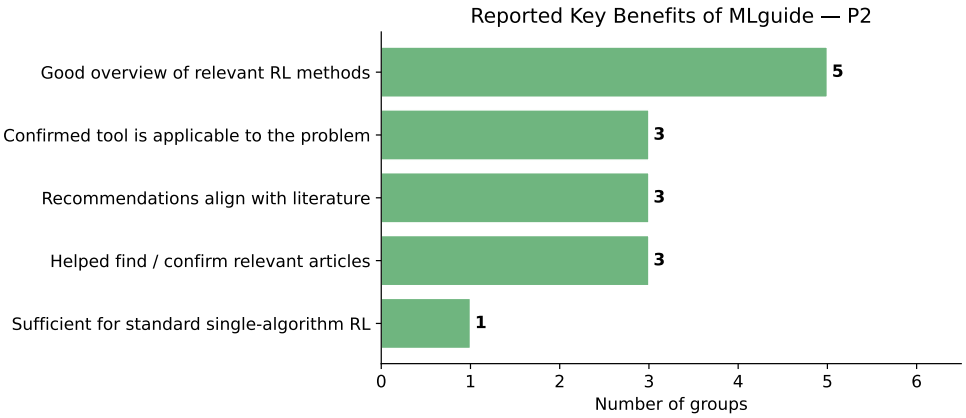

Figure 5.2.3: Key benefits reported by groups.

Figure 5.2.4 compares, for each method, the number of groups that were recommended the method with the number of groups that used it. DQN, Proximal Policy Optimization (PPO), and Q-learning were the most frequently used methods, and were also among the most frequently recommended. Deep RL and Neural

Network were recommended without being used by any group. PI-QT-Opt and NSGA-III were used by one group each without being recommended.

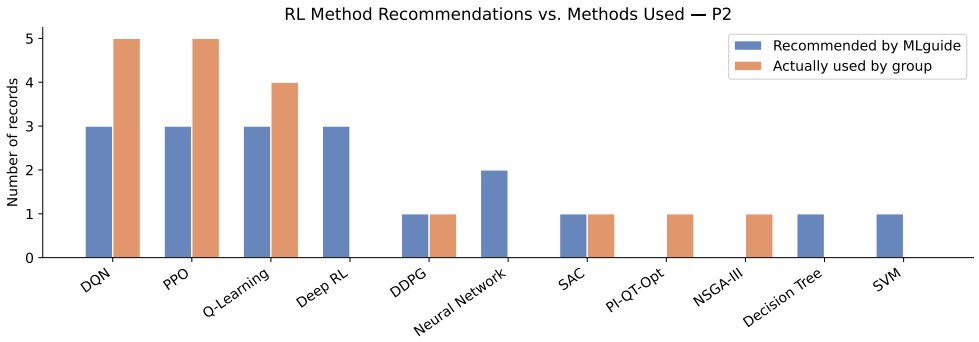

Figure 5.2.4: Methods Used vs. Recommended.

5.3 Example-Based Evaluation

CHAPTER

SIX

DISCUSSION

6.0.1 Feedback and Limitations

Skrives om

The feedback from both projects gave useful insight into the usability of MLguide and what could be improved. However, two limitations applied.

The first concerned the feedback format. Feedback was collected as part of the students' project reports, which fit naturally within the course structure and allowed students to reflect on MLguide in the context of their own problem and data. Since each group worked on a different problem, an open format seemed more suitable than a fixed set of questions. The trade-off was that responses varied in detail and focus, making it more difficult to draw systematic conclusions.

The second limitation was that the feedback arrived quite late in the development period. This was probably unavoidable, as a working system had to be in place before it could be tested. The feedback led to some adjustments, but larger changes such as new features or structural redesigns were not feasible within the remaining time. These can instead be left as directions for future work.

CHAPTER

SEVEN

CONCLUSIONS

DECLARATION OF AI USE

This chapter outlines how AI tools were used during the work on this thesis. The tools were used as assistants for writing and development, not as generators of content, arguments, or results. All output was reviewed, edited, and assessed before being included in the thesis or the implementation.

Writing

Claude Opus 4.7 was used as an assistant during the writing process. It was used on text that had already been written by the author, with the aim of improving structure, clarity, and the order in which points were presented. It was also used to suggest alternative and more varied phrasings when the original wording was repetitive or unclear. No sections or paragraphs were completely generated by the tool. All suggested changes were reviewed and edited by the author before being included.

Code and Development

Two AI coding assistants were used during the development of MLguide: Codex (GPT-5.4) and Claude Code (Claude Sonnet 4.6). They were used to support development and debugging, including help with resolving specific implementation problems and identifying causes of errors in existing code. They were also used to help maintain consistency with the project architecture that was defined at the start of development, so that new code followed the same structure and conventions as the rest of the system. The tools were also used to assist with writing SQL and SPARQL queries, and with writing code for Matplotlib visualisations used in the thesis. Code suggestions and proposed changes were not accepted automatically. Each suggestion was reviewed and assessed by the author, and adjusted or rejected where needed.

Literature Search

In addition to standard academic research databases, the AI-based tools ResearchRabbit and Consensus were used to help discover relevant literature and

follow citation trails. The final selection of sources was made by the author after reviewing the relevant material.

REFERENCES

Ali, Rahman, Sungyoung Lee, and Tae Choong Chung (2017). "Accurate multi-criteria decision making methodology for recommending machine learning algorithm". In: *Expert Systems with Applications* 71, pp. 257–278. ISSN: 0957-4174. DOI: 10.1016/j.eswa.2016.11.034.

Alzubaidi, Laith et al. (2021). "Review of deep learning: concepts, CNN architectures, challenges, applications, future directions". In: *Journal of Big Data* 8.1, p. 53. ISSN: 2196-1115. DOI: 10.1186/s40537-021-00444-8.

Arena, Simone et al. (2024). "A conceptual framework for machine learning algorithm selection for predictive maintenance". In: *Engineering Applications of Artificial Intelligence* 133, p. 108340. ISSN: 0952-1976. DOI: 10.1016/j.engappai.2024.108340.

Arnott, David and Graham Pervan (2012). "Design Science in Decision Support Systems Research: An Assessment using the Hevner, March, Park, and Ram Guidelines". In: *Journal of the Association of Information Systems* 13, pp. 923–949. DOI: 10.17705/1jais.00315.

Bajaj, Payal et al. (2018). *MS MARCO: A Human Generated MAchine Reading COmprehension Dataset*. arXiv: 1611.09268 [cs.CL]. URL: https://arxiv.org/abs/1611.09268.

Barbudo, Rafael, Sebastián Ventura, and José Raúl Romero (2023). "Eight years of AutoML: categorisation, review and trends". In: *Knowledge and Information Systems* 65.12, pp. 5097–5149. ISSN: 0219-3116. DOI: 10.1007/s10115-023-01935-1.

Bechhofer, Sean (2009). "OWL: Web Ontology Language". In: *Encyclopedia of Database Systems*. Ed. by Ling Liu and M. Tamer ÖZsu. Boston, MA: Springer US, pp. 2008–2009. ISBN: 978-0-387-39940-9. DOI: 10.1007/978-0-387-39940-9_1073.

Berenguer, Alberto, Jose-Norberto Mazón, and David Tomás (2024). "Word embeddings for retrieving tabular data from research publications". In: *Machine Learning* 113.4, pp. 2227–2248. ISSN: 1573-0565. DOI: 10.1007/s10994-023-06472-0.

Bergstø, Martin (Jan. 2026). "Supporting Method Selection for Machine Learning in Product Development and Production: A Multi-Dimensional Classification of Machine Learning Research". In: Specialisation Project.

Bishop, Christopher M. (2006). *Pattern Recognition and Machine Learning*. Information Science and Statistics. New York: Springer. ISBN: 978-0387310732.

Blagec, Kathrin et al. (2022). "A curated, ontology-based, large-scale knowledge graph of artificial intelligence tasks and benchmarks". In: *Scientific Data* 9.1, p. 322. ISSN: 2052-4463. DOI: 10.1038/s41597-022-01435-x.

Bobadilla, J. et al. (2013). "Recommender systems survey". In: *Knowledge-Based Systems* 46, pp. 109–132. ISSN: 0950-7051. DOI: 10.1016/j.knosys.2013.03.012.

Breiman, Leo (2001). "Random Forests". In: *Machine Learning* 45.1, pp. 5–32. ISSN: 1573-0565. DOI: 10.1023/A:1010933404324.

Chen, Tianqi and Carlos Guestrin (Aug. 2016). "XGBoost: A Scalable Tree Boosting System". In: *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. ACM, pp. 785–794. DOI: 10.1145/2939672.2939785.

Chen, Tingting et al. (2023). *Machine Learning in Manufacturing towards Industry 4.0: From 'For Now' to 'Four-Know'*. Electronic Article. DOI: 10.3390/app13031903.

Colangelo, Maria Teresa et al. (2025). "A Comparative Analysis of Sentence Transformer Models for Automated Journal Recommendation Using PubMed Metadata". In: *Big Data and Cognitive Computing* 9.3, p. 67. ISSN: 2504-2289. URL: https://www.mdpi.com/2504-2289/9/3/67.

Cormack, Gordon V., Charles L A Clarke, and Stefan Buettcher (2009). *Reciprocal rank fusion outperforms condorcet and individual rank learning methods*. Conference Paper. DOI: 10.1145/1571941.1572114.

Cortes, Corinna and Vladimir Vapnik (1995). "Support-vector networks". In: *Machine Learning* 20.3, pp. 273–297. ISSN: 1573-0565. DOI: 10.1007/BF00994018.

Cunningham, Pádraig and Sarah Jane Delany (July 2021). "k-Nearest Neighbour Classifiers - A Tutorial". In: *ACM Computing Surveys* 54.6, pp. 1–25. ISSN: 1557-7341. DOI: 10.1145/3459665.

Dellermann, Dominik et al. (2018). "Design principles for a hybrid intelligence decision support system for business model validation". In: *Electronic Markets* 29, pp. 1–19. DOI: 10.1007/s12525-018-0309-2.

Docker Inc. (2026). *Docker*. URL: https://www.docker.com.

Dogan, Alican and Derya Birant (2021). "Machine learning and data mining in manufacturing". In: *Expert Systems with Applications* 166, p. 114060. ISSN: 0957-4174. DOI: 10.1016/j.eswa.2020.114060.

Dorkin, Aleksei and Kairit Sirts (2025). *TartuNLP at SemEval-2025 Task 5: Subject Tagging as Two-Stage Information Retrieval*. arXiv: 2504.21547 [cs.CL]. URL: https://arxiv.org/abs/2504.21547.

Elsevier (2026). *Scopus API*. URL: https://api.elsevier.com.

Ester, Martin et al. (1996). *A density-based algorithm for discovering clusters in large spatial databases with noise*. Conference Paper.

Esteves, Diego et al. (2015). "MEX vocabulary: a lightweight interchange format for machine learning experiments". In: *Proceedings of the 11th International Conference on Semantic Systems*. ACM, pp. 169–176.

Galli, Carlo, Nikolaos Donos, and Elena Calciolari (2024). "Performance of 4 Pre-Trained Sentence Transformer Models in the Semantic Query of a Systematic Review Dataset on Peri-Implantitis". In: *Information* 15.2, p. 68. ISSN: 2078-2489. URL: https://www.mdpi.com/2078-2489/15/2/68.

Geron, Aurelien (2019). *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow: Concepts, Tools, and Techniques to Build Intelligent Systems*. O'Reilly Media, Inc. ISBN: 1492032646.

Gerschütz, Benjamin, Stefan Goetz, and Sandro Wartzack (2023). "AI4PD—Towards a Standardized Interconnection of Artificial Intelligence Methods with Product Development Processes". In: *Applied Sciences* 13.5, p. 3002. ISSN: 2076-3417. URL: https://www.mdpi.com/2076-3417/13/5/3002.

Gerschütz, Benjamin, Benjamin Schleich, and Sandro Wartzack (2021). *A semantic web approach for structuring data-driven methods in the product development process*. DOI: 10.35199/dfx2021.15.

Gibbins, Nicholas and Nigel Shadbolt (2009). "Resource Description Framework (RDF)". In: ISSN: 978-1-4398-9196-4. DOI: 10.1201/b11499-53.

Goodfellow, Ian, Yoshua Bengio, and Aaron Courville (2016). *Deep Learning*. http://www.deeplearningbook.org. MIT Press.

Grobe, Michael (2009). *RDF, Jena, SparQL and the 'Semantic Web'*. Conference Paper. DOI: 10.1145/1629501.1629525.

Gruber, Thomas R. (1995). "Toward principles for the design of ontologies used for knowledge sharing?" In: *International Journal of Human-Computer Studies* 43.5, pp. 907–928. ISSN: 1071-5819. DOI: 10.1006/ijhc.1995.1081.

He, Xin, Kaiyong Zhao, and Xiaowen Chu (2021). "AutoML: A survey of the state-of-the-art". In: *Knowledge-Based Systems* 212, p. 106622. ISSN: 0950-7051. DOI: 10.1016/j.knosys.2020.106622.

Hevner, Alan et al. (2004). "Design Science in Information Systems Research". In: *Management Information Systems Quarterly* 28, p. 75.

Hochreiter, Sepp and Jürgen Schmidhuber (1997). "Long Short-Term Memory". In: *Neural Computation* 9, pp. 1735–1780. DOI: 10.1162/neco.1997.9.8.1735.

Hotelling, H. (1932). "Analysis of a complex of statistical variables into principal components". In: *Journal of Educational Psychology* 24, pp. 417–441. DOI: 10.1037/h0071325.

Humeau, Samuel et al. (2020). *Poly-encoders: Transformer Architectures and Pre-training Strategies for Fast and Accurate Multi-sentence Scoring*. arXiv: 1905.01969 [cs.CL]. URL: https://arxiv.org/abs/1905.01969.

Humm, Bernhard (2026). "An Industry-Ready Machine Learning Ontology". In: *Applied Sciences* 16, p. 843. DOI: 10.3390/app16020843.

Jurafsky, Daniel and James H. Martin (2026). *Speech and Language Processing: An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition with Language Models*. 3rd. Online manuscript released January 6, 2026. URL: https://web.stanford.edu/~jurafsky/slp3/.

Kane, Andrew (2025). *pgvector: Open-source vector similarity search for Postgres*. URL: https://github.com/pgvector/pgvector.

Keet, C. Maria et al. (2015). "The Data Mining OPtimization Ontology". In: *Journal of Web Semantics* 32, pp. 43–53. ISSN: 1570-8268. DOI: 10.1016/j.websem.2015.01.001.

Kluyver, Thomas et al. (2016). *Jupyter Notebooks – a publishing format for reproducible computational workflows*.

Kumar, Kapil (Dec. 2024). *The Illustrated Guide to Cross-Encoders: From Deep to Shallow*. https://medium.com/@kakumar1611/the-illustrated-guide-

to-cross-encoders-from-deep-to-shallow-2a23a8630016. Accessed: 2025-05-11.

Landwehr, Julius et al. (2022). "Design Knowledge for Deep-Learning-Enabled Image-Based Decision Support Systems: Evidence From Power Line Maintenance Decision-Making". In: *Business & Information Systems Engineering* 64. DOI: 10.1007/s12599-022-00745-z.

Lecun, Y. et al. (1998). "Gradient-based learning applied to document recognition". In: *Proceedings of the IEEE* 86.11, pp. 2278–2324. ISSN: 1558-2256. DOI: 10.1109/5.726791.

LeCun, Yann, Yoshua Bengio, and Geoffrey Hinton (2015). "Deep learning". In: *Nature* 521.7553, pp. 436–444. ISSN: 1476-4687. DOI: 10.1038/nature14539.

Lee, In and Yong Jae Shin (2020). "Machine learning for enterprises: Applications, algorithm selection, and challenges". In: *Business Horizons* 63.2, pp. 157–170. ISSN: 0007-6813. DOI: 10.1016/j.bushor.2019.10.005.

Lickert, Hannah et al. (2021). "Selection of Suitable Machine Learning Algorithms for Classification Tasks in Reverse Logistics". In: *Procedia CIRP* 96, pp. 272–277. ISSN: 2212-8271. DOI: https://doi.org/10.1016/j.procir.2021.01.086.

Liu, F. T., K. M. Ting, and Z. H. Zhou (2008). "Isolation Forest". In: *2008 Eighth IEEE International Conference on Data Mining*, pp. 413–422. ISBN: 2374-8486. DOI: 10.1109/ICDM.2008.17.

Microsoft (n.d.). *Machine Learning Algorithm Cheat Sheet for Azure Machine Learning*. Accessed: May 2026. URL: https://learn.microsoft.com/en-us/azure/machine-learning/algorithm-cheat-sheet.

Mitchell, Tom M. (1997). *Machine Learning*. New York: McGraw-Hill.

Mongeon, Philippe and Adèle Paul-Hus (2016). "The journal coverage of Web of Science and Scopus: a comparative analysis". In: *Scientometrics* 106.1, pp. 213–228. ISSN: 1588-2861. DOI: 10.1007/s11192-015-1765-5.

Murphy, Kevin P. (2012). *Machine learning: a probabilistic perspective*. Adaptive computation and machine learning. Cambridge: MIT Press. ISBN: 0262018020.

Nogueira, Rodrigo and Kyunghyun Cho (2020). *Passage Re-ranking with BERT*. arXiv: 1901.04085 [cs.IR]. URL: https://arxiv.org/abs/1901.04085.

Noy, N. and Deborah McGuinness (2001). "Ontology Development 101: A Guide to Creating Your First Ontology". In: *Knowledge Systems Laboratory* 32.

Nti, Isaac Kofi et al. (2022). "Applications of artificial intelligence in engineering and manufacturing: a systematic review". In: *Journal of Intelligent Manufacturing* 33.6, pp. 1581–1601. ISSN: 1572-8145. DOI: 10.1007/s10845-021-01771-6.

Ontotext (2025). *GraphDB*. URL: https://graphdb.ontotext.com.

Ostendorff, Malte et al. (2022). *Specialized document embeddings for aspect-based similarity of research papers*. Conference Paper. DOI: 10.1145/3529372.3530912. URL: https://doi.org/10.1145/3529372.3530912.

Paleyes, Andrei, Raoul-Gabriel Urma, and Neil D. Lawrence (2022). "Challenges in Deploying Machine Learning: A Survey of Case Studies". In: *ACM Comput. Surv.* 55.6, Article 114. ISSN: 0360-0300. DOI: 10.1145/3533378.

Panov, Pance, Larisa Soldatova, and Sašo Džeroski (2014). "Ontology of core data mining entities". In: *Data Mining and Knowledge Discovery* 28, pp. 1222–1265. DOI: 10.1007/s10618-014-0363-0.

Panzer, Marcel and Benedict Bender (2022). "Deep reinforcement learning in production systems: a systematic literature review". In: *International Journal of Production Research* 60.13, pp. 4316–4341. ISSN: 0020-7543. DOI: 10.1080/00207543.2021.1973138.

Park, Sangwoo et al. (2026). *Chain of Retrieval: Multi-Aspect Iterative Search Expansion and Post-Order Search Aggregation for Full Paper Retrieval*. arXiv: 2507.10057 [cs.IR]. URL: https://arxiv.org/abs/2507.10057.

Peffers, Ken et al. (2007). "A design science research methodology for information systems research". In: *Journal of Management Information Systems* 24, pp. 45–77.

Plathottam, Siby Jose et al. (2023). "A review of artificial intelligence applications in manufacturing operations". In: *Journal of Advanced Manufacturing and Processing* 5.3, e10159. DOI: 10.1002/amp2.10159.

Pouyanfar, Samira et al. (2018). "A Survey on Deep Learning: Algorithms, Techniques, and Applications". In: *ACM Comput. Surv.* 51.5, Article 92. ISSN: 0360-0300. DOI: 10.1145/3234150.

Presciuttini, Anna et al. (2024). "Machine learning applications on IoT data in manufacturing operations and their interpretability implications: A systematic literature review". In: *Journal of Manufacturing Systems* 74, pp. 477–486. ISSN: 0278-6125. DOI: 10.1016/j.jmsy.2024.04.012.

Publio, Gustavo et al. (2018). *ML-Schema: Exposing the Semantics of Machine Learning with Schemas and Ontologies*. DOI: 10.48550/arXiv.1807.05351.

Rai, Rahul et al. (2021). "Machine learning in manufacturing and industry 4.0 applications". In: *International Journal of Production Research* 59.16, pp. 4773–4778. ISSN: 0020-7543. DOI: 10.1080/00207543.2021.1956675.

Ramírez, Sebastián (2026). *FastAPI*. URL: https://fastapi.tiangolo.com.

Reimers, Nils and Iryna Gurevych (2019). *Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks*. arXiv: 1908.10084 [cs.CL]. URL: https://arxiv.org/abs/1908.10084.

— (n.d.[a]). *MS MARCO Cross-Encoders*. https://huggingface.co/cross-encoder/ms-marco-MiniLM-L6-v2. Accessed: 2025-03-01.

— (n.d.[b]). *sentence-transformers/all-mpnet-base-v2*. https://huggingface.co/sentence-transformers/all-mpnet-base-v2. Accessed: 2025-02-01.

Ricci, Francesco, Lior Rokach, and Bracha Shapira (2010). "Recommender Systems Handbook". In: vol. 1-35, pp. 1–35. ISBN: 978-0-387-85819-7. DOI: 10.1007/978-0-387-85820-3_1.

Rieke, Johannes (2021). *Traingenerator: A Web App to Generate Template Code for Machine Learning*. https://github.com/jrieke/traingenerator. Accessed: 2026-05-12.

Riesener, Michael et al. (2020). "Identification of evaluation criteria for algorithms used within the context of product development". In: *Procedia CIRP* 91, pp. 508–515. ISSN: 2212-8271. DOI: https://doi.org/10.1016/j.procir.2020.02.207.

Rose, Michael E. and John R. Kitchin (2019). "pybliometrics: Scriptable bibliometrics using a Python interface to Scopus". In: *SoftwareX* 10, p. 100263. ISSN: 2352-7110. DOI: https://doi.org/10.1016/j.softx.2019.100263.

Sala, Roberto et al. (2018). "How to select a suitable machine learning algorithm: A feature-based, scope-oriented selection framework". In: URL: https://api.semanticscholar.org/CorpusID:165031947.

Sarker, Iqbal H. (2021). "Machine Learning: Algorithms, Real-World Applications and Research Directions". In: *SN Computer Science* 2.3, p. 160. ISSN: 2661-8907. DOI: 10.1007/s42979-021-00592-x.

Schopf, Tim and Florian Matthes (2024). *NLP-KG: A System for Exploratory Search of Scientific Literature in Natural Language Processing*. arXiv: 2406.15294 [cs.CL]. URL: https://arxiv.org/abs/2406.15294.

Scikit-learn developers (n.d.). *Choosing the right estimator*. Accessed: May 2026. URL: https://scikit-learn.org/stable/machine_learning_map.html.

Sentence-Transformers Documentation (2026). URL: https://www.sbert.net/ (visited on 05/11/2026).

Sewak, Mohit (2019). "Policy-Based Reinforcement Learning Approaches". In: *Deep Reinforcement Learning: Frontiers of Artificial Intelligence*. Ed. by Mohit Sewak. Singapore: Springer Singapore, pp. 127–140. ISBN: 978-981-13-8285-7. DOI: 10.1007/978-981-13-8285-7_10.

Shrestha, Ajay and Ausif Mahmood (2019). "Review of Deep Learning Algorithms and Architectures". In: *IEEE Access* PP, pp. 1–1. DOI: 10.1109/ACCESS.2019.2912200.

Sommerville, I. (2011). *Software Engineering*. Pearson. ISBN: 9780137053469. URL: https://books.google.no/books?id=l0egcQAACAAJ.

Song, Kaitao et al. (2020). *MPNet: Masked and Permuted Pre-training for Language Understanding*. arXiv: 2004.09297 [cs.CL]. URL: https://arxiv.org/abs/2004.09297.

Sonntag, Sebastian, Janosch Luttmer, et al. (2023). "A PATTERN LANGUAGE APPROACH TO IDENTIFY APPROPRIATE MACHINE LEARNING ALGORITHMS IN THE CONTEXT OF PRODUCT DEVELOPMENT". In: *Proceedings of the Design Society* 3, pp. 365–374. DOI: 10.1017/pds.2023.37.

Sonntag, Sebastian, Erik Pohl, et al. (2024). "A conceptual MCDA-based framework for machine learning algorithm selection in the early phase of product development". In: *Proceedings of the Design Society* 4, pp. 2257–2266. DOI: 10.1017/pds.2024.228.

Streamlit Inc. (2026). *Streamlit*. URL: https://streamlit.io.

Studer, Rudi, V. Richard Benjamins, and Dieter Fensel (1998). "Knowledge engineering: Principles and methods". In: *Data & Knowledge Engineering* 25.1, pp. 161–197. ISSN: 0169-023X. DOI: 10.1016/S0169-023X(97)00056-6.

Sutton, Richard S et al. (1999). "Policy Gradient Methods for Reinforcement Learning with Function Approximation". In: *Advances in Neural Information Processing Systems*. Ed. by S. Solla, T. Leen, and K. Müller. Vol. 12. MIT Press. URL: https://proceedings.neurips.cc/paper_files/paper/1999/file/464d828b85b0bed98e80ade0a5c43b0f-Paper.pdf.

Sutton, Richard S. and Andrew G. Barto (2015). *Reinforcement Learning: An Introduction*. 2nd. Second edition, in progress. Cambridge, MA: MIT Press. URL: https://web.stanford.edu/class/psych209/Readings/SuttonBartoIPRLBook2ndEd.pdf (visited on 05/27/2026).

Thakur, Nandan, Nils Reimers, Johannes Daxenberger, et al. (2021). *Augmented SBERT: Data Augmentation Method for Improving Bi-Encoders for Pairwise*

Sentence Scoring Tasks, pp. 296–310. DOI: 10.18653/v1/2021.naacl-main.28.

Thakur, Nandan, Nils Reimers, Andreas Rücklé, et al. (2021). *BEIR: A Heterogenous Benchmark for Zero-shot Evaluation of Information Retrieval Models*. arXiv: 2104.08663 [cs.IR]. URL: https://arxiv.org/abs/2104.08663.

The Pallets Team (2026). *Jinja, Version [3.1.6]*. URL: https://jinja.palletsprojects.com/.

The PostgreSQL Global Development Group (2025). *PostgreSQL*. URL: https://www.postgresql.org.

Tibshirani, Robert (1996). "Regression Shrinkage and Selection Via the Lasso". In: *Journal of the Royal Statistical Society: Series B (Methodological)* 58.1, pp. 267–288. ISSN: 0035-9246. DOI: 10.1111/j.2517-6161.1996.tb02080.x.

Troumpoukis, Antonis et al. (2025). "DeployAI: AI-on-Demand Platform and Marketplace for Industrial Applications within Europe". In: *Artificial Intelligence Applications and Innovations. AIAI 2025 IFIP WG 12.5 International Workshops*. Ed. by Antonios Papaleonidas et al. Springer Nature Switzerland, pp. 115–127. ISBN: 978-3-031-97317-8.

Vanschoren, Joaquin et al. (2012). "Experiment databases. A new way to share, organize and learn from experiments". In: *Machine Learning* 87, pp. 127–158.

Vaswani, Ashish et al. (2023). *Attention Is All You Need*. arXiv: 1706.03762 [cs.CL]. URL: https://arxiv.org/abs/1706.03762.

Verma, Shashank, Fengyi Jiang, and Xiangning Xue (2025). *Beyond Retrieval: Ensembling Cross-Encoders and GPT Rerankers with LLMs for Biomedical QA*. arXiv: 2507.05577 [cs.IR]. URL: https://arxiv.org/abs/2507.05577.

Waring, Jonathan, Charlotta Lindvall, and Renato Umeton (2020). "Automated machine learning: Review of the state-of-the-art and opportunities for healthcare". In: *Artificial Intelligence in Medicine* 104, p. 101822. ISSN: 0933-3657. DOI: 10.1016/j.artmed.2020.101822.

Watkins, Christopher J. C. H. and Peter Dayan (1992). "Q-learning". In: *Machine Learning* 8.3, pp. 279–292. ISSN: 1573-0565. DOI: 10.1007/BF00992698.

Weichert, Dorina et al. (2019). "A review of machine learning for the optimization of production processes". In: *The International Journal of Advanced Manufacturing Technology* 104.5, pp. 1889–1902. ISSN: 1433-3015. DOI: 10.1007/s00170-019-03988-5.

Westergaard, David et al. (2018). "A comprehensive and quantitative comparison of text-mining in 15 million full-text articles versus their corresponding abstracts". In: *PLOS Computational Biology* 14.2, e1005962. DOI: 10.1371/journal.pcbi.1005962.

Wikimedia Foundation (2012). *Wikidata*. URL: https://www.wikidata.org.

World Wide Web Consortium (2026). *World Wide Web Consortium (W3C)*. URL: https://www.w3.org/ (visited on 05/12/2026).

Wouts, Marc (2026). *Jupytext: Jupyter notebooks as Markdown documents, scripts or Rmd files*. Version 1.19.1. URL: https://github.com/mwouts/jupytext.

Wuest, Thorsten et al. (2016). "Machine learning in manufacturing: advantages, challenges, and applications". In: *Production & Manufacturing Research* 4.1, pp. 23–45. ISSN: null. DOI: 10.1080/21693277.2016.1192517.

Zender, Alexander and Bernhard Humm (2022). "Ontology-based Meta AutoML". In: *Integrated Computer-Aided Engineering* 29, pp. 1–16. DOI: 10.3233/ICA-220684.

Zhang, Hongming and Tianyang Yu (2020). "Taxonomy of Reinforcement Learning Algorithms". In: *Deep Reinforcement Learning: Fundamentals, Research and Applications*. Ed. by Hao Dong, Zihan Ding, and Shanghang Zhang. Singapore: Springer Singapore, pp. 125–133. ISBN: 978-981-15-4095-0. DOI: 10.1007/978-981-15-4095-0_3.

APPENDICES

APPENDIX

A

SCOPUS SEARCH QUERIES

Table A.1 lists the complete set of Scopus queries used by the article retrieval pipeline. Each query is defined in a YAML configuration file with an identifier, a short description, and a Scopus search string. All queries use the `TITLE-ABS-KEY` operator, which matches against article titles, abstracts, and keywords.

Table A.1: Scopus search queries used by the article retrieval pipeline.

Search string
`TITLE-ABS-KEY("machine learning" AND "in-service")`
`TITLE-ABS-KEY("machine learning" AND remanufacturing)`
`TITLE-ABS-KEY("machine learning" AND "disassembly")`
`TITLE-ABS-KEY("machine learning" AND "end-of-life" AND product)`
`TITLE-ABS-KEY("machine learning" AND recycling AND manufacturing)`
`TITLE-ABS-KEY("machine learning" AND sustainability AND product)`
`TITLE-ABS-KEY("machine learning" AND manufacturing)`
`TITLE-ABS-KEY("machine learning" AND "product development")`
`TITLE-ABS-KEY("machine learning" AND planning AND production)`
`TITLE-ABS-KEY("machine learning" AND "product design")`
`TITLE-ABS-KEY("machine learning" AND manufacturing AND optimization)`
`TITLE-ABS-KEY("machine learning" AND production AND optimization)`
`TITLE-ABS-KEY("machine learning" AND "predictive maintenance")`
`TITLE-ABS-KEY("machine learning" AND "anomaly detection" AND production)`

Continued on next page

Table A.1 – Continued from previous page

Search string
TITLE-ABS-KEY("machine learning" AND "process control")
TITLE-ABS-KEY("machine learning" AND "process modeling")
TITLE-ABS-KEY("machine learning" AND (RUL OR "remaining useful life"))
TITLE-ABS-KEY("machine learning" AND "defect detection" AND manufacturing)
TITLE-ABS-KEY("machine learning" AND (PLM OR "product lifecycle management"))
TITLE-ABS-KEY("machine learning" AND "quality control" AND manufacturing)
TITLE-ABS-KEY("machine learning" AND "process optimization" AND manufacturing)
TITLE-ABS-KEY("machine learning" AND "supply chain" AND production)
TITLE-ABS-KEY("machine learning" AND automation AND manufacturing)
TITLE-ABS-KEY("machine learning" AND robotics AND production)

APPENDIX

B

MACHINE LEARNING METHOD DICTIONARY

Table B.1 lists the dictionary of machine learning methods used for method extraction in the article retrieval pipeline, as described in Section 4.5.3. The dictionary was built in the pre-master project (Bergstø 2026) and is reused unchanged in MLguide. Each method is linked to a set of phrasings, and a method is recorded for an article when any of its phrasings is found in the cleaned abstract using word-boundary matching.

Table B.1: Machine learning method dictionary used for method extraction, listing each method and the phrasings matched against article abstracts.

Method	**Phrasings**
Principal Component Analysis	principal component analysis, pca
Hotelling T2	hotelling's t2, hotelling t2
Squared Prediction Error	squared prediction error, spe
Independent Component Analysis	independent component analysis, ica
Partial Least Squares	partial least squares, pls, pls regression
Linear Regression	linear regression, lr
Logistic Regression	logistic regression
Naive Bayes	naive bayes, nb
Linear Discriminant Analysis	linear discriminant analysis, lda
Quadratic Discriminant Analysis	quadratic discriminant analysis, qda
Elastic Net	elastic net
Ridge Regression	ridge regression
Lasso Regression	lasso regression, lasso
Bayesian Method	bayesian, bayesian inference, bayesian network, bayesian networks
Robust Regression	robust regression, huber regression, theil sen

Continued on next page

Table B.1 – Continued from previous page

Method	Phrasings
Support Vector Machine	support vector machine, support vector machines, svm
Support Vector Regression	support vector regression, svr
Random Forest	random forest, random forests, rf
Extra Trees	extra trees, extremely randomized trees, extra trees classifier, extra trees regressor
Decision Tree	decision tree, decision trees, dt
K-Nearest Neighbors	k-nearest neighbors, knn, k nn, k-nn
Gaussian Process Regression	gaussian process regression, gpr
Gaussian Mixture Model	gaussian mixture model, gmm
Gradient Boosting	gradient boosting, gb
XGBoost	xgboost, xgb
AdaBoost	adaboost
LightGBM	lightgbm, lgbm
CatBoost	catboost
Bagging	bagging classifier, bagging regressor
Neural Network	neural network, neural networks, artificial neural network, ann
Deep Neural Network	deep neural network, dnn
Residual Network	residual network, resnet
Convolutional Neural Network	convolutional neural network, cnn
UNet	unet
Recurrent Neural Network	recurrent neural network, rnn
Gated Recurrent Unit	gated recurrent unit, gru
LSTM	lstm
Temporal Convolutional Network	temporal convolutional network, tcn
Multi-Layer Perceptron	multi layer perceptron, mlp
Transformers	transformer, transformers
Transformer Encoder	transformer encoder
Attention Mechanism	attention mechanism, attention model
Graph Neural Network	graph neural network, gnn
Graph Convolutional Network	graph convolutional network, gcn
Graph Attention Network	graph attention network, gat
GraphSAGE	graphsage
Autoencoder	autoencoder, auto encoders
Variational Autoencoder	variational autoencoder, vae
Normalizing Flow	normalizing flow, flow-based model
Diffusion Model	diffusion model, ddpm, ddim
Generative Adversarial Network	generative adversarial network, gan, gans
K-Means	k-means, k means, kmeans
K-Medoids	k-medoids, k medoids, pam
Hierarchical Clustering	hierarchical clustering, agglomerative clustering

Continued on next page

Table B.1 – Continued from previous page

Method	Phrasings
DBSCAN	dbscan
OPTICS	optics clustering, optics
Mean Shift	mean shift
Gaussian Mixture Model Clustering	gmm clustering, gaussian mixture clustering
t-SNE	t-sne, tsne
UMAP	umap
ARIMA	arima
SARIMA	sarima
SARIMAX	sarimax
Holt-Winters	holt winters, holt-winters exponential smoothing
Prophet	prophet
Kalman Filter	kalman filter, kalman filtering
State Space Model	state space model
VAR	vector autoregression, var model
N-BEATS	nbeats, n-beats
DeepAR	deepar
Deep Reinforcement Learning	deep reinforcement learning, drl
Q-Learning	q learning, q-learning, qlearning
REINFORCE	reinforce
Deep Q-Network	deep q network, deep q-network, dqn
Policy Gradient	policy gradient, policy-gradient
Actor-Critic	actor critic, actor-critic
A3C	a3c, asynchronous advantage actor critic
Proximal Policy Optimization	proximal policy optimization, ppo
Deep Deterministic Policy Gradient	deep deterministic policy gradient
Twin Delayed DDPG	twin delayed ddpg, td3
Soft Actor-Critic	soft actor critic, soft actor-critic, sac
Isolation Forest	isolation forest
Local Outlier Factor	local outlier factor, lof
One-Class SVM	one-class svm, one class svm
Elliptic Envelope	elliptic envelope
HBOS	hbos, histogram based outlier score
Minimum Covariance Determinant	minimum covariance determinant
Matrix Profile	matrix profile
Genetic Algorithm	genetic algorithm, ga
Particle Swarm Optimization	particle swarm optimization, pso
Simulated Annealing	simulated annealing
Differential Evolution	differential evolution
Ant Colony Optimization	ant colony optimization
Evolutionary Strategy	evolutionary strategy

Continued on next page

Table B.1 – Continued from previous page

Method	**Phrasings**
CMA-ES	cma es, covariance matrix adaptation evolution strategy
Firefly Algorithm	firefly algorithm
Harmony Search	harmony search
SHAP	shap
Explainable AI	xai, explainable ai
Granger Causality	granger causality, granger-test
DoWhy	dowhy
Causal Forest	causal forest
Causal Impact	causal impact
Invariant Causal Prediction	invariant causal prediction

APPENDIX

C

ML AREA KEYWORD LISTS

Table C.1 lists the keyword lists used to assign an ML area to each article, as described in Section 4.5.3. Separate lists are defined for supervised, unsupervised, and reinforcement learning.

Table C.1: Keyword lists used to assign an ML area to each article, with one list per ML area.

Supervised	**Unsupervised**	**Reinforcement**
classification	unsupervised	reinforcement learning
regression	clustering	rl
supervised	cluster	agent
labeled data	kmeans	policy
ground truth	k-means	reward
random forest	hierarchical clustering	q-learning
xgboost	pca	q learning
gradient boosting	principal component analysis	dqn
svm	ica	deep q network
support vector machine	autoencoder	actor-critic
decision tree	auto-encoder	markov decision process
logistic regression	gmm	mdp
knn	gaussian mixture	environment interaction
k-nearest neighbors	dimensionality reduction	policy optimization
naive bayes	anomaly detection	policy gradient
classification model	outlier detection	ppo
regression model	self-organizing map	proximal policy optimization
predictive model	som	a2c
classifier	dbscan	advantage actor-critic
regressor	density-based clustering	sarsa
labelled dataset	spectral clustering	temporal-difference learning
annotated dataset	t-sne	td-learning
supervised training	umap	value iteration
supervised classifier	latent variable model	policy iteration
linear regression	latent representation	credit assignment
ridge regression	representation learning	exploration exploitation
lasso regression	embedding space	exploration-exploitation
ensemble learning	autoencoding	epsilon-greedy
bagging	variational autoencoder	multi-armed bandit
boosting	vae	contextual bandit
gbm	deep clustering	rl agent
svc	deep embedding clustering	reward function
decision forest	subspace clustering	action-value function
random-forest	topic modeling	state-value function
mlp classifier	lda topic model	state-action space
multilayer perceptron	latent dirichlet allocation	decision process
training labels		sequential decision making
target variable		online decision making
supervised feature selection		
cross-entropy loss		
classification accuracy		